



RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE



Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

UNIVERSITÉ DES SCIENCES ET DE LA TECHNOLOGIE HOUARI BOUMEDIENE

Faculté et d'Informatique

Département Informatique

MÉMOIRE DE MASTER

Spécialité

RÉSEAU ET SYSTÈMES DISTRIBUÉS

Thème

Conception et déploiement d'une infrastructure
Cloud privée hyperconvergée basée sur
des technologies open source

Proposé et Dirigé par :

Mr. Neffah Mohammed

Prof. MERAZKA Fatiha

Réalisé et présenté par :

Mr. Bouzara Zakaria

Mr. Ouacherine Ilyes

Devant le jury composé de :

M

M

Soutenu le : ../../2026

Binôme N° 18/2026

Remerciements

Votre texte ici...

Table des matières

Table des matières	4
Table des figures	7
Liste des tableaux	8
Liste des abréviations	10
Introduction générale	1
I Présentation du Contexte	2
I.1 Introduction	3
I.2 Présentation de Sonatrach	3
I.3 Infrastructure Existante et Contexte Technologique	3
I.3.1 Infrastructure actuelle de Sonatrach	3
I.3.2 Les besoins en infrastructure légère pour les zones de test	4
I.4 Problématique	5
I.5 Objectifs du Projet	5
I.5.1 Objectif général	5
I.5.2 Objectifs spécifiques	5
I.6 Périmètre et Limitations du Projet	6
I.7 Conclusion	7
II État de l'Art et Concepts Théoriques	8
II.1 Introduction	9
II.2 L'Hyperconvergence (HCI)	9
II.2.1 Définition	9
II.2.2 Avantages de l'hyperconvergence	9
II.2.3 HCI vs Infrastructure Traditionnelle	10
II.3 Comparaison : Solutions Propriétaires vs Open Source	10
II.3.1 Solutions propriétaires	10
II.3.2 Solutions open source	10
II.4 Analyse des Solutions Open Source HCI	11
II.4.1 OpenStack	11
II.4.2 Harvester HCI	11
II.4.3 Proxmox VE	11
II.4.4 Tableau comparatif	12
II.5 Présentation des Technologies Retenues	12
II.5.1 Proxmox VE : L'hyperviseur	12
II.5.2 Ceph : Le stockage distribué	16
II.5.3 Open vSwitch (OVS) : La virtualisation réseau	19
II.6 L'Équilibrage Dynamique de Charge dans les Clusters Virtualisés	21

II.6.1	Problématique du Déséquilibre de Charge	22
II.6.2	Mécanismes d'Équilibrage dans les Plateformes Existantes	22
II.6.3	L'Algorithme CSLB : Base Théorique du CLBS	22
II.7	Outils Complémentaires d'Infrastructure	23
II.7.1	HAProxy : Équilibrage de Charge Applicatif	23
II.7.2	Stack de Supervision : InfluxDB, Prometheus et Grafana	23
II.7.3	Ansible : Automatisation d'Infrastructure	23
II.7.4	PXE et TFTP : Provisionnement Réseau	24
II.7.5	OPNsense : Pare-feu et Sécurité Périmétrique	24
II.7.6	NTP : Synchronisation Temporelle	24
II.7.7	GitLab CE : Intégration et Déploiement Continu	24
II.8	Déploiement sur Infrastructure Physique Distribuée	25
II.9	Conclusion	25
III	Présentation du Contexte	26
III.1	Introduction	27
III.2	Vue Globale de l'Architecture	27
III.3	Organisation de la Solution	28
III.4	Partie I : Calcul	29
III.4.1	Infrastructure Physique	29
III.4.2	Architecture du Cluster Proxmox	30
III.4.2.1	Organisation du Cluster	30
III.4.2.2	Mécanisme de Quorum et Corosync	30
III.4.2.3	Haute Disponibilité	31
III.4.2.4	Matrice des Scénarios de Défaillance	31
III.4.3	Mécanisme d'Équilibrage Dynamique : CLBS	31
III.4.3.1	Contexte et Besoin	32
III.4.3.2	Base Algorithmique : CSLB	32
III.4.3.3	Modifications Apportées au CSLB	32
III.4.3.4	Algorithme de Décision (Pseudocode)	33
III.4.4	Application de Démonstration : Sona-Web	35
III.4.4.1	Architecture Applicative	35
III.4.4.2	Scénario de Validation HA	36
III.4.5	Architecture du Monitoring	37
III.4.6	Automatisation	37
III.4.6.1	Ansible : Périmètre et Playbooks	39
III.4.6.2	Pipeline PXE/TFTP	39
III.5	Partie II : Stockage	40
III.5.1	Architecture du Cluster Ceph	40
III.5.1.1	Composants du Cluster	40
III.5.1.2	Dimensionnement du Pool	41
III.5.1.3	Configuration CRUSH et Domaines de Défaillance	41
III.6	Partie III : Réseau	42
III.6.1	Contexte et Contraintes Réseau	42
III.6.2	Topologie Réseau	42
III.6.2.1	Deux Niveaux de Segmentation	42
III.6.2.2	Bridges Open vSwitch par Nœud	44
III.6.3	Plan d'Adressage	44
III.6.4	VM OPNsense : Routeur et Pare-feu Centralisé	45
III.6.4.1	Interfaces et Rôles	45

III.6.4.2	Haute Disponibilité d'OPNsense	45
III.6.5	Architecture du Pare-feu	45
III.6.5.1	Pare-feu OPNsense : Filtrage Inter-réseaux	45
III.6.5.2	Pare-feu Proxmox : Filtrage au Niveau Hyperviseur	46
III.7	Services Complémentaires d'Infrastructure	47
III.7.1	Système de Notification : Grafana Alertmanager	47
III.7.2	Centralisation des Journaux : Loki, Alloy et Grafana	48
III.7.3	Synchronisation Temporelle : Serveur NTP	49
III.7.4	Gestion des Identités : OpenLDAP	50
III.7.5	Gestion du Code Source et CI/CD : GitLab	50
III.7.6	Automatisation Opérationnelle : Rundeck	51
III.8	Conclusion	51
IV	Implémentation de la Solution Proposée	52
IV.1	Introduction	53
IV.2	Partie I – Couche Calcul	53
IV.2.1	Préparation de l'Infrastructure Physique	53
IV.2.1.1	Installation de Proxmox VE sur les nœuds	53
IV.2.1.2	Configuration de la connectivité Internet via le laptop gateway	54
IV.2.2	Formation du cluster Proxmox et haute disponibilité	56
IV.2.2.1	Création et jonction au cluster	56
IV.2.2.2	Configuration de la migration à chaud	57
IV.2.3	Déploiement de la stack de monitoring	59
IV.2.3.1	Déploiement d'InfluxDB	59
IV.2.3.2	Déploiement de Prometheus	59
IV.2.3.3	Déploiement de Grafana	61
IV.2.4	Implémentation du mécanisme CLBS	64
IV.2.4.1	Structure du service	65
IV.2.4.2	Logique de décision	65
IV.2.4.3	Déploiement comme service systemd	65
IV.2.4.4	Validation du mécanisme	66
IV.2.5	Déploiement de l'application Sona-Web	67
IV.2.5.1	Création des machines virtuelles	67
IV.2.5.2	Validation de l'application	68
IV.2.5.3	Résultats de l'application Sona-Web	68
IV.2.6	Automatisation avec Ansible	69
IV.2.6.1	Environnement et structure du projet	69
IV.2.6.2	Inventaire	70
IV.2.6.3	Playbook de provisionnement	71
IV.2.6.4	Provisionnement en lot et pipeline PXE/TFTP	72
IV.3	Partie II — Couche Stockage	74
IV.3.1	Déploiement du cluster Ceph	74
IV.3.1.1	Initialisation de Ceph	74
IV.3.1.2	Déploiement des MON et MGR	75
IV.3.1.3	Déploiement des OSDs	77
IV.3.1.4	Création du pool de stockage	77
IV.3.2	Validation de la tolérance aux pannes	79
IV.4	Partie III — Couche Réseau	79
IV.4.1	Configuration Open vSwitch et tunnels VxLAN	79
IV.4.1.1	Installation d'Open vSwitch	79

IV.4.1.2	Configuration complète des interfaces sur PVE-01	80
IV.4.2	Déploiement de la VM OPNsense	85
IV.4.2.1	Création et interfaces réseau	85
IV.4.2.2	NAT sortant	86
IV.4.3	Configuration du service DHCP	87
IV.4.4	Configuration du pare-feu Proxmox	87
IV.4.4.1	Architecture à trois niveaux	87
IV.4.4.2	Configuration au niveau datacenter	87
IV.4.4.3	Configuration au niveau nœud	91
IV.4.4.4	Règles au niveau des VMs	94
IV.4.5	Validation de la haute disponibilité réseau et applicative	95
IV.5	Partie IV – Services Complémentaires d’Infrastructure	95
IV.5.1	Synchronisation Temporelle : NTP/Chrony	96
IV.5.1.1	Installation et configuration du serveur Chrony	96
IV.5.1.2	Validation de la synchronisation	96
IV.5.2	Centralisation des Journaux : Loki et Alloy	97
IV.5.2.1	Déploiement de Loki	97
IV.5.2.2	Déploiement de l’agent Grafana Alloy	97
IV.5.2.3	Validation	98
IV.5.3	Système de Notification : Alertmanager	98
IV.5.3.1	Configuration des règles d’alerte	98
IV.5.3.2	Configuration du contact point Gmail	98
IV.5.3.3	Validation	99
IV.5.4	Gestion du Code Source et CI/CD : GitLab	99
IV.5.4.1	Déploiement du LXC Turnkey GitLab CE	99
IV.5.4.2	Création des dépôts	100
IV.5.4.3	Configuration des pipelines CI/CD	100
IV.5.4.4	Validation	100
IV.5.5	Automatisation Opérationnelle : Rundeck	100
IV.5.5.1	Déploiement du LXC Rundeck	100
IV.5.5.2	Connexion au dépôt ansible-infra	101
IV.5.5.3	Validation	101
IV.6	Conclusion	101

Bibliographie

103

Table des figures

II.1	Pile de virtualisation KVM + QEMU dans Proxmox VE	14
III.1	Architecture Globale de l'Infrastructure HCI	28
III.2	Organisation de la Solution en Trois Parties et leurs Interactions	29
III.3	Pseudocode de l'algorithme CLBS	33
III.4	Architecture Applicative Sona-Web	36
III.5	Vérification de la répartition des requêtes en base	36
III.6	Pipeline d'Automatisation : PXE/TFTP et Ansible	38
III.7	Topologie Réseau et Plan VxLAN	43
III.8	Architecture du Pare-feu : OPNsense et Proxmox Firewall	46
III.9	Diagramme de déploiement : répartition des services sur les trois nœuds physiques	47
III.10	Pipeline de notification Grafana Alertmanager	48
III.11	Architecture de centralisation des journaux : Alloy, Loki et Grafana	49
III.12	Hierarchie NTP : synchronisation temporelle de l'infrastructure	50
III.13	Pipeline CI/CD GitLab pour les projets <code>clbs</code> et <code>ansible-infra</code>	51
IV.1	Interface web Proxmox au premier login sur PVE-01	54
IV.2	Activation du routage IP sur le laptop	54
IV.3	Configuration NAT avec iptables	55
IV.4	Persistance des règles iptables	55
IV.5	Configuration réseau de base d'un nœud Proxmox	55
IV.6	Ajout des routes statiques vers OPNsense	55
IV.7	Création du cluster Proxmox	56
IV.8	Jonction de PVE-02 et PVE-03 au cluster	56
IV.9	Vérification de l'état du cluster	56
IV.10	Vue du cluster Proxmox montrant les trois nœuds en ligne	57
IV.11	Avant la migration live d'une machine virtuelle	58
IV.12	Après la migration live d'une machine virtuelle	58
IV.13	Explorateur InfluxDB montrant les métriques reçues depuis Proxmox	59
IV.14	Extrait du fichier <code>prometheus.yml</code>	60
IV.15	Interface Prometheus montrant les cibles Ceph et CLBS avec l'état UP	61
IV.16	Dashboard Grafana Infrastructure pour les nœuds Proxmox	63
IV.17	Dashboard Grafana Ceph affichant l'état du cluster, des pools et la latence des OSDs	64
IV.18	Gestion du service CLBS avec <code>systemd</code>	65
IV.19	Journaux <code>systemd</code> du service CLBS lors d'un cycle de détection et de migration	66
IV.20	Génération d'une charge CPU de test	66
IV.21	Dashboard CLBS montrant une migration déclenchée automatiquement	67
IV.22	Test de répartition HAProxy	68
IV.23	Interface principale de l'application Sona-Web	69

IV.24	Structure du projet Ansible	70
IV.25	Fichier ansible.cfg	70
IV.26	Extrait de l'inventaire hosts.yml	70
IV.27	Exemple d'utilisation du playbook provision_vm.yml	71
IV.28	Formule d'attribution automatique de l'adresse IP	71
IV.29	Configuration cloud-init d'une VM Proxmox	71
IV.30	Options DHCP pour le boot PXE	73
IV.31	Installation et arborescence du serveur TFTP	73
IV.32	Menu de boot PXE	74
IV.33	Initialisation du cluster Ceph	75
IV.34	Création d'un monitor Ceph	75
IV.35	Interface Proxmox affichant les monitors et managers Ceph actifs sur les trois nœuds	76
IV.36	Activation du module Prometheus pour Ceph	76
IV.37	Création d'un OSD Ceph	77
IV.38	Affichage de l'arbre CRUSH	77
IV.39	Création du pool Ceph	77
IV.40	État global du cluster Ceph	77
IV.41	État du cluster Ceph montrant les MON et OSD	78
IV.42	État du cluster Ceph montrant le pool RBD	78
IV.43	Installation d'Open vSwitch	79
IV.44	Création du bridge OVS principal	79
IV.45	Vue d'ensemble de la configuration réseau de PVE-01 — Quatre parties logiques	80
IV.46	Interfaces de base et bridge OVS de management	81
IV.47	VxLAN Corosync (clé 20) et Ceph interne (clé 30)	81
IV.48	Ceph public (clé 40), migration (clé 50) et sauvegarde (clé 90)	82
IV.49	VxLAN applicatifs (clés 60, 70, 80) et bridges Linux	84
IV.50	Configuration des interfaces de la VM OPNsense	86
IV.51	Capture du trafic DHCP avec tcpdump	87
IV.52	Aliases définis au niveau datacenter dans le pare-feu Proxmox	88
IV.53	IPSets configurés au niveau datacenter regroupant les adresses par domaine	89
IV.54	Groupes de sécurité Proxmox avec leurs règles de ports par service	90
IV.55	Règles de pare-feu au niveau nœud PVE-01 dans l'interface web Proxmox	91
IV.56	Vérification des règles iptables actives générées par Proxmox sur PVE-01	92
IV.57	Chaîne iptables PVEFW-HOST-IN active sur PVE-01 (vérification SSH)	93
IV.58	Vérification des IPSets actifs sur PVE-01	93
IV.59	IPSets actifs sur PVE-01 vérifiés via SSH (<code>ipset list</code>)	94
IV.60	Configuration du serveur Chrony	96
IV.61	Activation du service Chrony	96
IV.62	Sources NTP actives sur le serveur Chrony interne [CAPTURE : se connecter en SSH au LXC Chrony, exécuter la commande <code>chronyc sources -v</code> , capturer la sortie montrant les serveurs pool.ntp.org avec l'étoile (*) indiquant la source active]	97
IV.63	Clients NTP synchronisés sur le serveur Chrony interne [CAPTURE : sur le même LXC Chrony, exécuter <code>chronyc -n clients</code> , capturer la sortie listant les adresses IP des nœuds Proxmox (192.168.10.100, .213, .84) et des VMs avec leur dernier accès NTP]	97
IV.64	Installation de Loki	97
IV.65	Configuration de l'agent Alloy pour la collecte des journaux	97

IV.66	Statut de l'agent Grafana Alloy sur PVE-01 [CAPTURE : SSH sur PVE-01, exécuter <code>systemctl status alloy</code> , capturer la sortie montrant le service en état <code>active (running)</code>]	98
IV.67	Journaux de l'infrastructure visualisés dans Grafana via LogQL [CAPTURE : ouvrir Grafana dans le navigateur, aller dans Explore, sélectionner la datasource Loki, saisir la requête <code>{job="varlogs"}</code> , capturer la page affichant des lignes de logs réels avec leurs labels (host, job, filename visibles dans le panneau)]	98
IV.68	Paramètres SMTP du contact point Gmail	99
IV.69	Règles d'alerte configurées dans Grafana Alerting [CAPTURE : ouvrir Grafana, aller dans Alerting -> Alert Rules, capturer la liste des règles définies (ex : CPU nœud, santé Ceph) avec leur état affiché (Normal / Pending / Firing)]	99
IV.70	Contact point Gmail configuré dans Grafana pour les notifications [CAPTURE : dans Grafana -> Alerting -> Contact Points, capturer la configuration du contact point Gmail avec l'adresse email visible (masquer le mot de passe SMTP si affiché)]	99
IV.71	Page Projects de l'instance GitLab avec les dépôts clbs et ansible-infra [CAPTURE : ouvrir l'interface web GitLab dans le navigateur, aller sur la page principale des projets, capturer la liste affichant les deux projets avec leur dernière activité (dernier commit)]	100
IV.72	Pipeline CI/CD réussi pour le projet clbs dans GitLab [CAPTURE : dans GitLab -> projet clbs -> CI/CD -> Pipelines, capturer un pipeline récent avec ses stages (Lint, Tests, Build) tous affichés en vert (passed), montrant le numéro de pipeline et le commit associé]	100
IV.73	Installation de Rundeck	100
IV.74	Interface Rundeck affichant les jobs de provisionnement Ansible [CAPTURE : ouvrir l'interface web Rundeck (port 4440), aller dans Jobs, capturer la liste des jobs configurés (provisionnement VM, intégration nœud, etc.) avec leurs noms visibles]	101
IV.75	Résultat d'exécution d'un job Rundeck avec le log Ansible [CAPTURE : dans Rundeck -> Activity, ouvrir le détail d'une exécution récente, capturer la page affichant le statut <code>Succeeded</code> et les tâches Ansible déroulées avec leurs états (ok, changed, skipped)]	101

Liste des tableaux

III.1 Dimensionnement physique des serveurs Dell PowerEdge R720	30
III.2 Adresses IP de management des nœuds Proxmox	30
III.3 Matrice des scénarios de défaillance et comportements attendus	31
III.4 Répartition du stockage Ceph par usage	41
III.5 Paramètres CRUSH appliqués automatiquement par Proxmox	41
III.6 Bridges OVS et tunnels VxLAN par nœud Proxmox	44
III.7 Adressage IP statique des nœuds Proxmox par réseau logique	44
IV.1 Dashboards Grafana configurés	62
IV.2 Comparaison des protocoles utilisés dans le pipeline PXE	72
IV.3 Interfaces de la VM OPNsense	86
IV.4 Plages DHCP configurées sur OPNsense	87
IV.5 Règles de pare-feu au niveau nœud (appliquées sur PVE-01, PVE-02, PVE-03)	92
IV.6 Règles de pare-feu par VM applicative	95

Liste des abréviations

API	: Application Programming Interface
BSD	: Berkeley Software Distribution
CD	: Continuous Deployment
CI	: Continuous Integration
CLI	: Command Line Interface
CPU	: Central Processing Unit
CRUSH	: Controlled Replication Under Scalable Hashing
DHCP	: Dynamic Host Configuration Protocol
DNS	: Domain Name System
DRS	: Distributed Resource Scheduler
EC	: Erasure Coding
GUI	: Graphical User Interface
HA	: High Availability
HCI	: Hyper Converged Infrastructure
HTTP	: HyperText Transfer Protocol
I/O	: Input / Output
IP	: Internet Protocol
ISO	: International Organization for Standardization (image disque)
KPI	: Key Performance Indicator
KVM	: Kernel-based Virtual Machine
LDAP	: Lightweight Directory Access Protocol
LXC	: Linux Containers
MDS	: MetaData Server
MGR	: Manager
MON	: Monitor
NAT	: Network Address Translation
NIC	: Network Interface Card
NTP	: Network Time Protocol
OSD	: Object Storage Daemon
OVS	: Open Virtual Switch
PBS	: Proxmox Backup Server
PC	: Personal Computer
PG	: Placement Group
PVE	: Proxmox Virtual Environment
PXE	: Preboot eXecution Environment
QoS	: Quality of Service
RADOS	: Reliable Autonomic Distributed Object Store
RAM	: Random Access Memory
RBD	: RADOS Block Device
RSD	: Réseau Et Systèmes Distribués
SMTP	: Simple Mail Transfer Protocol
SPOF	: Single Point Of Failure
SSH	: Secure Shell
TCO	: Total Cost of Ownership
TCP	: Transmission Control Protocol
TFTP	: Trivial File Transfer Protocol
UDP	: User Datagram Protocol
vCPU	: Virtual Central Processing Unit
VE	: Virtual Environment
VLAN	: Virtual Local Area Network
VM	: Virtual Machine
VxLAN	: Virtual eXtensible LAN
WAL	: Write-Ahead Log

Introduction générale

Chapitre I

Présentation du Contexte

I.1 Introduction

Ce chapitre vise à présenter le contexte général dans lequel s'inscrit notre projet de fin d'études. Nous y aborderons les fondements de l'hyper convergence (HCI), les enjeux liés à la virtualisation et à la gestion unifiée des ressources informatiques, ainsi que les besoins spécifiques de l'entreprise Sonatrach. Ce cadre introductif nous permettra de mieux comprendre les choix technologiques effectués et les objectifs fixés pour le déploiement d'une infrastructure Cloud privée hyperconvergente basée sur des solutions open source.

I.2 Présentation de Sonatrach

Sonatrach est une entreprise publique algérienne créée le 31 décembre 1963, peu après l'indépendance de l'Algérie. Elle a été fondée dans le but de gérer, développer et exploiter les ressources hydrocarbures du pays, qui représentent une richesse stratégique pour l'économie nationale. À ses débuts, Sonatrach se consacrait essentiellement au transport et à la commercialisation du pétrole brut extrait en Algérie [1].

Avec le temps, Sonatrach a évolué pour devenir la plus grande entreprise en Afrique dans le domaine des hydrocarbures. Elle intervient aujourd'hui dans l'ensemble de la chaîne de valeur pétrolière et gazière : exploration, production, transport, transformation et commercialisation. L'entreprise est également active dans la pétrochimie et les énergies renouvelables.

Sonatrach entretient de nombreuses collaborations internationales avec des compagnies pétrolières et gazières étrangères, impliquées dans de grands projets de partenariat via des joint-ventures et accords technologiques avec des entreprises européennes, américaines, asiatiques et africaines. Ces coopérations ont permis un transfert de savoir-faire et de technologie vers l'Algérie.

Le projet est réalisé au sein du Centre d'innovation de Sonatrach. Cette direction joue un rôle stratégique dans l'innovation technologique et l'optimisation des processus internes. Elle dispose d'infrastructures informatiques avancées, notamment des solutions de virtualisation et de stockage haute capacité, destinées à soutenir les travaux de simulation, de test et de pré-production.

I.3 Infrastructure Existante et Contexte Technologique

I.3.1 Infrastructure actuelle de Sonatrach

Sonatrach dispose actuellement d'une infrastructure hyperconvergente basée sur la solution propriétaire Nutanix, déployée à l'échelle nationale sur plusieurs wilayas et utilisée pour les environnements de production critiques. Cette solution est reconnue pour sa stabilité, ses performances et la richesse de ses fonctionnalités, notamment en matière de gestion centralisée, de haute disponibilité et de scalabilité.

Cependant, cette infrastructure présente des contraintes structurelles qui deviennent de plus en plus préoccupantes dans une perspective d'évolution à long terme :

- **Coûts de licences croissants** : les frais de licence Nutanix suivent une trajectoire haussière, représentant une charge financière significative et récurrente pour l'entreprise, indépendamment du niveau d'utilisation réel de la plateforme.
- **Dépendance technologique forte** : le recours exclusif à un éditeur propriétaire unique expose Sonatrach à un risque de 'vendor lock-in', limitant sa capacité à négocier les conditions contractuelles et à adopter librement de nouvelles technologies.
- **Manque de souveraineté technique** : les évolutions de la plateforme, les mises à jour et les décisions d'architecture sont entièrement soumises à l'orientation technique de l'éditeur, sans possibilité d'adaptation aux besoins spécifiques de l'entreprise.
- **Inadaptation aux petits déploiements** : pour les environnements de test, de développement et de pré-production, le déploiement de Nutanix sur de petits clusters de 3 à 4 nœuds est disproportionné par rapport aux besoins réels, tant en complexité qu'en coût.

Ces constats positionnent ce projet non pas comme un simple exercice académique, mais comme une réponse concrète à une problématique stratégique réelle : explorer et valider la viabilité d'une infrastructure hyperconvergée open source capable de répondre aux exigences de Sonatrach, aujourd'hui en environnement de test et demain potentiellement en production.

I.3.2 Les besoins en infrastructure légère pour les zones de test

Au-delà des besoins immédiats en environnements de test et de pré-production, Sonatrach a besoin d'une infrastructure qui réponde à des exigences de niveau entreprise, tout en s'affranchissant des contraintes des solutions propriétaires. Ces besoins couvrent plusieurs dimensions :

- **Fiabilité et haute disponibilité** : l'infrastructure doit garantir la continuité de service en cas de défaillance d'un ou plusieurs nœuds, avec basculement automatique des charges de travail sans intervention humaine.
- **Performance et scalabilité** : la solution doit pouvoir monter en charge progressivement, en ajoutant des nœuds au cluster sans interruption de service, pour accompagner la croissance des besoins de l'entreprise.
- **Gestion unifiée des ressources** : calcul, stockage et réseau doivent être administrés depuis une interface centralisée unique, réduisant la complexité opérationnelle et les besoins en compétences spécialisées multiples.
- **Équilibrage dynamique de charge** : la plateforme doit être capable de redistribuer automatiquement les charges de travail entre les nœuds en fonction de l'utilisation des ressources, optimisant ainsi l'efficacité globale du cluster.
- **Souveraineté et indépendance technologique** : en s'appuyant exclusivement sur des technologies open source, l'entreprise reprend le contrôle total de son infrastructure, avec la liberté d'auditer, de modifier et d'adapter la solution à ses besoins propres.
- **Réduction du coût total de possession (TCO)** : l'absence de licences propriétaires représente une économie substantielle sur le long terme, réallouable vers des investissements matériels ou humains.

I.4 Problématique

La problématique centrale de ce projet est la suivante :

Comment concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement basée sur des technologies open source, capable de répondre aux exigences de fiabilité, de performance et de scalabilité d’une grande entreprise industrielle, tout en constituant une alternative crédible et souveraine aux solutions propriétaires de type Nutanix ?

Cette problématique soulève plusieurs questions techniques structurantes :

- Comment concevoir un système de stockage distribué, redondant et performant garantissant l’intégrité des données en cas de défaillance d’un nœud ?
- Comment assurer le basculement et l’équilibrage automatique des charges de travail dans un cluster à plusieurs nœuds, sans perte de données ni interruption prolongée de service ?
- Comment segmenter et isoler strictement les flux réseau au sein d’une infrastructure hyperconvergée, en séparant les trafics de gestion, de réplication, de migration et de production ?
- Dans quelle mesure une solution open source peut-elle répondre aux exigences d’une infrastructure de niveau production dans le contexte d’une grande entreprise ?

I.5 Objectifs du Projet

I.5.1 Objectif général

Concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement open source, sur trois serveurs physiques Dell PowerEdge R720 mis à disposition par Sonatrach, offrant des garanties de fiabilité, de performance et d’évolutivité comparables aux solutions propriétaires de type Nutanix, et constituant une alternative stratégique à moyen et long terme.

I.5.2 Objectifs spécifiques

- **Déployer un cluster de trois nœuds physiques à haute disponibilité** sur les serveurs Dell PowerEdge R720 fournis par Sonatrach, avec gestion centralisée, administration simplifiée et support de la migration à chaud.
- **Configurer Proxmox VE sur chaque serveur physique** en exploitant pleinement les ressources matérielles disponibles (256 Go de RAM, 5 disques SAS par nœud).
- **Concevoir et mettre en place un système de stockage distribué Ceph** en s’appuyant sur les disques SAS 600 Go de chaque nœud pour le stockage distribué, et les disques 300 Go pour le système d’exploitation.

- **Implémenter une segmentation réseau avancée** via Open vSwitch, en configurant une architecture multi-VLAN séparant les flux de gestion, de réplication, de migration et de production sur l'infrastructure physique réelle.
- **Développer un mécanisme d'équilibrage dynamique de la charge**, capable de redistribuer automatiquement les machines virtuelles selon les métriques de performance en temps réel.
- **Valider la solution** à travers un scénario de haute disponibilité démontrant la tolérance aux pannes sans perte de données ni interruption de service.
- **Fournir une base d'infrastructure évolutive**, accompagnée d'une documentation technique et d'une conception architecturale réutilisable à plus grande échelle au sein de Sonatrach.

I.6 Périmètre et Limitations du Projet

Ce projet vise le déploiement d'un cluster hyperconvergé open source destiné aux environnements de test, de développement et de pré-production. L'infrastructure est déployée sur un cluster physique de trois serveurs Dell PowerEdge R720 mis à disposition par Sonatrach, offrant les ressources matérielles nécessaires à l'exécution de l'hyperviseur Proxmox VE directement sur le matériel physique, du stockage distribué Ceph et des machines virtuelles de démonstration.

Le passage à une architecture physique distribuée lève la contrainte majeure du point de défaillance unique (SPOF) matériel. Néanmoins, le périmètre de ce projet se limite volontairement à un environnement de laboratoire structuré, ce qui implique les contraintes suivantes :

- **Capacité de stockage** : La taille du pool de stockage distribué est techniquement contrainte par les disques physiques alloués au stockage (3 disques de 600 Go par serveur). Avec une réplication de facteur 3, le cluster offre environ 1,8 To d'espace utilisable. Cela rend cette infrastructure parfaitement adaptée aux charges de développement et de validation, mais insuffisante pour des charges de données à très grande échelle.
- **Limite de configuration du commutateur** : la segmentation réseau avancée repose sur une configuration précise des commutateurs physiques (affectation des ports aux VLANs, mode trunk). Cette configuration est du ressort exclusif de l'équipe réseau de Sonatrach et ne peut être modifiée dans le cadre de ce projet.
- **Absence de multilocation** : Proxmox VE, dans la configuration mise en œuvre, est orienté Cloud privé. La gestion multilocataire stricte (isolation forte entre différents clients ou départements totalement hermétiques), propre aux environnements Cloud publics, n'entre pas dans le périmètre de ce projet.
- **Maintenance à long terme** : La gestion opérationnelle continue de l'infrastructure (cycle de mises à jour, configuration d'un serveur de sauvegarde externe sécurisé, gestion des pannes matérielles physiques) n'est pas couverte par la phase de déploiement de ce projet et constitue une perspective d'évolution naturelle pour l'équipe d'exploitation.

I.7 Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet, les besoins identifiés au sein de Sonatrach, et la problématique qui motive notre démarche. Nous avons défini les objectifs, le périmètre et les limites du projet. Ce cadre nous permettra d'aborder, dans le chapitre suivant, l'état de l'art des solutions disponibles et les fondements théoriques de l'hyperconvergence, de la virtualisation et des technologies retenues.

Chapitre II

État de l'Art et Concepts Théoriques

II.1 Introduction

Dans un contexte de transformation numérique accélérée, les organisations scientifiques et industrielles cherchent des solutions d'infrastructure à la fois performantes, flexibles et économiques. Ce chapitre présente les concepts théoriques fondamentaux sur lesquels repose notre projet : l'hyperconvergence, la virtualisation, le stockage distribué et la virtualisation du réseau. Il dresse également un état de l'art comparatif des solutions disponibles, afin de justifier les choix technologiques retenus.

II.2 L'Hyperconvergence (HCI)

II.2.1 Définition

L'hyperconvergence (HCI, Hyper-Converged Infrastructure) est une architecture informatique qui regroupe, dans un même système, les ressources de calcul, de stockage et de réseau. Ces ressources sont pilotées par une couche logicielle centralisée, supprimant ainsi la nécessité de gérer des équipements spécialisés distincts pour chaque fonction [2].

L'infrastructure hyperconvergée repose sur trois composantes fondamentales :

- **Le calcul** : des processeurs virtuels (vCPU) alloués dynamiquement aux machines virtuelles.
- **Le stockage** : un système de stockage distribué logiciel, agrégé à partir des disques locaux de chaque nœud.
- **Le réseau** : un switch virtuel logiciel permettant la segmentation et l'isolation du trafic réseau.

II.2.2 Avantages de l'hyperconvergence

L'approche HCI présente plusieurs avantages par rapport aux architectures traditionnelles à trois niveaux (compute, storage, network séparés) :

- **Simplicité d'administration** : une interface unique pour gérer l'ensemble des ressources.
- **Réduction des coûts matériels** : pas besoin de baies de stockage ou de switches réseau dédiés.
- **Évolutivité horizontale** : l'ajout d'un nœud augmente simultanément la capacité de calcul, de stockage et de réseau.
- **Haute disponibilité native** : la réplication des données entre les nœuds assure la continuité de service en cas de panne.

II.2.3 HCI vs Infrastructure Traditionnelle

Critère	Infrastructure Traditionnelle	Infrastructure HCI
Gestion	Silos séparés (compute, storage, network)	Interface unifiée
Coût initial	Elevé (équipements spécialisés)	Réduit (commodity hardware)
Évolutivité	Verticale (upgrade de matériel)	Horizontale (ajout de nœuds)
Complexité	Haute	Faible
Haute disponibilité	Optionnelle et complexe	Native

Tableau 2-1 : Comparaison du HCI avec Infrastructure Traditionnelle

II.3 Comparaison : Solutions Propriétaires vs Open Source

II.3.1 Solutions propriétaires

Les solutions propriétaires telles que **VMware vSAN** ou **Nutanix** offrent un support professionnel, des fonctionnalités avancées et une interface utilisateur soignée. Cependant, elles présentent plusieurs limites importantes dans notre contexte :

- **Coûts élevés** : les licences et la maintenance annuelle représentent un investissement significatif, difficilement justifiable pour des environnements de test.
- **Dépendance fournisseur (Vendor Lock-in)** : la migration vers une autre solution est souvent complexe et coûteuse.
- **Faible flexibilité** : les environnements propriétaires offrent peu de liberté pour personnaliser ou adapter l'infrastructure.

II.3.2 Solutions open source

Les solutions open source présentent plusieurs avantages stratégiques :

- **Indépendance technologique** : aucune dépendance vis-à-vis d'un éditeur.
- **Absence de coûts de licence** : seuls les coûts d'infrastructure matérielle et de main-d'œuvre sont à prévoir.
- **Transparence et auditabilité** : le code source est accessible et auditable.
- **Communauté active** : documentation abondante et mises à jour régulières.

Dans un contexte de zones de test et de petits déploiements, l'approche open source est clairement la plus adaptée.

II.4 Analyse des Solutions Open Source HCI

Le critère principal de sélection est la légèreté et la capacité à fonctionner de manière stable sur un cluster de 3 nœuds.

II.4.1 OpenStack

OpenStack est une plateforme cloud open source très complète et standardisée, utilisée par de nombreuses grandes entreprises et fournisseurs cloud. Elle propose une large gamme de services : calcul (Nova), stockage objet (Swift), stockage en bloc (Cinder), réseau (Neutron), identité (Keystone), etc.

Cependant, OpenStack présente des inconvénients majeurs pour notre cas d'usage :

- **Complexité de déploiement** : l'installation et la configuration d'OpenStack impliquent l'orchestration de nombreux services interdépendants (Nova, Neutron, Cinder, Keystone, etc.), ce qui rend le déploiement particulièrement long et difficile à maintenir.
- **Consommation de ressources élevée** : les services de base d'OpenStack consomment à eux seuls plusieurs Go de RAM, avant même de démarrer des VMs de production.
- **Inadapté aux petits clusters** : OpenStack est conçu pour des déploiements de grande envergure. Sur 3 nœuds, ses performances et sa stabilité sont compromises.

II.4.2 Harvester HCI

Harvester est une solution HCI open source développée par SUSE, basée sur Kubernetes et KubeVirt. Elle propose une interface moderne et une intégration native avec Rancher pour la gestion de clusters Kubernetes [4].

Ses limites dans notre contexte :

- **Consommation en RAM et CPU élevée** : Harvester nécessite des ressources importantes pour faire fonctionner la couche Kubernetes sous-jacente, avant même d'allouer des ressources aux VMs.
- **Maturité relative pour les VMs classiques** : Harvester est davantage orienté vers les charges de travail conteneurisées. La gestion de VMs Linux/Windows classiques est moins optimisée que sur une solution dédiée [3].

II.4.3 Proxmox VE

Proxmox Virtual Environment (VE) est une plateforme de virtualisation open source basée sur KVM (pour les VMs) et LXC (pour les conteneurs). Elle propose une interface web centralisée, un support natif de Ceph pour le stockage distribué, et une gestion intégrée du clustering et de la haute disponibilité [5].

Ses atouts pour notre cas d'usage sont nombreux :

- **Légèreté** : moins de 1 Go de RAM consommé à vide sur le nœud, laissant l'essentiel des ressources aux VMs.
- **Déploiement rapide** : un cluster de 3 nœuds est opérationnel en quelques heures.
- **Interface web intuitive** : la console Proxmox (accessible sur le port 8006) centralise la gestion des VMs, conteneurs, stockage, réseau et haute disponibilité.
- **Intégration native de Ceph** : Proxmox intègre nativement les outils de gestion Ceph, simplifiant considérablement le déploiement du stockage distribué.
- **Communauté active** : documentation officielle complète et forums très actifs.

II.4.4 Tableau comparatif

Critère	OpenStack	Harvester	Proxmox VE
Légèreté	Faible	Moyenne	Elevée
Facilité de déploiement	Difficile	Moyenne	Facile
Adapté au déploiement à 3 nœuds	Non	Partiel	Oui
Intégration stockage distribué	Via Cinder/Ceph	Via Longhorn	Ceph natif
Gestion des VMs classiques	Bonne	Moyenne	Excellente
Maturité	Très mature	En cours	Mature
Coût de licence	Gratuit	Gratuit	Gratuit (Enterprise payant)

Tableau 2-2 : OpenStack vs Harvester vs Proxmox VE

Proxmox VE est la solution la mieux adaptée au contexte de ce projet : légère, rapide à déployer sur un cluster de 3 nœuds, et capable de s'étendre à de plus grands clusters sans changement d'architecture. Elle sera retenue comme hyperviseur principal. Le choix de la solution de stockage distribué et de la couche réseau est abordé dans les sections suivantes.

II.5 Présentation des Technologies Retenues

II.5.1 Proxmox VE : L'hyperviseur

Présentation générale

Proxmox Virtual Environment (VE) est une plateforme de virtualisation open source basée sur un noyau Linux Debian modifié. Elle supporte deux types de virtualisation : **KVM** (*Kernel-based Virtual Machine*) pour les machines virtuelles complètes, compatibles avec tout système d'exploitation (Linux, Windows, BSD), et **LXC** (*Linux Containers*) pour les conteneurs légers partageant le noyau de l'hôte. L'ensemble est administré via une interface web centralisée accessible sur le port 8006, qui unifie la gestion des VMs, conteneurs, stockage, réseau et haute disponibilité depuis un point d'entrée unique.

La pile de virtualisation : KVM, QEMU et libvirt

La virtualisation des machines dans Proxmox repose sur une pile logicielle à trois niveaux qui collaborent étroitement.

KVM – Kernel-based Virtual Machine

KVM est un module du noyau Linux (chargé via `kvm.ko` et `kvm_intel.ko` ou `kvm_amd.ko` selon le processeur) qui transforme le noyau Linux lui-même en hyperviseur de type 1 [22]. KVM expose au système des périphériques virtuels (`/dev/kvm`) et exploite les extensions matérielles de virtualisation du processeur – **Intel VT-x** ou **AMD-V** – pour exécuter le code des VMs directement sur le CPU physique sans émulation. Cela signifie que les instructions non privilégiées de la VM s'exécutent à la vitesse native du processeur, tandis que les instructions privilégiées (accès mémoire, I/O) sont interceptées et gérées par le noyau hôte. Le résultat est une performance CPU très proche du bare metal, typiquement entre 95% et 99% des performances natives mesurées en benchmarks.

QEMU – Quick EMUlator

QEMU est l'émulateur de machine qui accompagne KVM [23]. Il opère en espace utilisateur et assure plusieurs fonctions essentielles :

- Il **émule les périphériques** de la VM : carte réseau virtuelle (VirtIO, e1000, RTL8139), contrôleur de disque (VirtIO SCSI, IDE, SATA), carte graphique, USB, BIOS ou UEFI (via OVMF). Chaque VM en cours d'exécution correspond à un processus `qemu-system-x86_64` sur l'hôte.
- Il gère le **CPU virtuel (vCPU)** : le nombre de cœurs, le modèle de CPU exposé à la VM (host-passthrough, kvm64, Haswell, Broadwell, etc.), et les flags CPU visibles par le système invité.
- Il assure la **gestion mémoire** : allocation de RAM, support du *Memory Ballooning* (ajustement dynamique de la RAM d'une VM en cours d'exécution), et des *Huge Pages* pour réduire la pression sur le TLB.
- Il fournit les **interfaces de migration** utilisées pour la live migration : QEMU implémente le protocole de transfert de l'état complet d'une VM (mémoire, état CPU, état des périphériques) vers un autre hôte sans interruption de service.

Lorsque KVM est disponible, QEMU délègue l'exécution du code de la VM à KVM et n'émule que les périphériques. C'est cette combinaison KVM+QEMU qui constitue l'hyperviseur réel.

libvirt : couche d'abstraction de la virtualisation Linux

libvirt est une bibliothèque open source et une API unifiée de gestion de la virtualisation sous Linux [24]. Elle offre une interface commune au-dessus de plusieurs hyperviseurs (KVM, QEMU, LXC, Xen) et est utilisée par de nombreux outils de la communauté : `virt-manager`, OpenStack Nova, ou encore oVirt. libvirt simplifie la création, la gestion du cycle de vie et la supervision des VMs en s'affranchissant des spécificités de chaque hyperviseur.

Proxmox VE fait un choix architectural différent : il n'utilise pas libvirt mais sa propre couche de gestion écrite en Perl (`PVE::QemuServer`), qui pilote QEMU directement. Cela lui confère un contrôle plus fin sur les paramètres QEMU et une intégration plus étroite avec l'API REST et les mécanismes de cluster (Corosync, HA, Ceph).

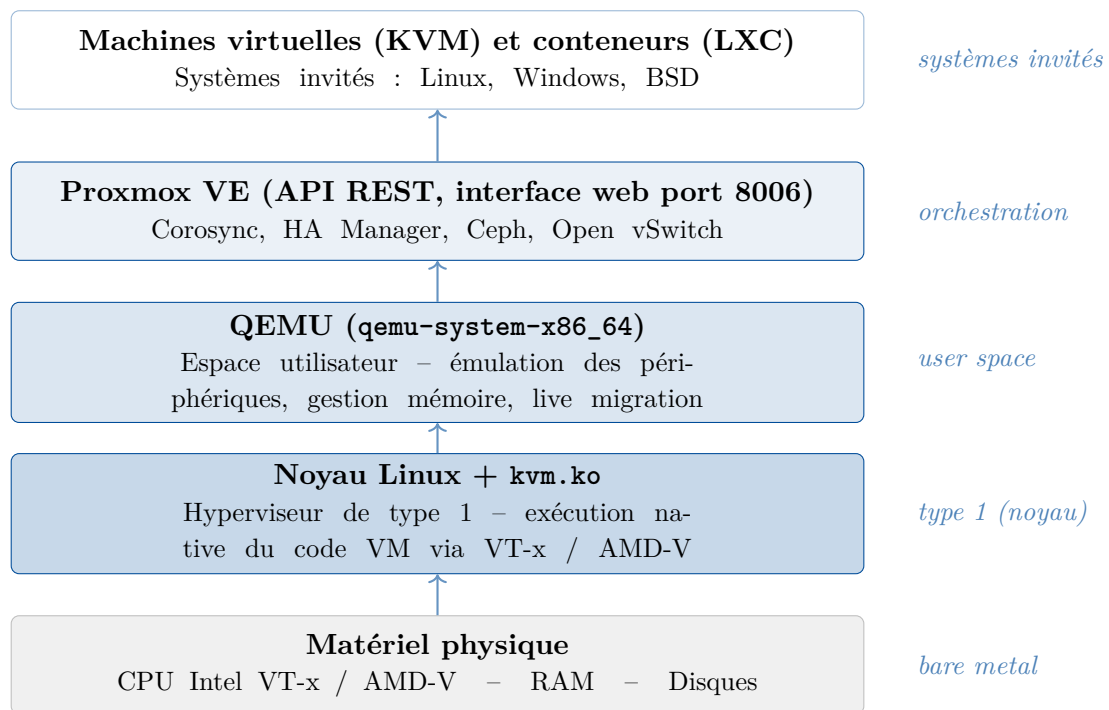


FIGURE II.1 – Pile de virtualisation KVM + QEMU dans Proxmox VE

La figure II.1 illustre les cinq niveaux de la pile de virtualisation Proxmox. KVM, intégré au noyau Linux, exploite les extensions matérielles VT-x et AMD-V pour exécuter le code des VMs directement sur le CPU physique. QEMU opère en espace utilisateur et émule les périphériques. Proxmox VE orchestre l'ensemble via son API REST et unifie la gestion du cluster, du stockage Ceph et du réseau Open vSwitch.

Les conteneurs Linux : LXC

LXC (*Linux Containers*) est une technologie de virtualisation légère reposant sur deux mécanismes natifs du noyau Linux : les **namespaces** et les **cgroups**. Contrairement aux VMs KVM qui émulent un matériel complet avec un noyau invité indépendant, les conteneurs LXC partagent le noyau de l'hôte. Cette approche réduit significativement la consommation de ressources : un conteneur démarre en quelques secondes et son surcoût en mémoire est de l'ordre de quelques dizaines de mégaoctets.

Les **namespaces** assurent l'isolation de chaque conteneur en lui attribuant des espaces de noms séparés pour les processus (PID), les interfaces réseau, les points de montage et les utilisateurs. Les **cgroups** (*control groups*) permettent de limiter et de comptabiliser la consommation de ressources (CPU, mémoire, I/O disque) de chaque conteneur, garantissant qu'aucun conteneur ne peut monopoliser les ressources de l'hôte.

Proxmox VE intègre LXC nativement et propose des modèles de distributions populaires (Debian, Ubuntu, Alpine). LXC est particulièrement adapté pour les services qui ne nécessitent pas de noyau spécifique : agents de supervision, serveurs NTP, services PXE, systèmes d'intégration continue et autres services Linux standards. Cette approche minimise la consommation de ressources par rapport à des machines virtuelles complètes pour ces cas d'usage [10].

Critère	VM KVM	Conteneur LXC
Isolation	Matériel + noyau dédié	Noyau partagé (namespaces)
Temps de démarrage	20 à 60 secondes	1 à 5 secondes
Surcoût RAM	~512 MB	~50 MB
Systèmes supportés	Linux, Windows, BSD	Linux uniquement
Cas d'usage	Applications nécessitant un noyau spécifique	Services Linux standards

Tableau 2-3 : Comparaison VM KVM et conteneur LXC [10]

Gestion du cluster : Corosync et le quorum

La gestion du cluster Proxmox repose sur **Corosync**, un framework de communication de cluster qui assure le consensus entre les nœuds via un mécanisme de *quorum* [14]. Corosync envoie des messages *heartbeat* à intervalles réguliers entre les nœuds, sur un réseau dédié, pour détecter les défaillances. Un cluster Proxmox nécessite un minimum de **3 nœuds** pour garantir un quorum stable.

Le quorum définit le nombre minimum de nœuds qui doivent être en accord pour que le cluster soit considéré comme opérationnel. Avec 3 nœuds, la majorité est de 2 : si un nœud tombe, les 2 restants ont toujours quorum et le cluster continue de fonctionner. Si 2 nœuds tombent simultanément, le nœud isolé n'a plus quorum et se met en retrait pour éviter le *split-brain*, une situation où deux parties du cluster croient chacune être la seule à fonctionner et modifient les données de manière concurrente.

Haute disponibilité (HA Manager)

Proxmox intègre un gestionnaire de haute disponibilité natif composé de deux services [25] :

- **pve-ha-crm** (*Cluster Resource Manager*) : le cerveau centralisé du HA. Il surveille l'état de toutes les ressources protégées (VMs, conteneurs) à l'échelle du cluster et prend les décisions de basculement.
- **pve-ha-lrm** (*Local Resource Manager*) : l'agent local sur chaque nœud. Il exécute les ordres du CRM (démarrer, arrêter, migrer une VM) et remonte l'état des ressources locales.

Lorsqu'un nœud devient inaccessible (perte de heartbeat Corosync), le CRM déclenche le **fencing** (isolation forcée du nœud défaillant via le watchdog matériel ou logiciel), puis ordonne le redémarrage des VMs protégées sur les nœuds survivants. Le délai typique de basculement est de 60 à 120 secondes. Les VMs enregistrées dans le HA Manager avec leurs disques sur un stockage partagé (Ceph RBD) peuvent être redémarrées sur n'importe quel nœud du cluster sans déplacement de données.

Live Migration

La migration à chaud (*live migration*) permet de déplacer une VM en cours d'exécution d'un nœud physique vers un autre sans interruption de service perceptible par l'utilisateur. Proxmox implémente ce mécanisme via QEMU en trois phases :

1. **Pré-copie mémoire** : QEMU copie itérativement les pages mémoire de la VM vers le nœud cible pendant que la VM continue de fonctionner. Les pages modifiées pendant la copie (*dirty pages*) sont recopiées lors de passes successives.
2. **Basculement** : lorsque le volume de dirty pages devient suffisamment faible (quelques dizaines de millisecondes de transfert), la VM est brièvement suspendue, les dernières pages sont transférées, et la VM reprend sur le nœud cible.
3. **Nettoyage** : les ressources de la VM sont libérées sur le nœud source.

Cette fonctionnalité est le mécanisme central d'un service d'équilibrage dynamique de charge, qui déclenche automatiquement des migrations à chaud pour redistribuer la charge CPU entre les nœuds. Le trafic de migration doit transiter sur un segment réseau dédié, isolé des autres flux pour garantir la bande passante nécessaire sans impacter les charges de travail de production.

Interface de gestion et API REST

Proxmox expose une **API REST complète** sur le port 8006 (HTTPS), utilisée aussi bien par l'interface web que par des outils externes. Chaque opération réalisable depuis l'interface graphique est accessible via l'API : création de VMs, gestion du stockage, déclenchement de migrations, consultation des métriques en temps réel. Cette API permet notamment d'interroger l'état des nœuds, de lister les VMs en cours d'exécution, et de déclencher des migrations programmatically via des appels HTTP authentifiés par ticket de session.

Récapitulatif de la pile technique Proxmox

Composant	Rôle
KVM (<code>kvm.ko</code>)	Hyperviseur noyau, exécution native du code VM via VT-x / AMD-V
QEMU (<code>qemu-system-x86_64</code>)	Émulation des périphériques, gestion mémoire, live migration
LXC	Conteneurs Linux légers partageant le noyau hôte
Corosync	Heartbeat cluster, consensus, détection de panne
pve-ha-crm / lrm	Gestionnaire de haute disponibilité, fencing, basculement automatique
API REST (port 8006)	Interface de pilotage programmatique et d'automatisation

II.5.2 Ceph : Le stockage distribué

Ceph est une solution libre de stockage distribuée (Software-Defined Storage) qui s'impose aujourd'hui comme une référence dans le domaine des infrastructures open source [17]. Conçu pour répondre aux besoins de fiabilité, d'extensibilité et de performance, Ceph constitue le choix idéal pour assurer la couche de stockage d'une infrastructure hyperconvergée telle que celle que nous proposons. Sa capacité à fonctionner sur du matériel standard (serveurs x86) et son architecture auto-réparatrice en font un composant central de notre projet.

Architecture de Ceph

L'architecture de Ceph repose sur un composant fondamental appelé RADOS (Reliable Autonomic Distributed Object Store). RADOS constitue le cœur du système, chargé de la

gestion, de la réplication et de la répartition des données sur l'ensemble du cluster. Autour de ce cœur gravitent plusieurs types de démons qui interagissent pour former un cluster cohérent et performant :

- **Ceph Monitor (MON)** : Les Moniteurs sont les arbitres d'état du cluster. Ils maintiennent une carte maîtresse (*cluster map*) décrivant la topologie complète du cluster et assurent le consensus entre les nœuds via un mécanisme de quorum. En cas de désaccord, la majorité des Monitors décide de l'état authoritative du cluster, évitant ainsi les situations de *split-brain*.
- **Ceph Manager (MGR)** : Le Manager agit comme un chef d'orchestre. Il collecte toutes les métriques et informations d'état du cluster en temps réel (utilisation du stockage, performances, statistiques). Il expose ces données via une interface web de monitoring et les met à disposition des systèmes de gestion externes, facilitant ainsi l'administration et la supervision.
- **Ceph OSD (Object Storage Daemon)** : L'OSD est le démon qui gère les périphériques de stockage physiques ou logiques (disques, partitions, volumes logiques). Chaque OSD est responsable de la lecture et de l'écriture des données sur son disque. Les OSDs communiquent entre eux pour assurer la réplication, le rééquilibrage et la réparation des données en cas de panne. Dans une architecture hyperconvergée, les OSDs résident sur les mêmes nœuds que l'hyperviseur (Proxmox), chaque nœud contribuant ainsi au stockage global.
- **Ceph Metadata Server (MDS)** : Ce démon est spécifique au système de fichiers distribué CephFS. Il gère les métadonnées des fichiers (noms, permissions, arborescences). Pour un usage principalement en mode bloc (disques de machines virtuelles), ce composant n'est pas requis.

La gestion intelligente des données : CRUSH et Placement Groups

L'intelligence de Ceph réside dans sa manière unique de gérer l'emplacement des données. Contrairement aux systèmes de stockage traditionnels qui utilisent une table de correspondance centralisée, Ceph utilise un algorithme appelé CRUSH (Controlled Replication Under Scalable Hashing) [18]. Cet algorithme permet à un client de calculer mathématiquement l'emplacement exact d'un objet (ou de sa réplique) en se basant sur la carte CRUSH, sans avoir à interroger un serveur central. Cette approche élimine un point de défaillance unique et améliore considérablement la scalabilité.

La carte CRUSH est une structure hiérarchique qui définit l'organisation physique du cluster. L'administrateur peut y définir des "domaines de défaillance" (nœuds, racks, salles, data centers). Ceph utilise ensuite ces informations pour placer les répliques des données sur des nœuds distincts, garantissant ainsi que la perte d'un équipement n'entraîne pas de perte de données.

Pour optimiser la gestion des milliers d'objets, Ceph les regroupe dans des unités logiques appelées Placement Groups (PGs). Les PGs constituent une abstraction intermédiaire entre l'objet et l'OSD, permettant un équilibrage efficace des données et une meilleure performance du cluster.

Les modes de protection des données

Pour assurer la fiabilité et la disponibilité des données, Ceph propose deux mécanismes principaux, adaptés à différents besoins :

- **Réplication** : C'est le mode le plus simple et le plus performant. Chaque objet est dupliqué en N copies (généralement 3) et chaque copie est placée sur des OSDs différents, dans des domaines de défaillance distincts. En cas de panne d'un OSD ou d'un nœud, les données restent disponibles sur les autres répliques. Ce mode offre une reconstruction rapide et est idéal pour les charges de travail exigeant de hautes performances en écriture, comme les disques de machines virtuelles. Dans le cadre d'un cluster à 3 nœuds, une réplication de facteur 3 garantit une haute disponibilité en cas de panne d'un nœud.
- **Erasur Coding (Codage d'effacement)** : Ce mode est plus efficace en termes d'espace de stockage. Les données sont découpées en K fragments de données, puis encodées avec M fragments de parité, pour un total de K+M fragments répartis sur différents OSDs. Ce mode permet de tolérer la perte de M fragments tout en reconstruisant les données initiales. Il est souvent utilisé pour les données froides ou l'archivage, car il nécessite moins d'espace que la réplication (ex : 1,5x l'espace brut contre 3x pour la réplication), mais il est plus coûteux en calcul lors des opérations de reconstruction.

Intégration de Ceph dans une architecture hyperconvergée

L'intégration de Ceph avec Proxmox VE, l'hyperviseur retenu, est native et profondément intégrée. Proxmox offre une interface unifiée qui permet de créer et de gérer un cluster Ceph directement depuis son interface web, simplifiant considérablement les opérations de déploiement et de maintenance.

Dans une architecture hyperconvergée, chaque nœud joue un double rôle :

- **Rôle de calcul** : Il exécute les machines virtuelles via KVM.
- **Rôle de stockage** : Il contribue au stockage global via ses disques alloués au pool Ceph.

Cette convergence des rôles est l'essence même de l'hyperconvergence. Les volumes de stockage des VMs sont fournis via Ceph RBD (RADOS Block Device), qui expose des périphériques de stockage en bloc sur le réseau. Ces volumes sont distribués et répliqués sur l'ensemble du cluster, assurant ainsi la persistance des données même en cas de panne d'un nœud.

Les avantages apportés par Ceph dans ce type d'infrastructure sont multiples :

1. **Évolutivité horizontale** : L'ajout d'un nouveau nœud apporte simultanément de la puissance de calcul et de la capacité de stockage. Ceph rééquilibre automatiquement les données, permettant une croissance linéaire et sans interruption.
2. **Haute disponibilité** : L'absence de point de défaillance unique (SPOF) et la réplication des données sur plusieurs nœuds garantissent que le système de stockage reste opérationnel même en cas de panne d'un ou plusieurs serveurs.

3. **Auto-réparation** : En cas de panne d'un disque ou d'un nœud, le cluster détecte l'incident et commence immédiatement à recréer les répliques manquantes sur les autres nœuds, assurant ainsi la pérennité des données sans intervention manuelle.
4. **Coût maîtrisé** : Ceph est open source et fonctionne sur du matériel standard (x86), ce qui le rend bien moins coûteux que les solutions propriétaires comme les SAN ou les appliances HCI propriétaires.
5. **Simplicité de gestion** : L'unification du stockage via Proxmox et les outils modernes comme `cephadm` (permettant un déploiement conteneurisé et simplifié) réduisent considérablement la complexité administrative.

II.5.3 Open vSwitch (OVS) : La virtualisation réseau

Présentation générale

Open vSwitch (OVS) est un commutateur logiciel multicouche open source, distribué sous licence Apache 2.0, conçu pour fournir une plateforme de commutation de qualité production dans les environnements de virtualisation [6]. Il fonctionne selon deux composants principaux : `ovs-vswitchd`, le démon qui implémente le switch avec son module noyau Linux pour un transfert haute performance, et `ovsdb-server`, une base de données légère de configuration persistante. Parmi ses fonctionnalités natives figurent le modèle VLAN 802.1Q, le bonding avec ou sans LACP, la supervision via NetFlow et sFlow, la configuration QoS, ainsi que le support natif des protocoles de tunneling GRE, VXLAN et Geneve [7].

Dans un cluster HCI, OVS joue le rôle de commutateur central de chaque nœud, assurant la segmentation de tout le trafic en flux distincts et isolés depuis un bridge virtuel unique par serveur.

Pourquoi segmenter le réseau dans un cluster HCI ?

La nécessité d'une segmentation réseau stricte découle de la coexistence, sur la même infrastructure physique, de trafics aux profils radicalement différents et incompatibles :

- La **réplication Ceph** génère un trafic continu à très haute bande passante entre les disques de stockage distribués, pouvant saturer un réseau partagé.
- Les **heartbeats Corosync** sont extrêmement sensibles à la latence : tout délai ou perte de paquet peut déclencher un basculement non souhaité du cluster.
- Les **migrations à chaud** (live migration) produisent des pics de transfert ponctuels mais intenses, correspondant au déplacement complet de la mémoire vive d'une VM entre deux nœuds.
- Les **charges de travail applicatives** génèrent un trafic variable et imprévisible, qui ne doit pas interférer avec les flux d'infrastructure.

Sans isolation stricte, ces flux en compétition se dégradent mutuellement et menacent la stabilité globale du cluster. Cette problématique est identique à celle que résout Nutanix avec la séparation entre le trafic du contrôleur interne et le trafic des VMs utilisateur. Cette logique se traduit en segments réseau dédiés gérés par OVS.

Les recommandations officielles de la documentation Proxmox et les guides Ceph préconisent explicitement la séparation du réseau interne Ceph (réplication entre nœuds de

stockage) du réseau public Ceph (accès des VMs au stockage), ainsi que l'isolation du trafic de synchronisation du cluster sur une interface dédiée.

Les neuf types de trafic à isoler

Ces contraintes conduisent à identifier neuf types de trafic aux profils distincts, chacun justifiant un segment réseau dédié. Le plan d'adressage précis et l'attribution des identifiants de tunnels sont définis dans le chapitre suivant ; ce qui importe ici est la nature du besoin qui motive chaque isolation :

- **Administration et gestion** : flux d'accès à la plateforme de virtualisation (interface web, SSH, API). Volume faible mais disponibilité permanente requise. Ce segment doit être strictement isolé des réseaux applicatifs pour prévenir tout accès non autorisé à la console d'administration depuis une VM compromise.
- **Synchronisation du cluster** : heartbeats et mécanismes de consensus entre les nœuds. Ce trafic est extrêmement sensible à la latence et aux pertes de paquets : tout délai supérieur aux seuils Corosync peut déclencher un basculement non souhaité et déstabiliser l'ensemble du cluster.
- **Réplication interne du stockage distribué** : communications directes entre les processus de stockage pour la réplication, le rééquilibrage et la reconstruction des données après panne. Ce flux est continu et à très haute bande passante ; le mélanger avec d'autres trafics dégrade directement les performances de stockage et risque de provoquer des délais de réplication.
- **Accès des machines virtuelles au stockage** : flux entre les VMs et le pool de stockage distribué (montage des volumes en mode bloc). Séparé du réseau de réplication interne pour éviter que les opérations de rééquilibrage ou de resynchronisation ne pénalisent les performances applicatives des VMs en production.
- **Migrations à chaud** : transferts de l'état mémoire complet d'une VM entre deux nœuds physiques. Ces flux sont ponctuels mais intenses, pouvant temporairement saturer la bande passante disponible ; leur isolation garantit que les migrations n'impactent ni le trafic applicatif ni la réplication du stockage.
- **Machines virtuelles de production** : trafic applicatif des services exposés aux utilisateurs finaux. Ce segment est le seul accessible depuis les points d'entrée contrôlés (répartiteur de charge) et doit être isolé des environnements de développement.
- **Machines virtuelles de test et de développement** : trafic des environnements de développement et de validation. Strictement isolé de la production pour éviter toute interférence entre les charges de travail de test et les services en production.
- **Zone exposée** : services destinés à être accessibles depuis l'extérieur du cluster. Cette zone est protégée par le pare-feu périmétrique et accessible uniquement via le répartiteur de charge comme point d'entrée unique et contrôlé.
- **Sauvegarde** : transferts de sauvegarde et d'instantanés de l'infrastructure. Ces opérations peuvent générer un trafic soutenu et doivent rester hors bande par rapport aux réseaux applicatifs afin de ne pas dégrader les performances en production.

Contrainte d'infrastructure rencontrée et adaptation : adoption de VXLAN

L'architecture initiale reposait sur une segmentation par **VLANs 802.1Q**, nécessitant que le switch physique soit configuré en mode *trunk* pour propager les tags VLAN entre les trois nœuds. Cette approche est standard et parfaitement supportée par OVS.

Cependant, le déploiement sur l'infrastructure de Sonatrach a révélé une contrainte opérationnelle déterminante : les trois serveurs Dell R720 sont connectés à un switch administré dont la configuration est gérée exclusivement par l'équipe réseau de l'entreprise. Ce switch dispose déjà de VLANs définis (VLAN 10 pour le management, VLAN 20 pour un autre usage interne), et l'équipe projet ne dispose d'aucun droit d'administration pour modifier sa configuration, créer de nouveaux VLANs, ou reconfigurer les ports en mode trunk. Les serveurs sont simplement connectés en mode access sur le VLAN de gestion.

Face à cette contrainte, l'approche retenue est le remplacement des VLANs 802.1Q par des tunnels **VXLAN** (*Virtual eXtensible LAN*, RFC 7348) [26].

VXLAN : principe de fonctionnement

VXLAN est un protocole d'encapsulation réseau qui encapsule des trames Ethernet de niveau 2 dans des paquets UDP/IP de niveau 4, permettant de créer des réseaux virtuels superposés (*overlay networks*) par-dessus un réseau physique existant, sans nécessiter aucune modification de ce dernier.

Le principe est le suivant : lorsqu'une VM sur le nœud A souhaite communiquer avec une VM sur le nœud B à travers un réseau logique donné, OVS encapsule la trame Ethernet originale dans un paquet UDP à destination de l'adresse IP physique du nœud B. Ce paquet transite sur le réseau physique sous-jacent comme un paquet IP ordinaire. À la réception, OVS sur le nœud B désencapsule le paquet et restitue la trame originale à la VM de destination. Le switch physique ne voit que du trafic UDP ordinaire et n'a aucune connaissance des réseaux virtuels qui transitent au-dessus de lui.

Chaque réseau logique est identifié par un **VNI** (*VXLAN Network Identifier*), un entier sur 24 bits, l'équivalent fonctionnel du tag VLAN. Chacun de ces neuf types de trafic se traduit en un tunnel VXLAN distinct, identifié par un VNI unique, créé et géré nativement par OVS sur chacun des nœuds. L'attribution précise des VNIs et des plages d'adressage est détaillée dans le chapitre suivant.

Impact sur l'architecture et bilan

L'adoption de VXLAN ne remet pas en cause l'architecture réseau à neuf segments définie initialement. Les neuf segments logiques (management, Corosync, Ceph interne, Ceph public, migration à chaud, VMs de production, test/dev, DMZ, backup) sont intégralement conservés, chacun matérialisé par un identifiant VNI dédié au lieu d'un tag VLAN. OVS assure la création, la gestion et l'isolation de ces tunnels de manière transparente, sans aucune intervention sur l'infrastructure réseau physique de l'entreprise.

Cette approche constitue en réalité une évolution positive sur plusieurs points. Premièrement, elle renforce l'indépendance de l'infrastructure HCI vis-à-vis du réseau physique sous-jacent : le cluster peut fonctionner sur n'importe quel réseau IP sans prérequis de configuration. Deuxièmement, VXLAN est le protocole d'isolation réseau standard des infrastructures cloud modernes (AWS, Azure, GCP utilisent tous des superpositions de type VXLAN ou similaire) [26], ce qui renforce la représentativité technologique du projet. Troi-

sièment, la capacité de 16 millions d'identifiants VNI, contre 4 094 VLANs 802.1Q, offre une scalabilité largement suffisante pour toute extension future du cluster.

La seule contrainte introduite est un surcoût d'encapsulation de 50 octets par paquet UDP, négligeable au regard des débits disponibles sur des liaisons Gigabit ou 10 Gigabit Ethernet. Les performances de réplication Ceph, des migrations à chaud et des heartbeats Corosync ne sont pas affectées de manière mesurable par cette encapsulation supplémentaire.

II.6 L'Équilibrage Dynamique de Charge dans les Clusters Virtualisés

II.6.1 Problématique du Déséquilibre de Charge

Dans un cluster de virtualisation, les VMs sont placées sur les nœuds physiques lors de leur création. Sans mécanisme d'équilibrage actif, ce placement reste statique pendant toute la durée de vie des VMs. Au fil du temps, les charges de travail évoluent et la charge se répartit inégalement : certains nœuds peuvent être saturés à plus de 90 % de CPU tandis que d'autres restent utilisés à moins de 20 %. Ce déséquilibre dégrade les performances des VMs sur les nœuds chargés et gaspille les ressources disponibles sur les nœuds sous-chargés.

II.6.2 Mécanismes d'Équilibrage dans les Plateformes Existantes

VMware vSphere propose le **DRS** (*Distributed Resource Scheduler*) [8], mécanisme d'équilibrage automatique qui analyse en continu les métriques de charge de chaque hôte et déclenche des migrations à chaud (vMotion) pour redistribuer les VMs. Le DRS calcule un score de déséquilibre pour le cluster et ne déclenche une migration que si le gain attendu dépasse un seuil configurable, afin d'éviter des migrations inutiles dans un cluster globalement sous-chargé.

D'autres plateformes proposent des mécanismes comparables. **Microsoft SCVMM** (*System Center Virtual Machine Manager*) intègre la *Dynamic Optimization* [27], qui rééquilibre les VMs Hyper-V selon des intervalles configurables en minimisant les migrations inutiles. **oVirt** (plateforme open source de Red Hat) dispose d'un planificateur de cluster (*cluster scheduler*) qui évalue la charge des hôtes et déclenche des migrations via libvirt [28]. **OpenStack Nova** implémente enfin un planificateur distribué capable de placer et de migrer les VMs selon des filtres et des pondérations paramétrables [29].

Proxmox VE ne dispose pas d'un tel mécanisme natif. Ce projet développe un service équivalent, nommé CLBS, s'appuyant sur l'API REST Proxmox et le stockage partagé Ceph.

II.6.3 L'Algorithme CSLB : Base Théorique du CLBS

Le CLBS est fondé sur l'algorithme CSLB (*Central Scheduler Load Balancing*) proposé par Wilcox pour les environnements de virtualisation distribués [20]. L'algorithme repose sur un scheduler centralisé qui opère en cinq phases successives [13] :

1. **Évaluation de la charge** : lecture des métriques CPU et RAM de chaque nœud sur une fenêtre temporelle glissante, pour s'affranchir des pics instantanés.

2. **Détermination de rentabilité** : calcul d'une bande modérée autour de la moyenne des charges du cluster. Les nœuds hors de cette bande sont candidats à une action d'équilibrage.
3. **Calcul du vecteur de transfert** : détermination de la quantité de ressources à migrer pour ramener le nœud surchargé dans la bande modérée.
4. **Sélection de la VM** : choix de la VM dont la charge correspond le mieux au vecteur de transfert calculé.
5. **Exécution de la migration** : déclenchement d'une live migration vers le nœud destination présentant la charge la plus faible.

La migration à chaud est le mécanisme d'exécution de toute décision d'équilibrage. Elle n'est applicable que si le stockage est partagé entre tous les nœuds (comme Ceph RBD), condition qui rend le disque de la VM accessible depuis le nœud de destination sans déplacement préalable des données [12].

II.7 Outils Complémentaires d'Infrastructure

II.7.1 HAProxy : Équilibrage de Charge Applicatif

HAProxy (*High Availability Proxy*) est un proxy TCP/HTTP open source conçu pour la haute disponibilité et l'équilibrage de charge applicatif [16]. Il agit comme point d'entrée unique pour un ensemble de serveurs applicatifs, distribue les requêtes selon différents algorithmes (round-robin, moins de connexions, hachage par IP source) et surveille la disponibilité des backends via des contrôles de santé périodiques. En cas d'indisponibilité d'un backend, HAProxy redirige automatiquement le trafic vers les instances opérationnelles sans interruption pour l'utilisateur.

II.7.2 Stack de Supervision : InfluxDB, Prometheus et Grafana

La supervision d'une infrastructure hyperconvergente nécessite des outils capables de collecter, stocker et visualiser des métriques en temps réel sur l'ensemble des composants. L'objectif est de couvrir quatre dimensions clés : l'état de santé et la charge de chaque nœud du cluster, les journaux système et événements d'infrastructure, l'historique des migrations à chaud et leur impact sur la répartition de charge, et l'état du pool de stockage distribué (utilisation, latence, état de santé des disques).

Trois outils complémentaires constituent la pile de supervision d'une telle infrastructure.

InfluxDB [9] est une base de données orientée séries temporelles (*time series database*), optimisée pour le stockage de données horodatées à haute fréquence d'écriture. Elle reçoit les métriques de l'hyperviseur (CPU, RAM, réseau, stockage des nœuds et des VMs) via le mécanisme natif *Metric Server*, sans agent supplémentaire. Cet historique est utilisé à la fois pour la visualisation et pour alimenter les décisions de migration du service d'équilibrage.

Prometheus [30] est un système de collecte de métriques par interrogation HTTP, organisé autour d'un modèle de données par étiquettes. Il collecte les métriques détaillées du cluster de stockage distribué via le point d'accès exposé par son gestionnaire, ainsi que les métriques applicatives du service d'équilibrage.

Grafana [15] est un outil de visualisation connecté aux deux sources de données précédentes. Il permet de construire des tableaux de bord unifiés couvrant l'état des nœuds, l'état du stockage distribué et l'activité du service d'équilibrage, et d'y associer des règles d'alerte.

II.7.3 Ansible : Automatisation d'Infrastructure

Ansible [31] est un outil d'automatisation d'infrastructure (*Infrastructure as Code*) qui exécute des playbooks YAML via SSH, sans agent logiciel sur les machines cibles. Un playbook décrit l'état cible d'une machine (paquets installés, services configurés, fichiers déployés) et Ansible l'applique de manière **idempotente** : si la configuration est déjà conforme à l'état cible, aucune action n'est effectuée.

Chaque playbook est organisé en tâches séquentielles exécutées sur un inventaire de machines défini en YAML. La connexion SSH et les privilèges d'administration (**sudo**) sont les seuls prérequis sur les machines cibles. Ansible automatise typiquement l'intégration de nouveaux nœuds physiques dans le cluster, le provisionnement de machines virtuelles et de conteneurs, et la promotion d'environnements de développement vers la production.

II.7.4 PXE et TFTP : Provisionnement Réseau

PXE (*Preboot eXecution Environment*) est un mécanisme permettant à un serveur de démarrer depuis le réseau sans support de démarrage local. Il repose sur la coopération de trois protocoles :

- **DHCP** : attribution d'une adresse IP temporaire au serveur et indication de l'adresse du serveur de boot (options DHCP 66 et 67).
- **TFTP** (*Trivial File Transfer Protocol*, UDP/69) : téléchargement du bootloader et du noyau d'amorçage. TFTP est retenu pour cette phase initiale car c'est le seul protocole utilisable dans un firmware pre-OS.
- **HTTP** : téléchargement de l'image d'installation (plusieurs centaines de Mo). TFTP est inadapté aux fichiers volumineux en raison de l'absence de mécanisme de contrôle de flux.

Ce mécanisme est intégré au pipeline d'automatisation pour l'installation initiale de l'hyperviseur sur les serveurs physiques, sans aucune intervention manuelle sur les serveurs.

II.7.5 OPNsense : Pare-feu et Sécurité Périmétrique

OPNsense [32] est une distribution de pare-feu et de routeur open source basée sur FreeBSD. Il assure le filtrage stateful des flux réseau, la gestion des règles de pare-feu par interface et par VLAN, ainsi que des fonctionnalités avancées telles que la détection d'intrusion (Suricata), le VPN et le portail captif. Dans une infrastructure hyperconvergente, OPNsense constitue le point de contrôle périmétrique entre les segments réseau internes et l'extérieur, protégeant notamment la zone exposée (DMZ).

II.7.6 NTP : Synchronisation Temporelle

La synchronisation temporelle est un prérequis critique pour les systèmes distribués. Dans un cluster, une dérive d'horloge entre les nœuds peut provoquer des incohérences dans les journaux d'événements, perturber les mécanismes de consensus (Corosync, Ceph) et invalider les timestamps des métriques de supervision. Le protocole NTP (*Network Time Protocol*) [33] permet à chaque nœud de synchroniser son horloge avec un serveur de référence. Un serveur NTP interne au cluster garantit que tous les composants (hyperviseur, stockage distribué, service d'équilibrage, outils de supervision) partagent une base de temps cohérente.

II.7.7 GitLab CE : Intégration et Déploiement Continu

GitLab CE (*Community Edition*) [21] est une plateforme DevOps open source intégrant un gestionnaire de dépôts Git, un système d'intégration continue (CI) et de déploiement continu (CD). Les pipelines CI/CD de GitLab permettent d'automatiser les tests, la construction et le déploiement des applications à chaque modification du code source. Dans le contexte d'une infrastructure hyperconvergée, GitLab est utilisé pour versionner les configurations d'infrastructure (playbooks Ansible, fichiers de configuration), déclencher les déploiements automatisés et maintenir une traçabilité complète des modifications apportées à l'environnement.

II.8 Déploiement sur Infrastructure Physique Distribuée

La conception initiale du projet envisageait d'héberger l'ensemble du cluster sur un seul serveur via de la virtualisation imbriquée (*nested virtualization*). Cette approche présente un avantage de simplicité logistique, mais concentre tous les composants du cluster sur une seule machine physique, ce qui annule les propriétés de résilience propres à une architecture distribuée : la panne du serveur hôte entraînerait l'arrêt complet du cluster, les flux inter-nœuds transitent par le bus mémoire interne (faussant les mesures réseau), et les extensions matérielles de virtualisation KVM ne sont pas pleinement accessibles sous un hyperviseur parent.

L'évolution vers un déploiement physique sur trois serveurs distincts lève l'ensemble de ces contraintes. Chaque serveur constitue un nœud indépendant du cluster, avec ses propres ressources (CPU, RAM, disques). Les communications inter-nœuds (réplication du stockage distribué, synchronisation du cluster, migrations à chaud) transitent sur le réseau physique réel, ce qui permet de valider les performances dans des conditions représentatives d'un environnement de production. La panne d'un nœud est absorbée par le mécanisme de haute disponibilité et le quorum de synchronisation, sans impact sur la disponibilité globale. Les caractéristiques détaillées du déploiement physique (adressage, configuration réseau, rôles des nœuds) sont présentées dans le chapitre suivant.

II.9 Conclusion

Ce chapitre a posé les bases théoriques du projet. Le concept d'hyperconvergence a été défini et les principales solutions open source ont été comparées, aboutissant au choix de

Proxmox VE comme hyperviseur, Ceph comme système de stockage distribué, et Open vSwitch comme commutateur logiciel. La pile de virtualisation (hyperviseur noyau, émulateur de machine, bibliothèque de gestion) a été décrite en détail, ainsi que les mécanismes de clustering, de haute disponibilité et de migration à chaud. La virtualisation réseau par tunnels superposés a été présentée comme réponse aux contraintes d'un réseau physique non administrable. Les fondements théoriques de l'équilibrage dynamique de charge ont été établis, en référence aux mécanismes existants dans d'autres plateformes. Enfin, les outils complémentaires d'infrastructure (répartition de charge applicative, supervision, automatisation, provisionnement réseau, pare-feu, synchronisation temporelle, intégration continue) et la justification du passage à un déploiement physique distribué ont été présentés. Le chapitre suivant présente la conception détaillée de l'architecture proposée.

Chapitre III

Présentation du Contexte

III.1 Introduction

Ce chapitre présente la conception détaillée de l'infrastructure Cloud privée hyperconvergée proposée. Sur la base des technologies retenues dans le chapitre précédent — Proxmox VE, Ceph et Open vSwitch — nous décrivons ici les choix architecturaux guidant la construction de chaque couche de l'infrastructure. Nous présentons également la conception du mécanisme d'équilibrage dynamique de charge CLBS, ainsi que les outils d'automatisation retenus pour le déploiement et le provisionnement.

L'objectif de ce chapitre est de fournir une vision complète et cohérente de l'architecture avant toute phase d'implémentation, en justifiant chaque décision par des contraintes techniques, des recommandations officielles ou des exigences fonctionnelles identifiées précédemment.

III.2 Vue Globale de l'Architecture

L'architecture proposée repose sur un modèle à trois couches superposées et interdépendantes, chacune apportant une dimension essentielle à l'infrastructure hyperconvergée.

La première couche est la **couche physique**, constituée de trois serveurs Dell PowerEdge R720 mis à disposition par le Centre d'innovation de Sonatrach. Proxmox VE est installé directement en mode bare metal sur chacun de ces serveurs, sans couche de virtualisation intermédiaire, ce qui garantit des performances maximales.

La deuxième couche est la **couche de virtualisation et de cluster**, composée des trois nœuds Proxmox (PVE-01, PVE-02, PVE-03) interconnectés via le réseau Sonatrach. Ces nœuds forment un cluster haute disponibilité géré par Corosync. Chaque nœud exécute simultanément les daemons Ceph pour le stockage distribué et une instance Open vSwitch avec des tunnels VxLAN pour la segmentation réseau. Cette convergence des rôles est l'expression directe du principe HCI.

La troisième couche est la **couche applicative et de services**, hébergeant la VM OPNsense pour le routage et le pare-feu, les VMs de démonstration Sona-Web, le mécanisme CLBS, la stack de monitoring, et les outils d'automatisation Ansible.

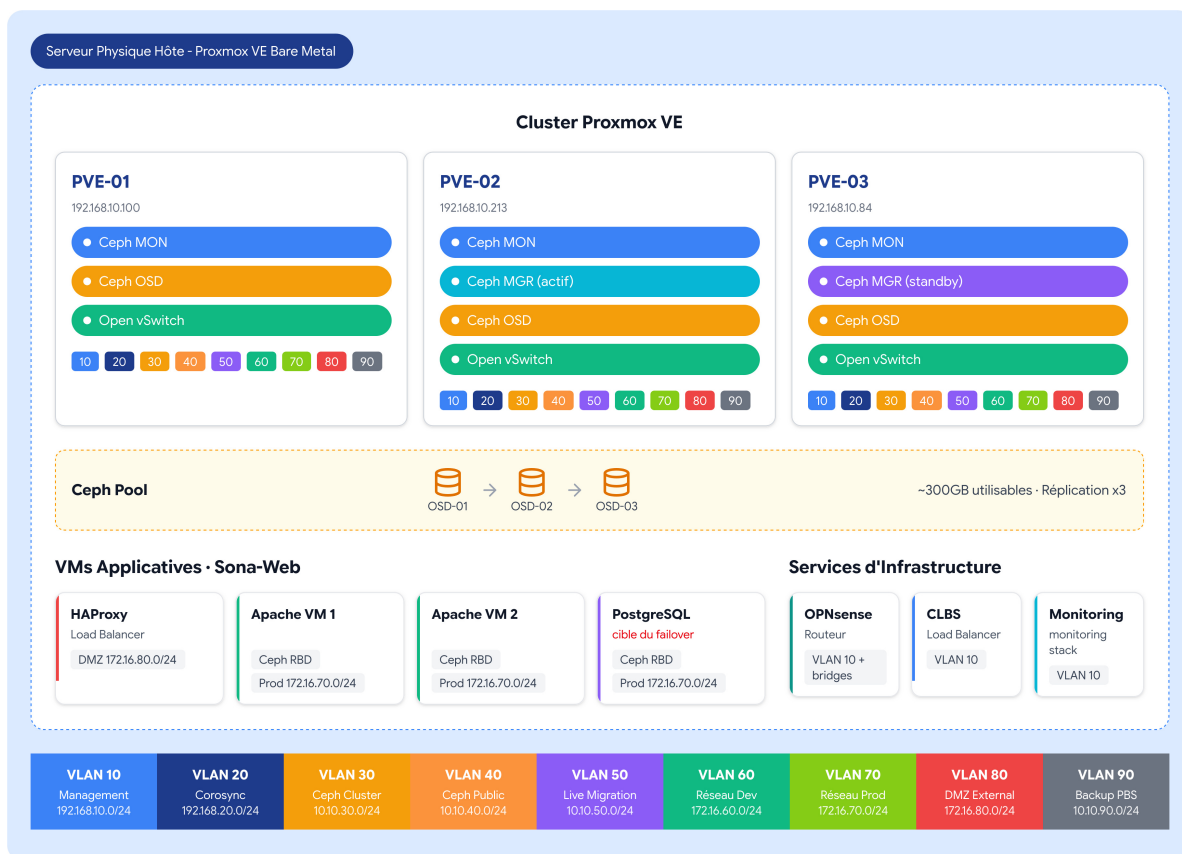


FIGURE III.1 – Architecture Globale de l'Infrastructure HCI

La figure III.1 représente les trois couches superposées de l'architecture HCI : la couche physique composée des serveurs Dell R720, la couche de virtualisation et de cluster portant Proxmox, Ceph et Open vSwitch, et la couche applicative hébergeant les services déployés. Elle met en évidence la convergence des rôles sur chaque nœud physique, principe fondateur de l'hyperconvergence.

III.3 Organisation de la Solution

La conception de l'infrastructure hyperconvergée a été structurée en trois parties complémentaires et interdépendantes : le **Calcul**, le **Stockage** et le **Réseau**. Cette partition reflète les trois piliers fondamentaux de tout système HCI, chacun apportant une dimension essentielle à l'ensemble. Les choix effectués dans chaque partie influencent directement les deux autres : le stockage distribué Ceph repose sur le réseau pour la réplication, le mécanisme CLBS dépend du stockage partagé pour les migrations à chaud, et la segmentation réseau conditionne la sécurité de l'ensemble. Le schéma ci-dessous illustre cette organisation ainsi que les interactions entre les trois parties.

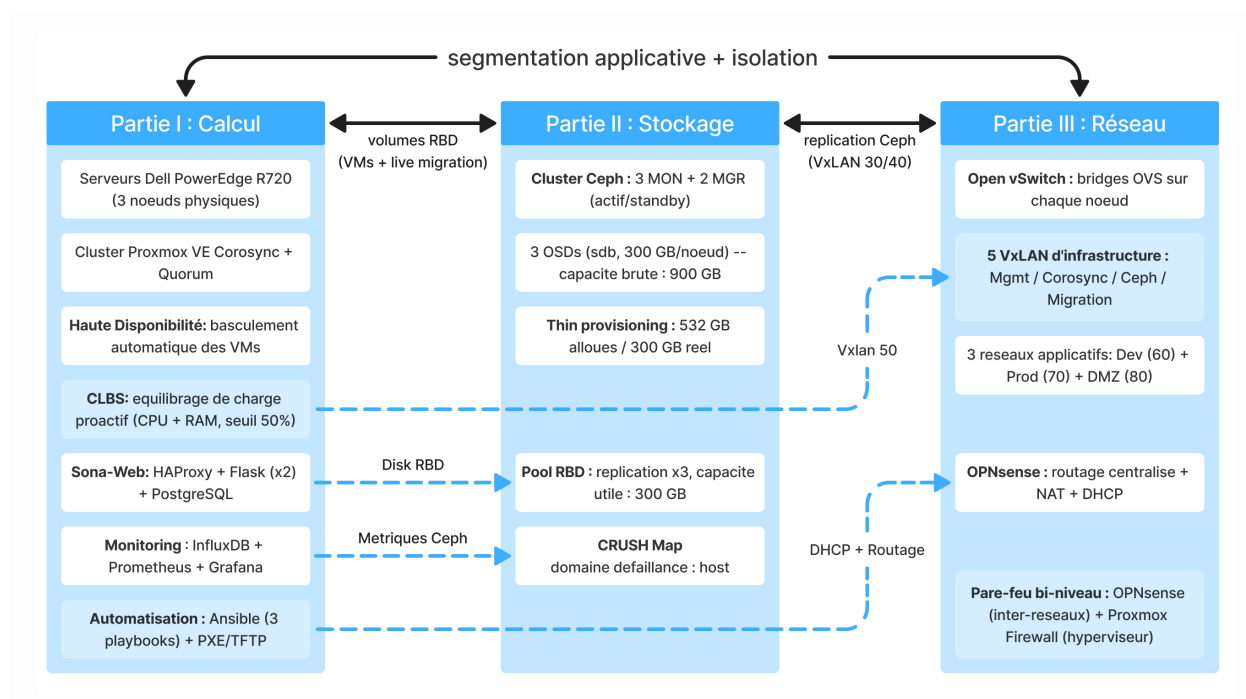


FIGURE III.2 – Organisation de la Solution en Trois Parties et leurs Interactions

La figure III.2 met en évidence les flux entre les parties : la partie Calcul s'appuie sur le Stockage pour les volumes RBD des VMs et pour les migrations à chaud réalisées par CLBS, le Stockage utilise le Réseau pour transporter les flux de réplication Ceph sur les VxLAN 30 et 40, et le Réseau fournit la segmentation applicative qui conditionne l'isolation et la sécurité de toutes les couches. Les relations détaillées au niveau composant (CLBS vers VxLAN 50, VMs Sona-Web vers pool RBD, Monitoring vers endpoint Ceph, Ansible vers OPNsense) illustrent le niveau d'interdépendance réel entre les trois parties.

III.4 Partie I : Calcul

Cette partie traite de la couche de calcul de l'infrastructure. Elle présente les serveurs physiques constituant les nœuds du cluster, l'organisation du cluster Proxmox et ses mécanismes de haute disponibilité via Corosync, le mécanisme d'équilibrage de charge CLBS développé spécifiquement pour ce projet, l'application de démonstration Sona-Web, la stack de monitoring déployée pour superviser l'ensemble de la plateforme, et enfin les outils d'automatisation Ansible et PXE permettant le déploiement et le provisionnement sans intervention manuelle.

III.4.1 Infrastructure Physique

L'infrastructure physique est constituée de trois serveurs Dell PowerEdge R720. Ces serveurs forment les nœuds du cluster Proxmox : PVE-01, PVE-02 et PVE-03. Proxmox VE est installé directement en mode bare metal sur chacun d'eux, sans couche de virtualisation intermédiaire. Le Dell PowerEdge R720 est un serveur rack 2U de la gamme Enterprise de Dell, équipé de processeurs Intel Xeon E5-2600, adapté aux charges de virtualisation intensive et au stockage distribué.

Chaque serveur dispose de cinq disques SAS répartis selon leurs rôles respectifs, conformément aux recommandations Proxmox concernant la séparation des entrées/sorties système et de stockage :

- **Disque système (sda — 1 × 300 GB SAS) :** dédié au système d'exploitation Proxmox VE. Cette séparation garantit que les opérations de réplication Ceph ne perturbent pas les I/O système du nœud.
- **Disques OSD Ceph (1 × 300 GB SAS + 3 × 600 GB SAS) :** les quatre disques restants sont affectés comme OSDs Ceph. Chaque nœud contribue ainsi avec quatre OSDs au cluster de stockage distribué, soit douze OSDs au total dans le cluster.

TABLE III.1 – Dimensionnement physique des serveurs Dell PowerEdge R720

Composant	Par nœud	Total cluster	Justification
Modèle serveur	Dell R720	3 serveurs	Fourni par Sonatrach
RAM	64 GB	192 GB	Proxmox + Ceph + VMs
Disque OS (sda)	1 × 300 GB SAS	3 × 300 GB	Isolation I/O hôte / Ceph
Disques OSD (sdb-sde)	1 × 300 GB + 3 × 600 GB SAS	12 OSDs (6,3 TB brut)	Stockage distribué Ceph
IP Management	Tableau 3-2		Réseau Sonatrach 192.168.10.0/24

III.4.2 Architecture du Cluster Proxmox

III.4.2.1 Organisation du Cluster

Le cluster Proxmox est constitué de trois nœuds physiques identiques, interconnectés via le réseau Sonatrach. Le trafic Corosync est transporté sur un réseau logique dédié (VxLAN ID 20) afin de garantir la stabilité du quorum indépendamment des autres flux. Cette organisation à trois nœuds est le minimum requis pour garantir un quorum stable et éviter les situations de split-brain.

TABLE III.2 – Adresses IP de management des nœuds Proxmox

Nœud	Nom d'hôte	Adresse IP Management
Nœud 1	PVE-01	192.168.10.100
Nœud 2	PVE-02	192.168.10.213
Nœud 3	PVE-03	192.168.10.84

III.4.2.2 Mécanisme de Quorum et Corosync

Corosync est le framework de communication utilisé par Proxmox pour assurer le consensus entre les nœuds. Il envoie des messages de heartbeat à intervalles réguliers via le VxLAN 20. Le quorum dans un cluster de trois nœuds est atteint dès que deux nœuds sur trois sont opérationnels. La perte simultanée de deux nœuds entraîne le gel du cluster pour prévenir toute corruption des données.

III.4.2.3 Haute Disponibilité

Le mécanisme de haute disponibilité de Proxmox s'appuie sur Corosync pour la détection des pannes et sur le Resource Manager HA pour les décisions de basculement. Lorsqu'un nœud est déclaré défaillant, le HA Manager identifie les VMs concernées et déclenche leur redémarrage automatique sur un nœud survivant. Ce mécanisme est rendu possible par le stockage partagé Ceph : les disques des VMs étant accessibles depuis n'importe quel nœud, le redémarrage ne nécessite aucun transfert de données préalable. Le délai de basculement est typiquement compris entre 60 et 120 secondes.

Toutes les VMs de l'application Sona-Web sont configurées en mode HA, avec une priorité de basculement définie : HAProxy en premier, suivi des serveurs Flask, puis de la base PostgreSQL.

III.4.2.4 Matrice des Scénarios de Défaillance

TABLE III.3 – Matrice des scénarios de défaillance et comportements attendus

Scénario	Composant affecté	Comportement	RTO
Panne d'un nœud physique	PVE-01, 02 ou 03	Corosync détecte la perte. HA redémarre les VMs sur les survivants. Ceph passe en degraded mais reste accessible.	60–120 s
Panne d'un OSD	Disque sdb	Ceph marque l'OSD down . Le pool passe en degraded (2/3 répliques). Backfill automatique après rétablissement.	Aucune coupure
Panne du MGR actif	PVE-02 (MGR)	Le MGR standby (PVE-03) prend le relais. Endpoint Prometheus disponible sans interruption.	< 30 s
Panne de deux nœuds	Deux des trois nœuds	Perte du quorum. Le nœud survivant se gèle. Intervention manuelle requise.	N/A
Panne VM OPNsense	VM routeur/pare-feu	Proxmox HA redémarre la VM sur un nœud survivant. Le trafic inter-réseaux est interrompu pendant la durée du redémarrage.	60–120 s
Split-brain	Réseau Vx-LAN 20	Le groupe minoritaire perd le quorum et se gèle. Le groupe majoritaire continue d'opérer.	Aucune coupure côté majoritaire
Panne CLBS	VM CLBS	Aucun impact applicatif. Le service redémarre automatiquement (Restart=always systemd).	Aucune coupure

III.4.3 Mécanisme d'Équilibrage Dynamique : CLBS

III.4.3.1 Contexte et Besoin

Proxmox VE intègre un mécanisme de haute disponibilité réactif : lorsqu'un nœud tombe en panne, les VMs sont redémarrées sur les nœuds survivants. Cependant, Proxmox ne dispose d'aucun mécanisme proactif d'équilibrage de charge, contrairement à VMware vSphere qui propose le DRS (Distributed Resource Scheduler). Sans équilibrage proactif, un nœud peut être saturé à 90% de CPU tandis qu'un autre reste à 20%, dégradant les performances et gaspillant les ressources disponibles.

III.4.3.2 Base Algorithmique : CSLB

Notre mécanisme, nommé **CLBS (Central Load Balancing Scheduler)**, est une adaptation du CSLB (Central Scheduler Load Balancing) [20]. L'algorithme original repose sur cinq phases : évaluation de charge, détermination de rentabilité, calcul du vecteur de transfert, sélection de VM, et exécution de la migration [13]. La charge CPU de chaque nœud est évaluée par rapport à un seuil dynamique (moyenne des charges de tous les nœuds) :

$$\text{seuil} = \frac{\sum_{i=1}^n \alpha_i}{n}, \quad \text{diff} = \left| \text{seuil} - \frac{\alpha_{\min} + \alpha_{\max}}{2} \right|$$

La bande modérée est définie à $\pm \max(\text{diff}, 10\%)$ autour du seuil. Les nœuds hors de cette bande sont candidats à une action d'équilibrage.

III.4.3.3 Modifications Apportées au CSLB

Notre adaptation introduit plusieurs modifications importantes par rapport à l'algorithme original.

- **Implémentation en Go** : retenu pour sa faible empreinte mémoire, sa gestion native de la concurrence via les goroutines, et la simplicité du déploiement sous forme de binaire statique dans un service systemd [12].
- **Extension à la RAM** : le CSLB original ne considère que le CPU. Notre implémentation calcule une bande modérée indépendante pour la RAM. Un nœud est surchargé si l'une ou l'autre dépasse sa bande respective.
- **Seuil minimum de migration** : une migration n'est déclenchée que si la charge dépasse un seuil absolu de 50% (CPU ou RAM), évitant des déplacements inutiles dans un cluster globalement sous-chargé.
- **Sélection du nœud destination pondérée** : le choix du nœud de destination se fonde sur un score pondéré :

$$\text{score} = \Delta_{\text{CPU}} \times 0.7 + \Delta_{\text{RAM}} \times 0.3$$

Avant validation, CLBS simule l'état post-migration du nœud cible pour s'assurer que l'ajout de la VM ne le fait pas dépasser la bande haute.

- **Exclusion des VMs système** : OPNsense, la VM monitoring et la VM CLBS sont explicitement exclues du périmètre de migration.

- **Cadence et unicité** : CLBS s'exécute toutes les 60 secondes et ne peut déclencher qu'une seule migration par cycle.
- **Cooldown post-migration** : une période de repos de 5 minutes est appliquée après chaque migration, le temps que les métriques InfluxDB intègrent l'impact du changement.
- **Mécanismes anti-oscillation** : si une même VM est migrée deux fois en moins de 10 minutes, elle est gelée 30 minutes et l'administrateur est notifié. Si 6 migrations ou plus sont enregistrées en 30 minutes, le service se suspend automatiquement pendant une heure (circuit breaker).
- **Collecte des métriques** : CLBS combine l'API REST Proxmox pour les métriques courantes, et InfluxDB via le client Go officiel pour les moyennes glissantes sur 5 minutes, garantissant des décisions fondées sur des données lissées plutôt que sur des pics instantanés.

III.4.3.4 Algorithme de Décision (Pseudocode)

```
toutes les 60 secondes :
    recuperer metriques CPU et RAM moyenees (5 min) depuis InfluxDB
    calculer seuil_cpu, bande_moderee_cpu
    calculer seuil_ram, bande_moderee_ram

pour chaque noeud N :
    si (cpu_N > bande_haute_cpu OU ram_N > bande_haute_ram)
        ET (cpu_N > 50% OU ram_N > 50%) :

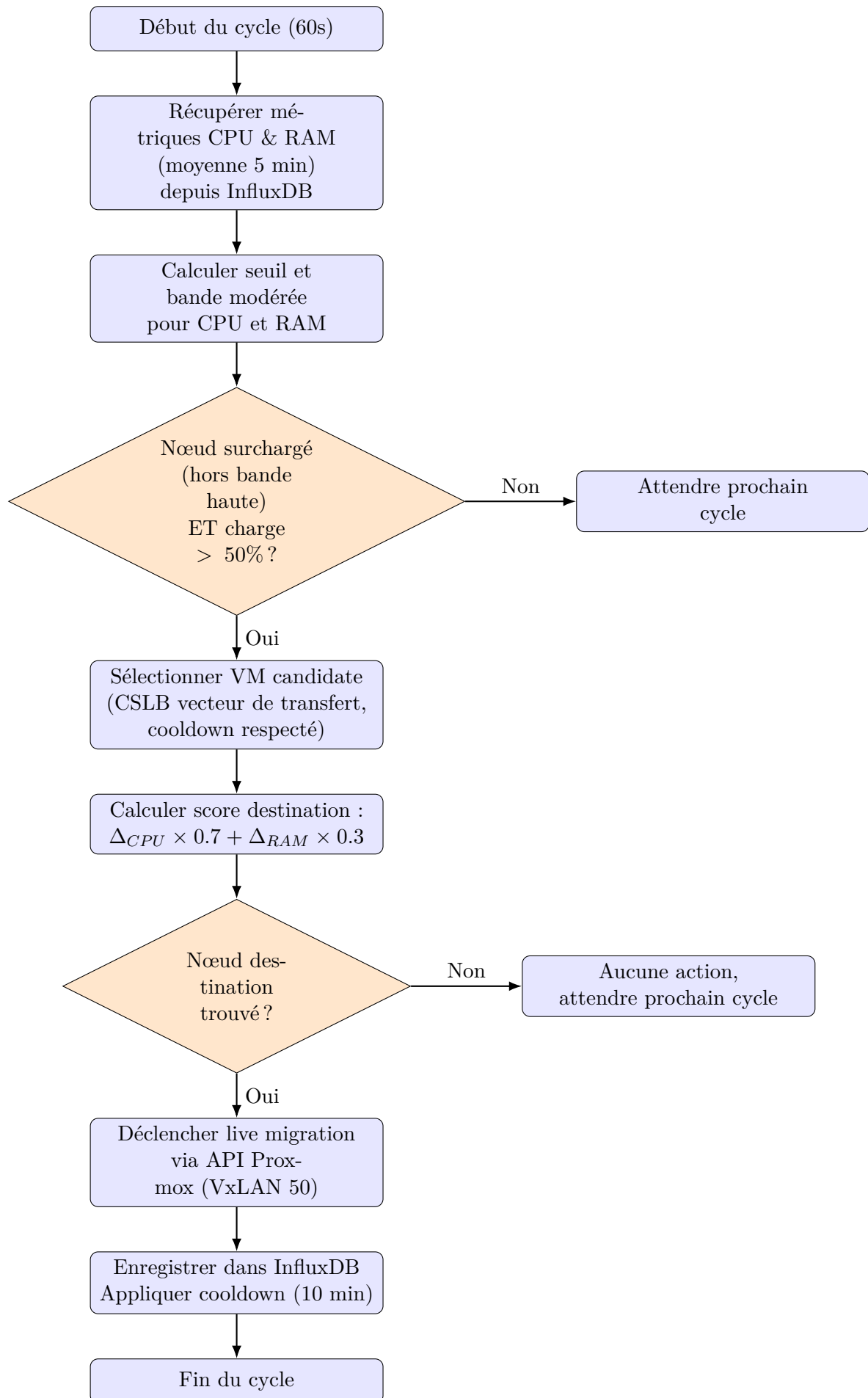
        candidats_vm = VMs sur N non migrees recemment (cooldown 10
            min)
        vm_cible = VM dont la charge est la plus proche du
            vecteur de transfert calcule (phase 3 CSLB)

        pour chaque noeud destination D (bande legere) :
            score_D = (cpu_N - cpu_D) * 0.7 + (ram_N - ram_D) * 0.3
        noeud_dest = D avec score_D maximal

    si noeud_dest trouve :
        declencher live migration vm_cible -> noeud_dest
            via API Proxmox (VxLAN 50)
        enregistrer migration dans InfluxDB
        appliquer cooldown anti-oscillation (10 min sur vm_cible)
```

Figure III.3 – Pseudocode de l'algorithme CLBS

La figure III.3 présente le pseudocode de l'algorithme CLBS sous sa forme condensée. Chaque cycle commence par la récupération des métriques CPU et RAM lissées sur 5 minutes depuis InfluxDB, afin d'éviter les décisions basées sur des pics instantanés. Le double critère de déclenchement (dépassement de la bande modérée ET seuil absolu de 50%) garantit qu'aucune migration n'est effectuée dans un cluster globalement sous-chargé. La sélection de la VM cible repose sur le vecteur de transfert du CSLB, et le score pondéré 0.7/0.3 en faveur du CPU reflète le fait que la pression CPU est généralement plus contraignante que la RAM dans les charges de virtualisation.



La figure III.4.3.4 présente le cycle de décision complet du service CLBS. A chaque intervalle de 60 secondes, le service évalue la charge de chaque noeud par rapport aux bandes modérées calculées dynamiquement, sélectionne la VM la mieux adaptée au transfert selon le vecteur de charge, choisit le noeud destination en maximisant le score pondéré CPU/RAM, et déclenche la migration via l'API Proxmox si toutes les conditions sont réunies. Les mécanismes anti-oscillation et le circuit breaker sont également représentés.

III.4.4 Application de Démonstration : Sona-Web

III.4.4.1 Architecture Applicative

Sona-Web est une application web déployée sur le cluster pour démontrer les capacités de haute disponibilité et de live migration. Elle est constituée de quatre VMs Debian 12 interconnectées :

- **HAProxy (VM-201, 192.168.10.111 :8080)** : point d'entrée unique de l'application. Il distribue les requêtes HTTP entre les deux serveurs applicatifs selon l'algorithme round-robin et surveille leur disponibilité toutes les 200 ms (`GET /health`), basculant vers l'instance disponible en moins de 200 ms (`fail 1`) en cas d'indisponibilité [16].
- **Flask app-1 (VM-202, 192.168.10.112 :8000) et app-2 (VM-203, 192.168.10.114 :8000)** : deux instances identiques de l'application Python Flask. Chaque instance enregistre de manière asynchrone en base de données toutes les requêtes reçues.
- **PostgreSQL (VM-204, 192.168.10.115 :5432)** : base de données centralisée dont le disque est stocké sur un volume Ceph RBD, garantissant un historique cohérent quel que soit le nœud hébergeant les VMs applicatives.

Les quatre VMs sont membres du groupe HA `sona-group` et leurs disques résident sur le pool Ceph RBD partagé. Les adresses IP sont configurées de manière statique pour garantir leur stabilité après migration.

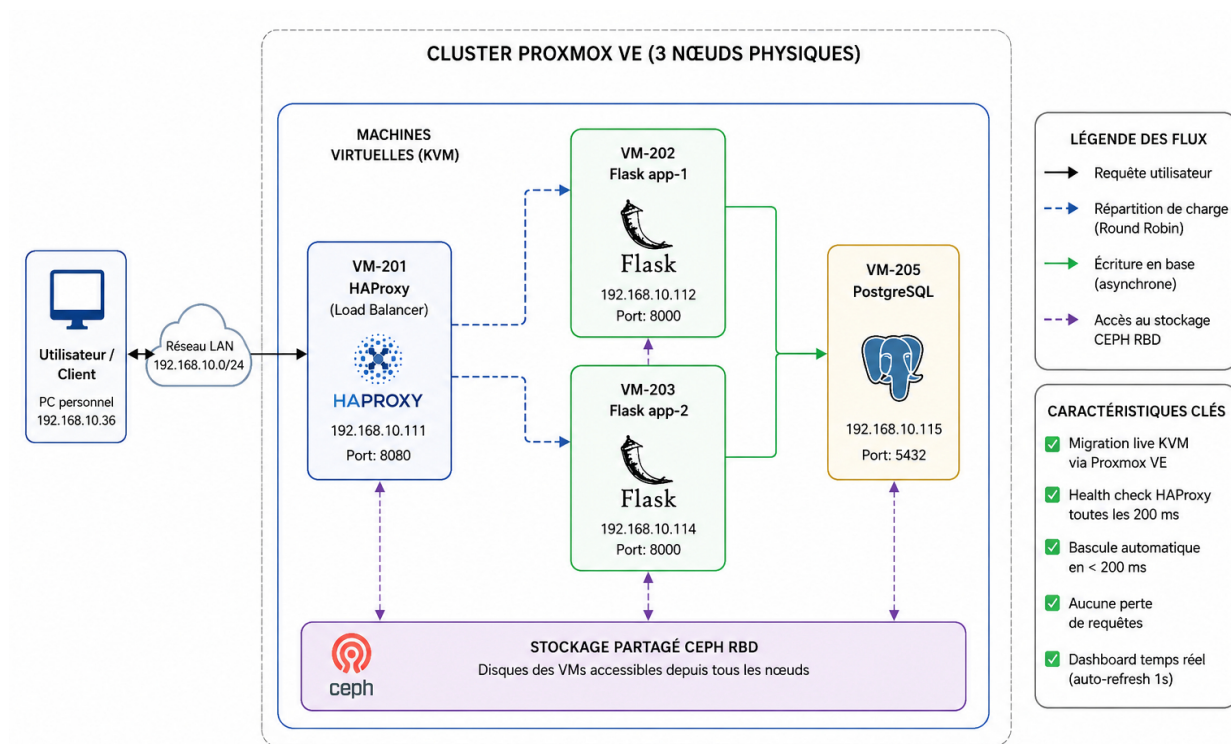


FIGURE III.4 – Architecture Applicative Sona-Web

La figure III.4 illustre l'architecture en quatre tiers de l'application Sona-Web. HAProxy constitue le point d'entrée unique et distribue le trafic entre les deux instances Flask selon un algorithme round-robin. Les deux serveurs applicatifs écrivent dans une base PostgreSQL centralisée dont le disque est stocké sur le pool Ceph RBD partagé, garantissant la cohérence des données quelle que soit la position des VMs dans le cluster.

III.4.4.2 Scénario de Validation HA

Le scénario de validation se déroule en plusieurs temps : affichage de l'alternance round-robin entre app-1 et app-2, puis déclenchement d'une live migration de VM-202 (app-1) vers un autre nœud. Pendant la migration, Proxmox transfère l'état mémoire de la VM sans l'arrêter. Lorsque la synchronisation atteint ~99%, la VM est suspendue quelques dizaines de millisecondes le temps de transférer le delta final, puis reprend sur le nœud cible.

HAProxy détecte l'indisponibilité transitoire en 200 ms maximum et redirige l'intégralité du trafic vers app-2. Une fois la migration terminée, app-1 est remise en service dès le premier health check réussi (rise 1). La requête suivante permet de vérifier la répartition post-migration :

```
SELECT instance_id,
       COUNT(*) AS total
FROM requests
WHERE request_time > NOW() - INTERVAL '1 hour'
GROUP BY instance_id;
```

Figure III.5 – Vérification de la répartition des requêtes en base

III.4.5 Architecture du Monitoring

La stack de monitoring est déployée dans un conteneur LXC dédié à l'adresse `192.168.10.99`. Elle comprend trois composants complémentaires couvrant des périmètres de métriques distincts.

InfluxDB reçoit les métriques générales de Proxmox (CPU, RAM, réseau, stockage des nœuds et des VMs) via le mécanisme natif *Metric Server*, sans agent supplémentaire. Cet historique est utilisé à la fois par Grafana pour la visualisation et par le service CLBS pour ses décisions de migration.

Prometheus collecte les métriques détaillées du cluster Ceph via l'endpoint exposé par le nœud MGR actif sur le port `9283`. Les métriques applicatives du service CLBS sont également exposées sur le port `9101` de la VM dédiée.

Grafana est connecté à InfluxDB et à Prometheus comme sources de données, permettant de construire des tableaux de bord unifiés couvrant l'ensemble de l'infrastructure : utilisation des ressources par nœud, état du cluster Ceph, historique des migrations CLBS, et alertes de surcharge [15].

III.4.6 Automatisation

Le schéma suivant présente l'organisation du pipeline d'automatisation, composé de deux outils complémentaires : Ansible pour la configuration logicielle et l'intégration au cluster, et le pipeline PXE/TFTP pour l'installation système initiale des nœuds physiques. Ces deux mécanismes couvrent l'intégralité du cycle de vie d'un serveur, depuis la mise sous tension jusqu'à son intégration complète dans la plateforme.

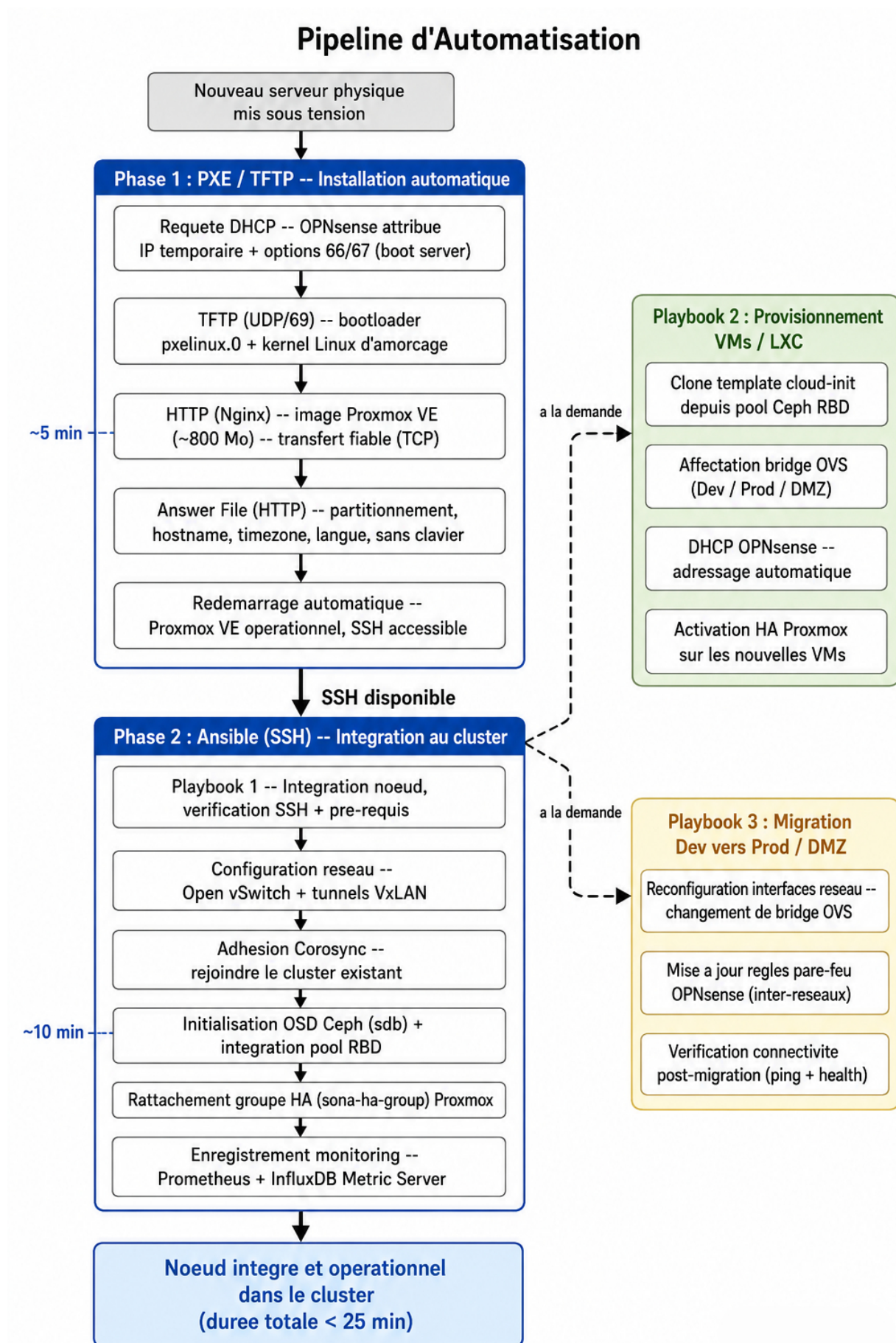


FIGURE III.6 – Pipeline d'Automatisation : PXE/TFTP et Ansible

La figure III.6 illustre le découpage en deux phases du pipeline d'automatisation. La Phase 1, entièrement automatique et opérant en environnement pre-OS, installe Proxmox VE sur le serveur via PXE, TFTP et HTTP sans aucune frappe clavier. La Phase 2 prend le relais dès que le système est démarré, en exécutant le Playbook 1 via SSH pour intégrer le nœud au cluster, configurer les VxLAN, rejoindre Ceph et enregistrer le monitoring. Les Playbooks 2 et 3, déclenchés à la demande, couvrent respectivement le provisionnement de nouvelles VMs depuis les templates cloud-init et la promotion d'environnements vers la production ou la DMZ.

III.4.6.1 Ansible : Périmètre et Playbooks

Ansible est l'outil d'automatisation principal, exécuté depuis les postes de travail des développeurs via SSH sur le réseau de management. Trois playbooks couvrent les principaux besoins d'automatisation.

- **Playbook 1 — Intégration d'un nouveau nœud** : automatise l'ensemble de la procédure d'intégration d'un serveur fraîchement installé (via PXE) dans le cluster existant. Il couvre la configuration réseau OVS/VxLAN, l'adhésion au cluster Corosync, l'initialisation de l'OSD Ceph, et la configuration des exports de métriques vers InfluxDB.
- **Playbook 2 — Provisionnement de VMs** : automatise la création de VMs et de conteneurs LXC à partir de templates cloud-init stockés dans le pool Ceph. Il intègre la configuration réseau via DHCP OPNsense et l'assignation au bon bridge OVS selon le réseau cible (Dev, Prod ou DMZ).
- **Playbook 3 — Migration Dev vers Prod/DMZ** : gère la promotion d'une VM ou d'un ensemble de VMs vers la production ou la DMZ, en reconfigurant les interfaces réseau, les règles de pare-feu OPNsense et en vérifiant la connectivité post-migration.

III.4.6.2 Pipeline PXE/TFTP

Dans une optique de scalabilité, un pipeline de provisionnement automatique en deux phases a été mis en place pour intégrer tout nouveau serveur physique sans intervention manuelle.

Phase 1 — Installation automatique via PXE. Lorsqu'un nouveau serveur est mis sous tension, la séquence suivante s'exécute automatiquement :

1. Le serveur envoie une requête DHCP ; OPNsense lui attribue une adresse temporaire et lui indique l'adresse du serveur de boot ainsi que le fichier bootloader à récupérer (options DHCP 66 et 67).
2. Le firmware PXE contacte le serveur TFTP (UDP/69) pour télécharger le bootloader (`pxelinux.0`) et le kernel Linux d'amorçage. TFTP est le seul protocole utilisable dans un firmware pre-OS.
3. Une fois le kernel démarré en mémoire, l'image d'installation Proxmox VE (~800 Mo) est téléchargée via HTTP depuis un serveur Nginx dédié. HTTP (TCP) est retenu pour ce transfert car TFTP (UDP) est inadapté aux fichiers volumineux.

4. L'installateur Proxmox lit un fichier de réponses (*Answer File*) hébergé sur le serveur HTTP, contenant le partitionnement, le nom d'hôte, le fuseau horaire et la langue. L'installation s'effectue sans aucune frappe clavier.
5. Le serveur redémarre automatiquement sur Proxmox VE fraîchement installé.

Phase 2 — Intégration au cluster via Ansible. À l'issue du redémarrage, le nœud dispose de Proxmox VE fonctionnel mais isolé. Un playbook Ansible prend le relais via SSH pour réaliser l'intégration complète :

- Adhésion au cluster **Corosync** ;
- Configuration des interfaces **Open vSwitch** et des tunnels **VxLAN** ;
- Intégration des disques physiques comme **OSDs** dans le cluster **Ceph** ;
- Rattachement au groupe HA Proxmox (**sona-ha-group**) ;
- Déclaration du nœud comme cible dans **Prometheus**.

Ce découpage en deux phases répond à une contrainte fondamentale : PXE et TFTP interviennent dans un environnement pre-OS où aucun client SSH ne peut fonctionner, tandis qu'Ansible nécessite un système d'exploitation opérationnel. L'ensemble du pipeline permet l'ajout d'un nouveau nœud en moins de 25 minutes, sans intervention manuelle.

III.5 Partie II : Stockage

Cette partie détaille la couche de stockage distribuée de l'infrastructure, reposant entièrement sur Ceph. Elle expose l'architecture du cluster Ceph avec ses composants Monitors, Managers et OSDs, le dimensionnement du pool de stockage en tenant compte du facteur de réplication, et la configuration de l'algorithme CRUSH qui garantit la distribution des répliques sur des domaines de défaillance physiquement distincts.

III.5.1 Architecture du Cluster Ceph

III.5.1.1 Composants du Cluster

Le cluster Ceph est composé des services suivants, répartis sur les trois nœuds physiques :

- **Monitors (MON)** : un monitor par nœud, soit trois au total. Les monitors maintiennent la carte du cluster (*cluster map*) et assurent le consensus distribué. Trois monitors garantissent un quorum stable (2/3 suffisants).
- **Managers (MGR)** : PVE-02 héberge le MGR actif, PVE-03 le MGR en standby. Le MGR expose les métriques du cluster via un endpoint REST utilisé par Prometheus.
- **OSDs (Object Storage Daemons)** : quatre OSDs par nœud (disques **sdb**, **sdc**, **sdd** et **sde**), soit douze OSDs au total dans le cluster [19].

III.5.1.2 Dimensionnement du Pool

Avec un facteur de réplication de 3, la capacité utilisable est calculée ainsi :

$$\text{Capacité utilisable} = \frac{3 \times (300 + 3 \times 600) \text{ GB}}{3} = \frac{3 \times 2100 \text{ GB}}{3} = 2100 \text{ GB} \approx 2,05 \text{ TB}$$

L'algorithme CRUSH est configuré pour respecter les domaines de défaillance au niveau du nœud physique, garantissant que les trois répliques d'un même bloc ne se retrouvent jamais sur le même serveur. Le tableau suivant présente la répartition prévue :

TABLE III.4 – Répartition du stockage Ceph par usage

Usage	Stockage alloué
VM OPNsense	20 GB
HAProxy VM	16 GB
Flask VM 1	32 GB
Flask VM 2	32 GB
PostgreSQL VM	100 GB
VM Monitoring (InfluxDB + Prometheus + Grafana + Loki)	150 GB
VM CLBS	32 GB
LXC GitLab	200 GB
LXC Rundeck	16 GB
LXC OpenLDAP	16 GB
LXC NTP	8 GB
Marge, snapshots et templates	~300 GB
Total estimé	~922 GB (thin)

Le total estimé reste inférieur à la capacité utilisable de 2 100 GB car le thin provisioning de Ceph RBD ne consomme l'espace que proportionnellement aux données réellement écrites.

III.5.1.3 Configuration CRUSH et Domaines de Défaillance

Lors de la création du cluster Ceph via Proxmox VE, une règle CRUSH par défaut est générée automatiquement, organisant les OSDs selon la hiérarchie `root > host > osd` avec la politique `chooseleaf firstn 0 type host`. Cela garantit que les trois répliques d'un objet sont placées sur trois hôtes physiques distincts.

TABLE III.5 – Paramètres CRUSH appliqués automatiquement par Proxmox

Paramètre	Valeur
Domaine de défaillance	<code>host</code> — généré automatiquement par Proxmox
Facteur de réplication (<code>size</code>)	3 — une réplique par nœud physique
<code>min_size</code>	2 (défaut Ceph) — pool accessible avec 2 nœuds opérationnels

III.6 Partie III : Réseau

Cette partie décrit la couche réseau de l'infrastructure, conçue pour contourner les limitations du réseau physique Sonatrach. Elle présente les contraintes réseau du site et la solution retenue basée sur VxLAN et Open vSwitch, la topologie en deux niveaux de segmentation (réseaux d'infrastructure et réseaux applicatifs), le plan d'adressage complet, la VM OPNsense assurant le routage centralisé et le filtrage, et l'architecture de pare-feu à deux niveaux couvrant à la fois OPNsense et le pare-feu natif Proxmox.

III.6.1 Contexte et Contraintes Réseau

Le réseau Sonatrach met à disposition un sous-réseau `192.168.10.0/24` utilisé pour l'accès SSH et l'interface web Proxmox. La connectivité Internet des nœuds est assurée par un poste jouant le rôle de routeur externe à l'adresse `192.168.10.36`, qui réalise du NAT et du forwarding IP.

Le switch Sonatrach n'autorise que le VLAN 10 natif et ne permet pas la configuration de VLANs supplémentaires. Pour créer les réseaux logiques nécessaires à l'infrastructure, **VxLAN (Virtual Extensible LAN)** a été retenu comme technologie d'encapsulation. VxLAN permet de créer des réseaux logiques de niveau 2 tunnelisés au-dessus du réseau IP existant, sans modifier la configuration du switch. Open vSwitch gère la création et la terminaison de ces tunnels sur chaque nœud Proxmox.

III.6.2 Topologie Réseau

III.6.2.1 Deux Niveaux de Segmentation

La topologie retenue s'articule autour de deux niveaux complémentaires.

Niveau 1 — Réseaux d'infrastructure (VxLAN inter-nœuds). Pour les réseaux de contrôle (Corosync, Ceph Cluster, Ceph Public, Live Migration), des tunnels VxLAN point-à-point en maillage complet sont établis entre les trois nœuds. Chaque nœud crée deux ports VxLAN vers chacun des deux autres, garantissant une connectivité directe sans point de transit central.

Niveau 2 — Réseaux applicatifs (OPNsense). Pour les réseaux Dev, Prod et DMZ, une architecture centralisée autour d'une VM OPNsense est utilisée. Trois bridges OVS internes sont configurés sur chaque nœud. OPNsense dispose de quatre interfaces virtuelles : WAN connectée au réseau de management (`192.168.10.65`, gateway `192.168.10.36`), et trois interfaces LAN connectées aux bridges Dev, Prod et DMZ.

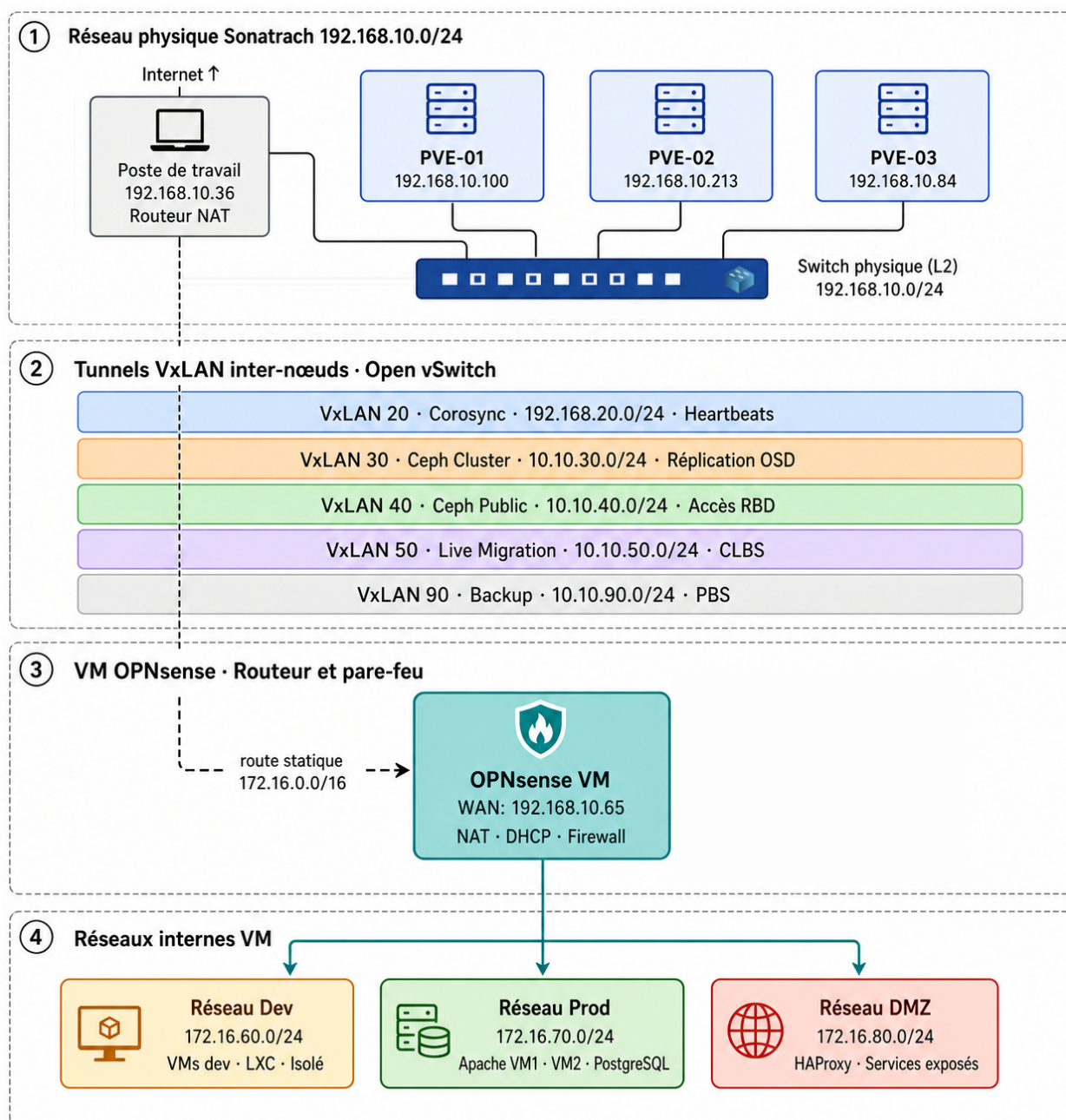


FIGURE III.7 – Topologie Réseau et Plan VxLAN

La figure III.7 présente les deux niveaux de segmentation réseau retenus. Le premier niveau repose sur des tunnels VxLAN point-à-point en maillage complet entre les trois nœuds physiques, dédiés aux flux de contrôle (Corosync, Ceph, migrations). Le second niveau utilise OPNsense comme routeur centralisé pour les réseaux applicatifs Dev, Prod et DMZ, chacun associé à un bridge OVS distinct sur chaque nœud.

III.6.2.2 Bridges Open vSwitch par Nœud

TABLE III.6 – Bridges OVS et tunnels VxLAN par nœud Proxmox

Bridge OVS	VxLAN associé	Sous-réseau	Rôle
vmbr0	Réseau physique natif	192.168.10.0/24	Bridge principal. Accès management, SSH, Web UI Proxmox, interface WAN d'OPNsense.
vmbr60	vxlan60-int	172.16.60.0/24	Réseau Dev. OPNsense y expose son interface LAN1.
vmbr70	vxlan70-int	172.16.70.0/24	Réseau Prod. OPNsense y expose son interface LAN2.
vmbr80	vxlan80-int	172.16.80.0/24	Réseau DMZ. OPNsense y expose son interface LAN3.

III.6.3 Plan d'Adressage

Le plan d'adressage est organisé en deux familles : les réseaux VxLAN d'infrastructure pour les flux de contrôle, et les réseaux applicatifs internes gérés par OPNsense.

Réseaux VxLAN d'infrastructure :

- **VxLAN 10 — Management (192.168.10.0/24)** : Web UI Proxmox, SSH, API (réseau physique existant).
- **VxLAN 20 — Corosync (192.168.20.0/24)** : Heartbeats et quorum cluster.
- **VxLAN 30 — Ceph Cluster (10.10.30.0/24)** : Réplication OSD interne.
- **VxLAN 40 — Ceph Public (10.10.40.0/24)** : Accès des VMs au stockage RBD.
- **VxLAN 50 — Live Migration (10.10.50.0/24)** : Migrations à chaud et CLBS.

Réseaux applicatifs internes :

- **Réseau Dev (172.16.60.0/24, gateway 172.16.60.1)** : environnements de développement et de test.
- **Réseau Prod (172.16.70.0/24, gateway 172.16.70.1)** : environnements de production.
- **Réseau DMZ (172.16.80.0/24, gateway 172.16.80.1)** : services exposés, dont HAProxy.

TABLE III.7 – Adressage IP statique des nœuds Proxmox par réseau logique

Nœud	Management	VxLAN 20	VxLAN 30	VxLAN 40	VxLAN 50
PVE-01	192.168.10.100	192.168.20.11	10.10.30.11	10.10.40.11	10.10.50.11
PVE-02	192.168.10.213	192.168.20.12	10.10.30.12	10.10.40.12	10.10.50.12
PVE-03	192.168.10.84	192.168.20.13	10.10.30.13	10.10.40.13	10.10.50.13

III.6.4 VM OPNsense : Routeur et Pare-feu Centralisé

III.6.4.1 Interfaces et Rôles

La VM OPNsense constitue le cœur du réseau applicatif. Il s'agit d'une distribution FreeBSD dédiée aux fonctions de routage et de sécurité réseau. Dans cette architecture, OPNsense est déployée avec quatre interfaces réseau virtuelles :

- **Interface WAN (192.168.10.65/24)** : connectée au bridge OVS de management, avec gateway 192.168.10.36. Assure l'accès Internet sortant via NAT pour toutes les VMs internes.
- **Interface LAN1 (172.16.60.1/24)** : passerelle du réseau Dev.
- **Interface LAN2 (172.16.70.1/24)** : passerelle du réseau Prod.
- **Interface LAN3 (172.16.80.1/24)** : passerelle du réseau DMZ.

OPNsense intègre nativement dnsmasq, utilisé comme serveur DHCP pour les trois réseaux internes. L'attribution des adresses IP aux VMs est ainsi automatisée dès leur connexion à l'un des bridges applicatifs, simplifiant le provisionnement via les playbooks Ansible.

III.6.4.2 Haute Disponibilité d'OPNsense

OPNsense étant le point central du routage et du filtrage, sa défaillance isolerait l'ensemble des réseaux internes. Sa résilience est assurée par son intégration au groupe HA Proxmox. En cas de défaillance du nœud hébergeant la VM, Corosync la détecte et le HA Manager déclenche son redémarrage sur un nœud survivant, directement depuis le volume Ceph RBD. Le délai de basculement est de 60 à 120 secondes.

Pour une architecture de production nécessitant un basculement transparent, une configuration active/passive avec le protocole CARP (natif dans OPNsense) réduirait ce délai à moins de 5 secondes.

III.6.5 Architecture du Pare-feu

III.6.5.1 Pare-feu OPNsense : Filtrage Inter-réseaux

OPNsense applique les politiques de filtrage entre les réseaux applicatifs Dev, Prod et DMZ. Les règles de conception retenues sont les suivantes :

Règles autorisées :

- Dev → WAN : autorisé pour les mises à jour et les dépôts de paquets.
- DMZ → WAN : autorisé pour les services exposés nécessitant un accès sortant.
- Prod → DMZ : autorisé pour la communication entre production et services exposés (HAProxy).

Règles bloquées :

- Dev → Prod : bloqué pour isoler les environnements de développement de la production.
- Dev → DMZ : bloqué pour empêcher l'accès aux services exposés depuis le développement.
- Prod → WAN direct : bloqué, le trafic de production passe obligatoirement par la DMZ.

III.6.5.2 Pare-feu Proxmox : Filtrage au Niveau Hyperviseur

Proxmox VE intègre nativement un mécanisme de pare-feu basé sur iptables, configurable aux niveaux cluster, nœud et VM individuelle. Les règles de conception retenues pour protéger la plateforme sont les suivantes :

- **Isolation des réseaux Ceph** : interdire tout trafic des réseaux applicatifs (172.16.x.0/24) vers les réseaux Ceph Cluster (VxLAN 30) et Ceph Public (VxLAN 40).
- **Isolation du réseau Corosync** : interdire tout accès extérieur au VxLAN 20. Seuls les nœuds Proxmox communiquent sur ce réseau.
- **Protection du management** : restreindre l'accès à l'interface web Proxmox (port 8006) et SSH (port 22) aux seules adresses du sous-réseau 192.168.10.0/24.
- **Moindre privilège par VM** : n'autoriser que les ports applicatifs strictement nécessaires sur chaque VM.

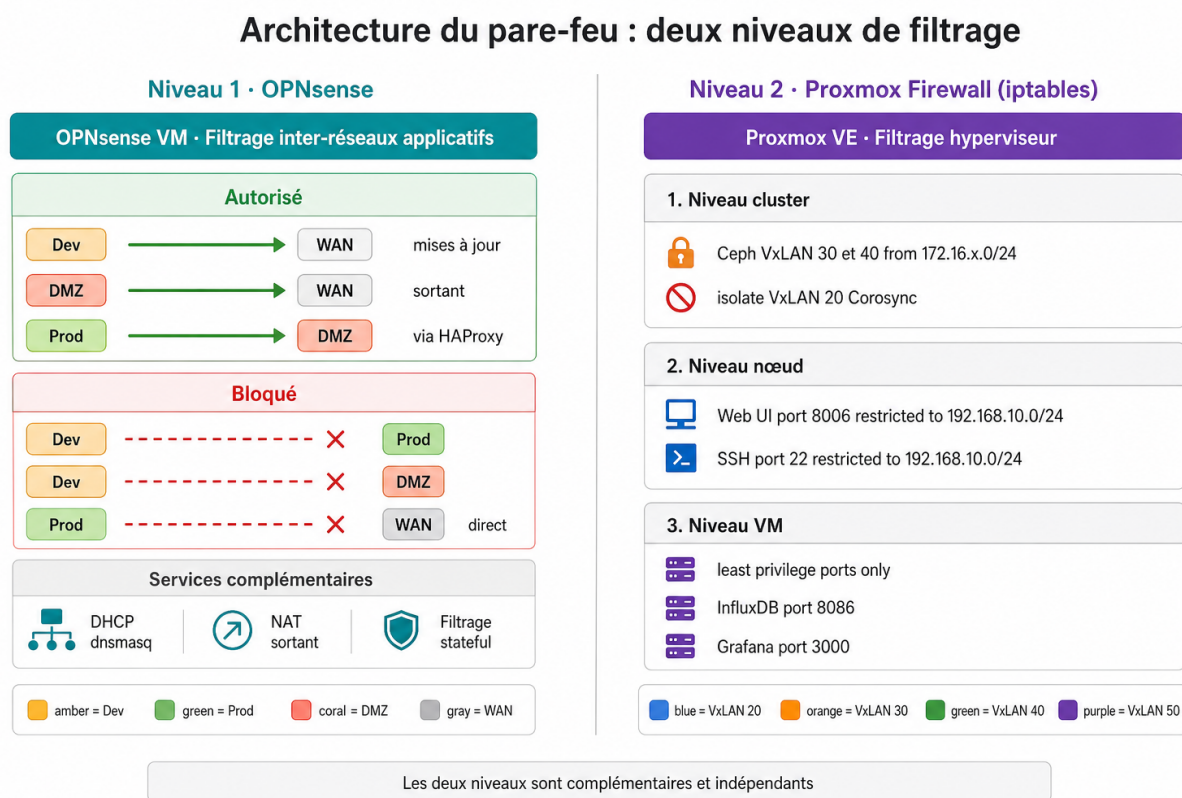


FIGURE III.8 – Architecture du Pare-feu : OPNsense et Proxmox Firewall

La figure III.8 représente les deux niveaux de filtrage coexistant dans l'infrastructure. OPNsense assure le filtrage inter-réseaux entre Dev, Prod et DMZ selon des politiques d'isolation applicative. Le pare-feu Proxmox, basé sur iptables, opère au niveau de l'hyperviseur pour protéger les réseaux d'infrastructure (Ceph, Corosync) et restreindre l'accès à l'interface de management. Cette défense en profondeur garantit qu'aucune VM applicative ne puisse atteindre les réseaux de contrôle du cluster.

III.7 Services Complémentaires d'Infrastructure

Cette section présente les services transverses qui complètent les trois couches de base (calcul, stockage, réseau). Ils sont déployés sous forme de conteneurs LXC sur le cluster Proxmox et couvrent la supervision avancée, la synchronisation temporelle, la gestion des identités, la gestion du code source et l'automatisation opérationnelle.

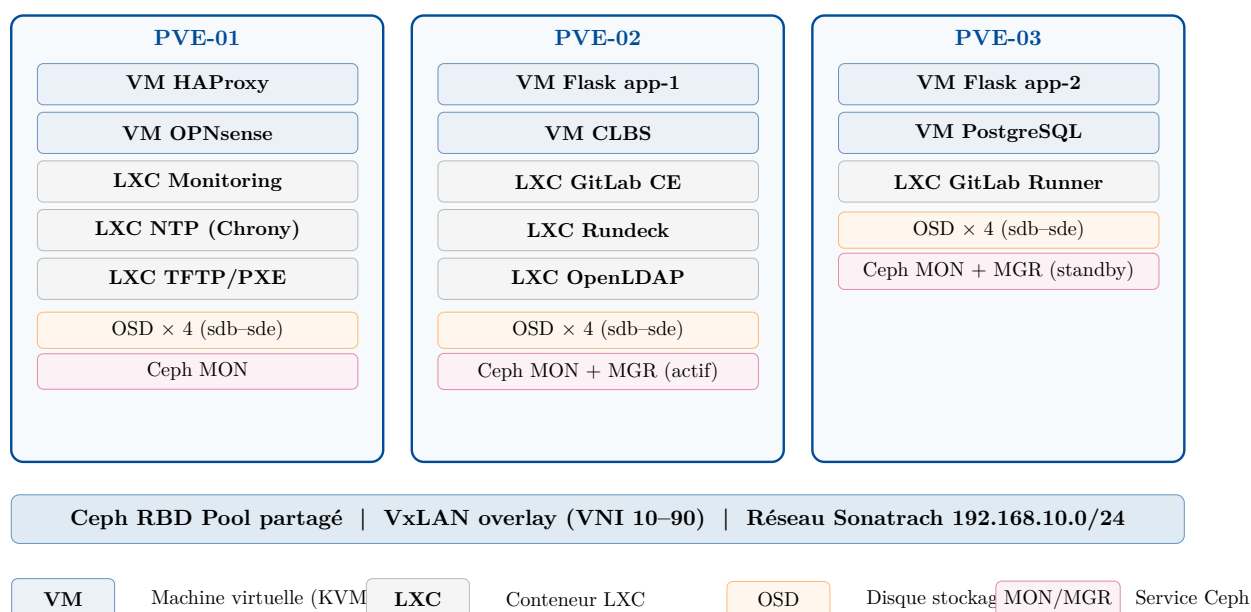


FIGURE III.9 – Diagramme de déploiement : répartition des services sur les trois nœuds physiques

La figure III.9 présente la répartition des services virtualisés sur les trois nœuds physiques du cluster. PVE-01 concentre les services réseau et de monitoring. PVE-02 héberge les services applicatifs de développement (GitLab, Rundeck, OpenLDAP) et le service CLBS. PVE-03 prend en charge les instances de l'application de démonstration et le GitLab Runner. Chaque nœud contribue avec quatre OSDs au pool Ceph partagé et héberge un Monitor Ceph. Les Managers actif et standby sont respectivement sur PVE-02 et PVE-03.

III.7.1 Système de Notification : Grafana Alertmanager

Grafana intègre nativement un système d'alerte permettant de définir des règles de déclenchement directement sur les tableaux de bord existants. Lorsqu'une condition est vérifiée, Grafana génère une alerte et la transmet via un contact point configuré. Dans notre infrastructure, les notifications sont acheminées vers une adresse Gmail via le protocole SMTP [15].

Les alertes couvrent les métriques critiques suivantes :

- **Cluster Ceph** : OSD en état down, pool en état degraded, espace disponible inférieur à 20 %.
- **Nœuds Proxmox** : charge CPU supérieure à 85 %, mémoire disponible inférieure à 10 %, perte de quorum Corosync.
- **Service CLBS** : nombre de migrations excessif déclenchant le circuit breaker, impossibilité de contacter l'API Proxmox ou InfluxDB.
- **Application Sona-Web** : backend HAProxy hors service, taux d'erreurs HTTP 5xx anormal.

À terme, Apprise est envisagé comme couche d'abstraction de notification, permettant d'acheminer les alertes vers plusieurs canaux (email, Discord, Slack) sans dépendre d'un serveur SMTP dédié.

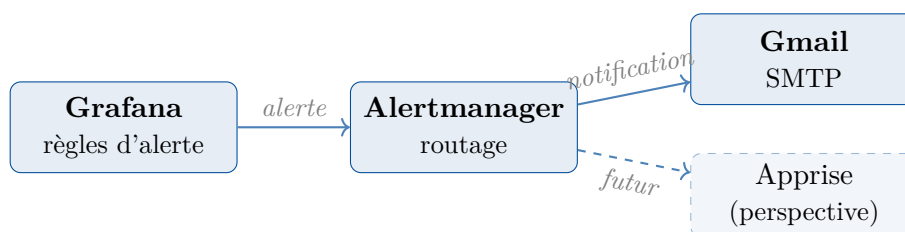


FIGURE III.10 – Pipeline de notification Grafana Alertmanager

La figure III.10 présente le pipeline de notification. Grafana évalue les règles d'alerte définies sur les tableaux de bord et transmet les déclenchements à l'Alertmanager intégré, qui les achemine vers le contact point configuré (Gmail via SMTP). La liaison vers Apprise est prévue en extension pour supporter plusieurs canaux de notification sans serveur SMTP dédié.

III.7.2 Centralisation des Journaux : Loki, Alloy et Grafana

La centralisation des journaux repose sur la pile Loki, Alloy et Grafana. Loki est une solution de stockage de logs qui indexe uniquement les métadonnées (labels) et stocke les messages compressés, ce qui lui confère une empreinte en ressources nettement inférieure à celle d'Elasticsearch. Grafana Alloy (anciennement Grafana Agent) est l'agent de collecte déployé sur chaque nœud et VM ; il transmet les logs vers Loki. Grafana permet ensuite d'interroger et de visualiser ces journaux via le langage LogQL.

Alloy est installé sur chaque nœud Proxmox et sur les VMs à superviser. Loki est hébergé dans le conteneur LXC monitoring existant, aux côtés d'InfluxDB et de Prometheus.

Les services retenus pour la collecte des journaux sont les suivants :

- **Nœuds Proxmox** : journaux système (`auth.log`, `syslog`, `kern.log`) pour la sécurité et le diagnostic matériel.
- **Cluster Ceph** : journaux des démons OSD et MON pour le suivi des événements de santé, de récupération et de rebalancing.

- **OPNsense** : journaux du pare-feu (connexions bloquées, règles déclenchées) et du service DHCP.
- **HAProxy** : journaux d'accès HTTP et événements de health check.
- **Service CLBS** : décisions de migration, activations du circuit breaker, erreurs de connexion à l'API Proxmox.
- **Authentication SSH** : tentatives de connexion et échecs d'authentification sur les nœuds et VMs (sécurité).
- **GitLab** : événements d'accès au dépôt et déclenchements de pipelines CI/CD.
- **Rundeck** : exécutions de jobs et résultats des playbooks Ansible.

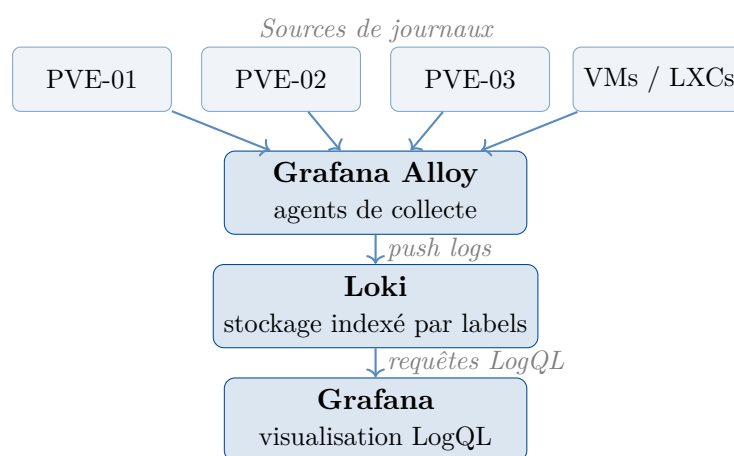


FIGURE III.11 – Architecture de centralisation des journaux : Alloy, Loki et Grafana

La figure III.11 présente l'architecture de collecte centralisée des journaux. Les agents Alloy, déployés sur chaque nœud Proxmox et sur les VMs, collectent les journaux et les transmettent vers Loki par push. Loki stocke les messages compressés en indexant uniquement leurs labels. Grafana interroge Loki via LogQL et affiche les journaux dans les mêmes tableaux de bord que les métriques, offrant une vue unifiée de l'infrastructure.

III.7.3 Synchronisation Temporelle : Serveur NTP

La synchronisation temporelle est un prérequis pour tout système distribué. Dans notre infrastructure, Ceph et Corosync dépendent de l'horloge système de chaque nœud : un écart supérieur à quelques secondes peut provoquer des erreurs de quorum Ceph, des incohérences dans les journaux distribués ou des refus d'authentification.

Un serveur NTP interne est déployé dans un conteneur LXC sur le cluster. L'implémentation retenue est **Chrony**, plus réactif que `ntpd` classique pour les corrections de dérive d'horloge. Le serveur Chrony se synchronise sur des serveurs NTP publics (`pool.ntp.org`) et constitue la référence temporelle unique pour l'ensemble des nœuds Proxmox et des VMs. Les nœuds sont configurés comme clients NTP pointant exclusivement vers ce serveur interne.

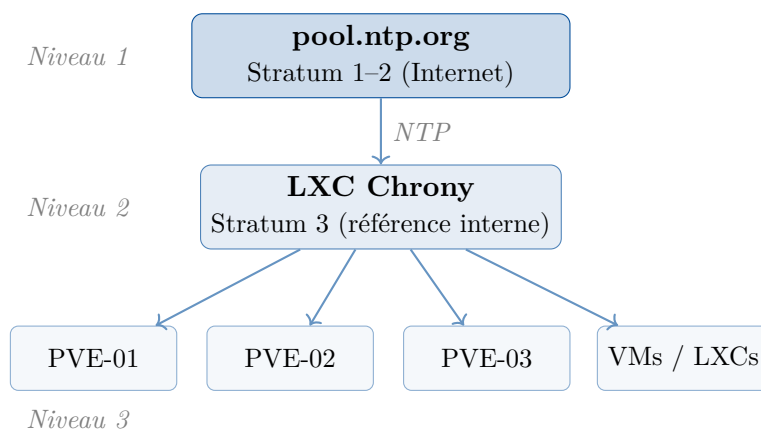


FIGURE III.12 – Hiérarchie NTP : synchronisation temporelle de l'infrastructure

La figure III.12 présente la hiérarchie de synchronisation temporelle. Le serveur Chrony interne (Stratum 3) se synchronise sur les serveurs publics du pool NTP. Tous les nœuds Proxmox et VMs du cluster utilisent ce serveur comme unique source de temps, garantissant la cohérence temporelle requise par Ceph et Corosync.

III.7.4 Gestion des Identités : OpenLDAP

OpenLDAP est retenu comme solution d'annuaire pour centraliser la gestion des identités de l'infrastructure. Bien que non encore déployé à ce stade du projet, OpenLDAP est prévu pour unifier l'authentification des services suivants : interface web Proxmox, Grafana, GitLab et Rundeck. Cette centralisation permet de gérer les accès depuis un point unique et d'appliquer des politiques de contrôle d'accès cohérentes sur l'ensemble des outils.

OpenLDAP sera déployé dans un conteneur LXC dédié. L'intégration avec les services se fera via le protocole LDAP standard, supporté nativement par Proxmox, Grafana et GitLab. Un schéma d'annuaire adapté à l'infrastructure sera défini, avec des groupes distincts pour les rôles administrateur, opérateur et lecture seule.

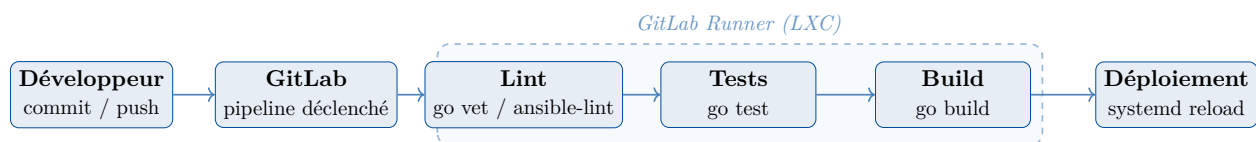
III.7.5 Gestion du Code Source et CI/CD : GitLab

GitLab est déployé sous forme de conteneur LXC Turnkey Linux sur le cluster Proxmox. Le template Turnkey GitLab repose sur Debian 12 et intègre GitLab Community Edition version 17, antérieure à la dernière version disponible (18.x au moment de la rédaction). Une mise à jour vers la dernière version sera réalisée après le déploiement initial, en suivant la procédure officielle de mise à niveau par paliers [21].

Les projets hébergés sur cette instance GitLab sont les suivants :

- **clbs** : code source du service d'équilibrage de charge en Go.
- **ansible-infra** : ensemble des playbooks Ansible d'automatisation de l'infrastructure.

Des pipelines CI/CD GitLab sont configurés pour automatiser les vérifications de qualité de code et les tests unitaires du service CLBS, ainsi que la validation des syntaxes des playbooks Ansible. Les GitLab Runners sont déployés sous forme de conteneurs LXC sur le cluster et enregistrés dans l'instance GitLab.

FIGURE III.13 – Pipeline CI/CD GitLab pour les projets `clbs` et `ansible-infra`

La figure III.13 présente le pipeline CI/CD configuré pour les deux projets hébergés sur l'instance GitLab. À chaque push, GitLab déclenche automatiquement le pipeline. Les trois stages (Lint, Tests, Build) s'exécutent sur le GitLab Runner déployé en LXC sur le cluster. Le stage de déploiement met à jour le service `systemd` correspondant sur la VM cible.

III.7.6 Automatisation Opérationnelle : Rundeck

Rundeck est un outil d'automatisation opérationnelle qui expose les playbooks Ansible via une interface web. Il permet aux administrateurs d'exécuter des jobs prédéfinis sans accès direct à la ligne de commande, d'historiser les exécutions avec leurs journaux complets, et de contrôler les permissions d'exécution par utilisateur ou groupe.

Dans notre infrastructure, Rundeck est déployé dans un conteneur LXC et connecté au dépôt `ansible-infra` hébergé sur GitLab. Les playbooks de provisionnement de VMs, d'intégration de nœuds et de promotion d'environnements sont ainsi accessibles via une interface graphique, sans nécessiter d'accès SSH direct aux nœuds Proxmox.

III.8 Conclusion

Ce chapitre a présenté la conception complète de l'infrastructure Cloud privée hyperconvergée. La partie Calcul couvre les serveurs physiques (5 disques SAS par nœud), le cluster Proxmox avec Corosync et HA, le mécanisme CLBS, l'application Sona-Web, la stack de monitoring et l'automatisation via Ansible et PXE. La partie Stockage détaille le cluster Ceph avec ses douze OSDs, la capacité utilisable de 2,05 TB et la politique CRUSH. La partie Réseau expose l'architecture VxLAN gérée par Open vSwitch, le plan d'adressage complet, le routage centralisé via OPNsense et le plan de pare-feu à deux niveaux. Enfin, les services complémentaires (Alertmanager, Loki, NTP, OpenLDAP, GitLab, Rundeck) complètent l'infrastructure pour en assurer la supervision, la sécurité et la gestion unifiée.

L'ensemble de ces choix de conception constitue une base cohérente et documentée pour la phase d'implémentation présentée dans le chapitre suivant.

Chapitre IV

Implémentation de la Solution Propo- sée

IV.1 Introduction

Ce chapitre présente la mise en œuvre concrète de la solution proposée dans le chapitre précédent. L’objectif est de montrer les étapes réelles de déploiement de l’infrastructure hyperconvergée, depuis l’installation de la plateforme de virtualisation jusqu’aux services applicatifs et aux mécanismes d’automatisation. Pour améliorer la lisibilité, l’implémentation est organisée en trois grandes parties : la couche calcul, la couche stockage, puis la couche réseau.

Les opérations ont été réalisées avec plusieurs outils complémentaires : l’interface web Proxmox VE, la ligne de commande Linux, les fichiers de configuration système, OPNsense, Ceph, ainsi que des playbooks Ansible pour automatiser certaines tâches répétitives. Des captures d’écran sont prévues aux points importants afin d’illustrer visuellement les résultats obtenus.

IV.2 Partie I – Couche Calcul

Cette partie couvre l’ensemble des opérations réalisées au niveau de la couche de calcul de l’infrastructure : installation de Proxmox VE, formation du cluster, déploiement du monitoring, implémentation du mécanisme CLBS, et déploiement des services applicatifs.

IV.2.1 Préparation de l’Infrastructure Physique

La préparation de l’infrastructure physique constitue la première étape du déploiement. Elle comprend l’installation de Proxmox VE sur chaque nœud ainsi que la configuration de la connectivité réseau nécessaire au bon fonctionnement du cluster.

IV.2.1.1 Installation de Proxmox VE sur les nœuds

L’installation de la plateforme de virtualisation a commencé par le téléchargement de l’image ISO de Proxmox VE version 9.1.1. Cette image a ensuite été copiée sur une clé USB bootable utilisée pour démarrer chaque serveur Dell PowerEdge R720. Une installation séparée a été réalisée sur chacun des trois nœuds du cluster.

Pendant l’installation, le disque système `sda` a été utilisé pour héberger Proxmox VE. Une configuration réseau statique a aussi été appliquée afin de donner à chaque nœud une adresse de management fixe sur le réseau physique `192.168.10.0/24`. Les adresses retenues sont `192.168.10.100` pour PVE-01, `192.168.10.213` pour PVE-02 et `192.168.10.84` pour PVE-03.

Une fois l’installation terminée, la vérification a été effectuée à travers l’interface web de Proxmox, accessible sur le port `8006`. Comme le montre la figure [IV.1](#), cette étape a permis de confirmer que chaque nœud était opérationnel et prêt pour la suite du déploiement.

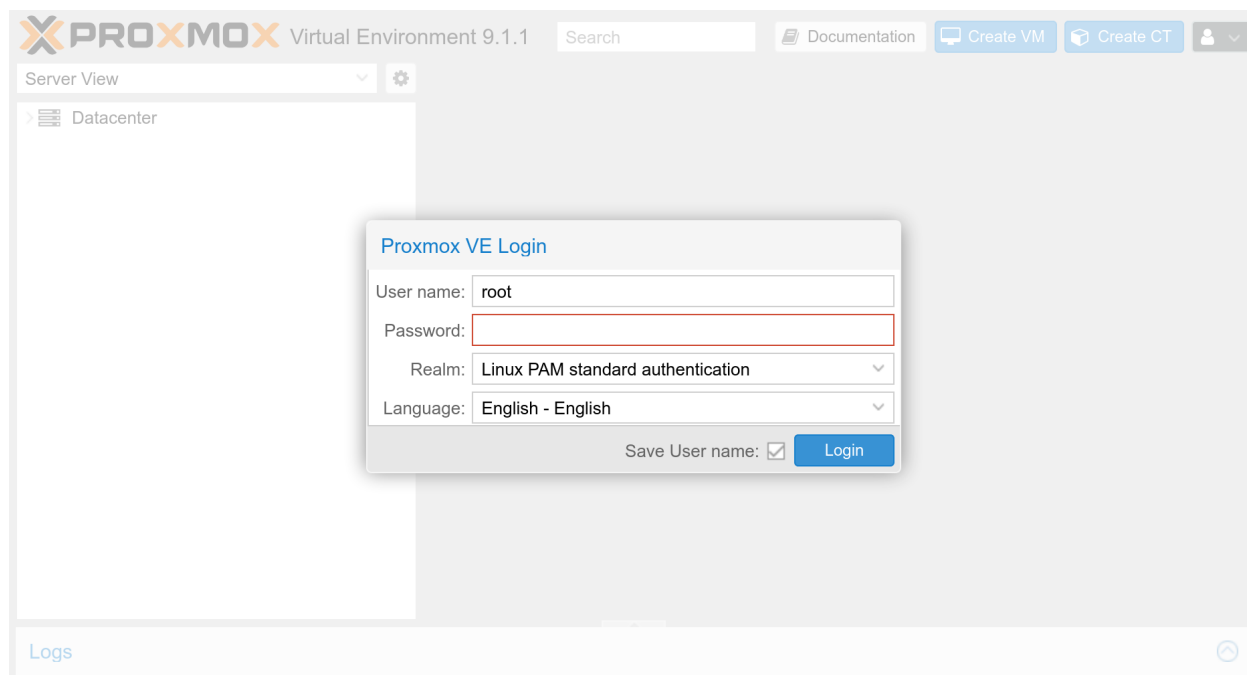


FIGURE IV.1 – Interface web Proxmox au premier login sur PVE-01

La figure IV.1 montre l'interface web de Proxmox VE après le premier démarrage sur PVE-01. On y retrouve l'écran de connexion avec les champs d'authentification, confirmant que le service web est actif et accessible sur le port 8006 du réseau de management.

IV.2.1.2 Configuration de la connectivité Internet via le laptop gateway

Dans l'environnement de laboratoire, aucun routeur physique dédié n'était disponible pour fournir l'accès Internet aux nœuds Proxmox. Pour cette raison, un laptop a été configuré pour jouer le rôle de passerelle NAT entre le réseau interne de l'infrastructure et Internet. Il possède deux interfaces : `enp0s20f0u4c2` côté infrastructure avec l'adresse `192.168.10.36/24`, et `wlan0` côté Internet avec l'adresse `10.247.78.8/24`.

La première étape consiste à activer le routage IP afin que le système puisse transférer les paquets d'une interface à l'autre. Cette activation doit être faite de manière immédiate puis rendue permanente.

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo vim /etc/sysctl.d/99-ipforward.conf
# Ajouter :
net.ipv4.ip_forward=1
```

Figure IV.2 – Activation du routage IP sur le laptop

La première commande active le forwarding IP immédiatement, la seconde le rend permanent via le fichier `99-ipforward.conf`. Sans cette étape, le laptop ne peut pas transmettre les paquets entre ses deux interfaces réseau.

Ensuite, des règles `iptables` ont été ajoutées pour réaliser la translation d'adresses NAT et autoriser le forwarding du trafic entre les interfaces du laptop.

```
sudo iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE
sudo iptables -A FORWARD -i enp0s20f0u4c2 -o wlan0 -j ACCEPT
sudo iptables -A FORWARD -i wlan0 -o enp0s20f0u4c2 -m state --state
    RELATED,ESTABLISHED -j ACCEPT
```

Figure IV.3 – Configuration NAT avec iptables

Ces trois règles assurent respectivement : le masquage des adresses sources sur l'interface Internet, l'autorisation du trafic sortant, et l'autorisation des réponses correspondant à des connexions établies. Ensemble, elles permettent aux nœuds Proxmox d'atteindre Internet à travers le laptop.

Pour éviter de perdre ces règles après redémarrage, elles ont été sauvegardées puis activées automatiquement au boot du laptop.

```
sudo pacman -S iptables-nft
sudo iptables-save | sudo tee /etc/iptables/iptables.rules
sudo systemctl enable iptables
```

Figure IV.4 – Persistance des règles iptables

La sauvegarde dans `/etc/iptables/iptables.rules` combinée à l'activation du service garantit que le NAT sera opérationnel dès le démarrage du laptop, sans intervention manuelle à chaque redémarrage.

Sur chaque nœud Proxmox, la passerelle par défaut a été définie vers `192.168.10.36`, avec des serveurs DNS publics. Cette configuration a été ajoutée dans le fichier `/etc/network/interfaces`.

```
auto vmbr0
iface vmbr0 inet static
    address 192.168.10.XX/24
    gateway 192.168.10.36
    dns-nameservers 8.8.8.8 8.8.4.4
```

Figure IV.5 – Configuration réseau de base d'un nœud Proxmox

Cette configuration attribue une adresse IP statique à chaque nœud, définit le laptop comme passerelle par défaut et utilise les DNS publics de Google. Le paramètre `XX` est remplacé par la valeur spécifique à chaque nœud (100, 213 ou 84).

Enfin, des routes statiques ont été ajoutées sur le laptop afin de rediriger les réseaux internes `172.16.60.0/24`, `172.16.70.0/24` et `172.16.80.0/24` vers la VM OPNsense, qui assure ensuite le routage vers ces sous-réseaux.

```
nmcli connection modify "Wired connection 1" \
+ipv4.routes "172.16.60.0/24 192.168.10.65" \
+ipv4.routes "172.16.70.0/24 192.168.10.65" \
+ipv4.routes "172.16.80.0/24 192.168.10.65"

nmcli connection up "Wired connection 1"
```

Figure IV.6 – Ajout des routes statiques vers OPNsense

Ces routes indiquent au laptop que tout trafic destiné aux réseaux internes (172.16.x.0/24) doit être transmis vers l'adresse IP d'OPNsense (192.168.10.65), qui se charge du routage vers la zone applicative appropriée.

Des tests simples comme `ping 8.8.8.8`, `ping google.com` et `apt update` ont confirmé que les nœuds disposaient correctement d'un accès Internet.

IV.2.2 Formation du cluster Proxmox et haute disponibilité

Après avoir installé et connecté les trois nœuds à Internet, l'étape suivante a consisté à les regrouper en un cluster unifié et à configurer les mécanismes de haute disponibilité.

IV.2.2.1 Création et jonction au cluster

Après l'installation de Proxmox VE sur les trois serveurs, la phase suivante a consisté à former un cluster unique. Cette opération a été lancée depuis le premier nœud PVE-01, qui devient le nœud initial du cluster.

```
pvecm create Cluster-Proxmox-VE
```

Figure IV.7 – Création du cluster Proxmox

Cette commande initialise le cluster sur PVE-01, configure automatiquement Corosync pour le mécanisme de heartbeat et génère les fichiers de configuration dans `/etc/pve/`. PVE-01 devient le nœud de référence pour les adhésions suivantes.

Les deux autres nœuds ont ensuite rejoint ce cluster en utilisant l'adresse IP du nœud principal.

```
pvecm add 192.168.10.100
```

Figure IV.8 – Jonction de PVE-02 et PVE-03 au cluster

Cette commande, exécutée successivement sur PVE-02 et PVE-03, contacte le nœud initial, échange les certificats d'authentification et intègre le nœud au cluster Corosync existant.

Le réseau Corosync, utilisé pour le heartbeat entre les nœuds, a été isolé sur le VxLAN 20 afin de séparer le trafic de cluster du reste du trafic d'administration ou de production. Une fois la configuration terminée, la commande de vérification a confirmé la présence des trois nœuds avec quorum atteint.

```
pvecm status
```

Figure IV.9 – Vérification de l'état du cluster

La commande `pvecm status` affiche l'état du cluster Corosync : nombre de membres actifs, version du quorum, et confirmation de l'atteinte du quorum. Une sortie avec **Quorum: Yes** et trois membres confirme que le cluster est correctement formé. Comme l'illustre la figure IV.10, ce résultat est également visible dans l'interface web.

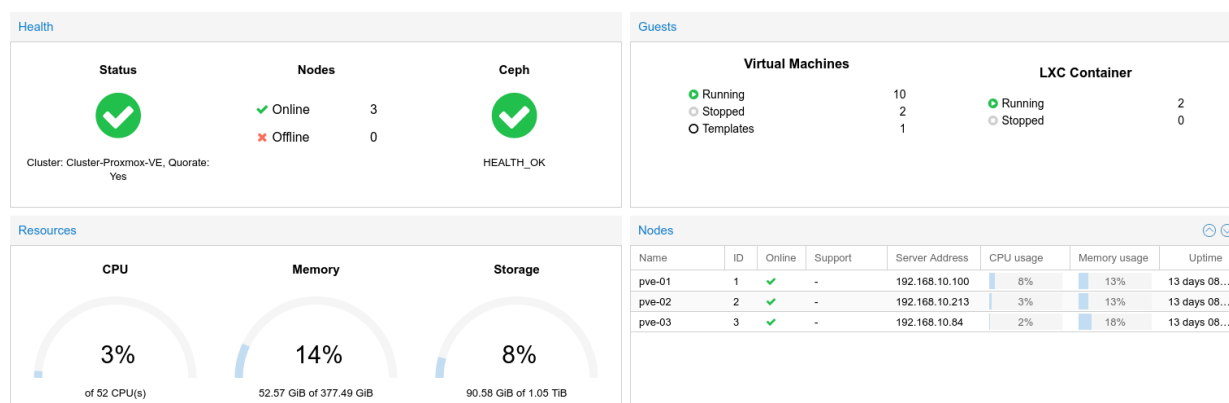


FIGURE IV.10 – Vue du cluster Proxmox montrant les trois nœuds en ligne

La figure IV.10 affiche la vue d'ensemble du datacenter dans l'interface web Proxmox, où les trois nœuds PVE-01, PVE-02 et PVE-03 apparaissent simultanément en ligne. Le tableau de bord indique également l'état de santé du cluster Ceph et le nombre de VMs en cours d'exécution.

IV.2.2.2 Configuration de la migration à chaud

La migration à chaud des machines virtuelles a été configurée pour utiliser le réseau dédié VxLAN 50. Ce choix permet de ne pas mélanger le trafic de migration avec le trafic utilisateur ou le trafic de stockage.

Dans la configuration du datacenter Proxmox, les paramètres de migration ont été ajustés pour utiliser un réseau spécifique et garder un niveau de sécurité acceptable. Une migration manuelle de test a ensuite été réalisée depuis l'interface web afin de vérifier le déplacement transparent d'une VM d'un nœud à un autre sans interruption notable du service. Les figures IV.11 et IV.12 illustrent respectivement l'état avant et après cette opération.

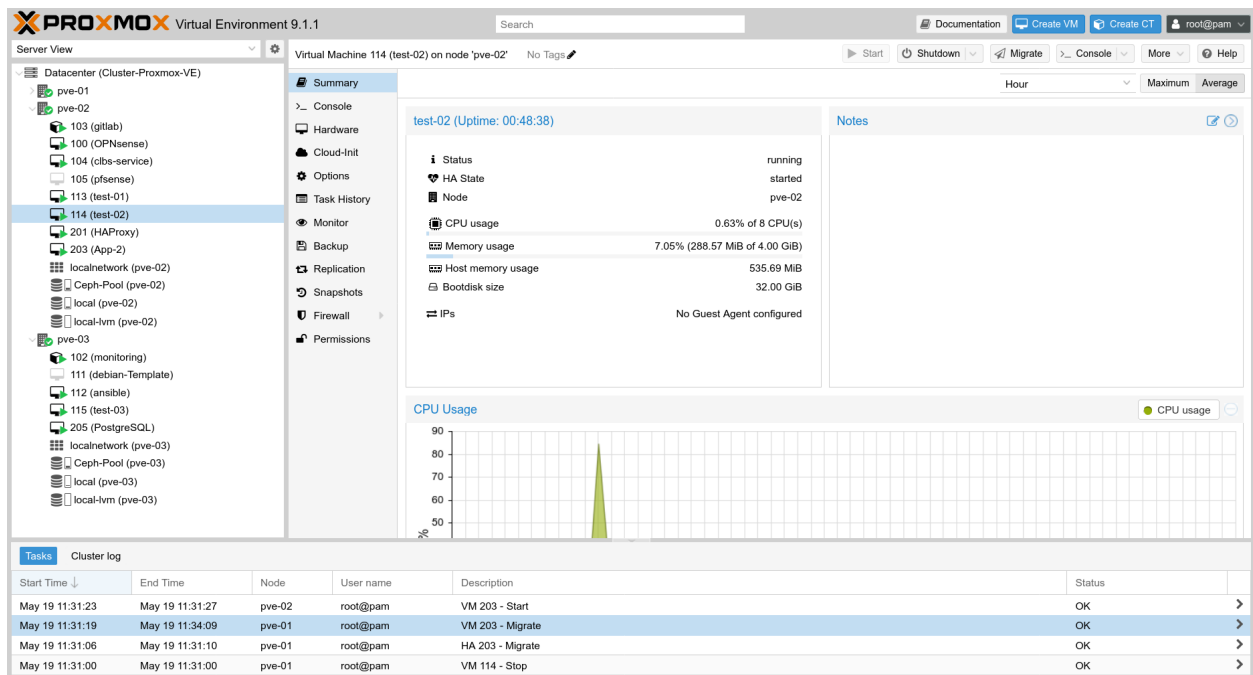


FIGURE IV.11 – Avant la migration live d’une machine virtuelle

La figure IV.11 montre l’état de la VM avant la migration, hébergée sur le nœud PVE-02. On peut observer les métriques de consommation CPU et mémoire ainsi que le nœud d’hébergement actuel dans le panneau de résumé.

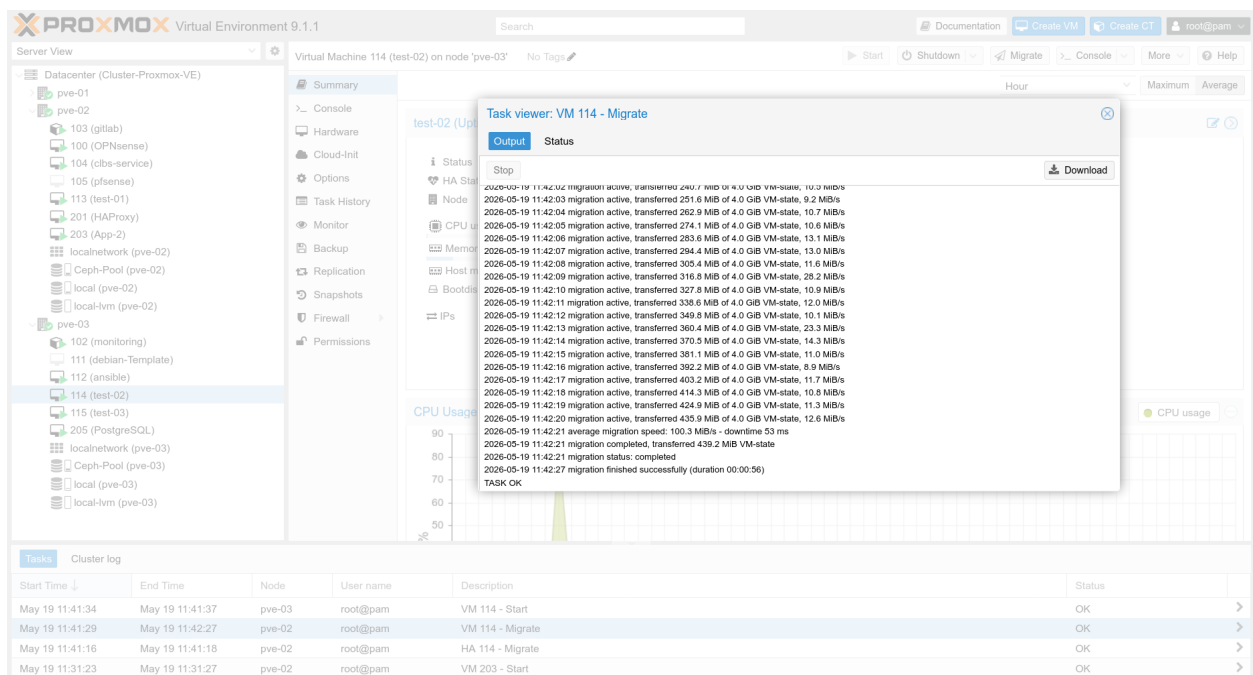


FIGURE IV.12 – Après la migration live d’une machine virtuelle

La figure IV.12 confirme que la VM a été déplacée avec succès vers le nœud PVE-03 sans interruption du service. Le journal de tâches affiche les étapes successives de la migration, incluant le transfert de l’état mémoire et la commutation finale.

IV.2.3 Déploiement de la stack de monitoring

Afin de superviser l'état global de l'infrastructure, une stack de monitoring a été déployée dans un conteneur LXC dédié appelé `monitoring`, avec l'adresse `192.168.10.99`. Ce conteneur centralise InfluxDB, Prometheus et Grafana.

IV.2.3.1 Déploiement d'InfluxDB

InfluxDB a été installé pour recevoir les métriques envoyées directement par Proxmox à travers le système natif *Metric Server*. Un bucket nommé `proxmox-metrics` a été créé afin de stocker les mesures CPU, mémoire, réseau et stockage de chaque nœud du cluster.

Une fois la configuration validée dans l'interface web Proxmox, les métriques ont commencé à être transmises automatiquement sans agent supplémentaire. Comme le montre la figure IV.13, la présence des séries temporelles dans l'interface InfluxDB a confirmé le bon fonctionnement de ce mécanisme.



FIGURE IV.13 – Explorateur InfluxDB montrant les métriques reçues depuis Proxmox

La figure IV.13 présente l'interface Data Explorer d'InfluxDB avec les séries temporelles reçues depuis Proxmox. On y distingue les différentes mesures disponibles (CPU, mémoire, réseau) regroupées par nœud dans le bucket `proxmox-metrics`, confirmant que le pipeline de collecte fonctionne correctement.

IV.2.3.2 Déploiement de Prometheus

Prometheus a été installé dans le même conteneur afin de collecter les métriques exposées par Ceph et par le service applicatif CLBS. La configuration du fichier `prometheus.yml` contient deux jobs principaux : un job pour les métriques Ceph via le module MGR Prometheus, et un job pour l'application CLBS développée en Go.

```
global:
  scrape_interval: 30s
  evaluation_interval: 30s

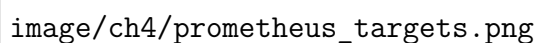
scrape_configs:
  - job_name: ceph
    static_configs:
      - targets:
          - "192.168.10.213:9283"
          - "192.168.10.83:9283"
          - "192.168.10.100:9283"
        labels:
          cluster: "proxmox-lab"

  - job_name: "clbs"
    static_configs:
      - targets: ["192.168.10.104:9101"]
        scrape_interval: 30s
        scrape_timeout: 10s
```

Figure IV.14 – Extrait du fichier prometheus.yml

Ce fichier définit deux jobs de collecte : le premier interroge les endpoints Ceph MGR des trois nœuds sur le port 9283, le second scrute le service CLBS sur le port 9101. L'intervalle de 30 secondes constitue un bon compromis entre fraîcheur des données et charge générée sur les cibles.

Après déploiement, l'interface web Prometheus a permis de confirmer que l'ensemble des cibles déclarées apparaissaient avec l'état UP, comme l'illustre la figure [IV.15](#).



image/ch4/prometheus_targets.png

FIGURE IV.15 – Interface Prometheus montrant les cibles Ceph et CLBS avec l'état UP

La figure IV.15 présente la page *Targets* de l'interface web Prometheus. Chacune des cibles déclarées dans le fichier `prometheus.yml` — les trois endpoints Ceph MGR et le service CLBS — affiche l'état **UP**, confirmant que la collecte des métriques est opérationnelle sur l'ensemble du cluster.

IV.2.3.3 Déploiement de Grafana

Grafana a été ajouté comme couche de visualisation unifiée pour les métriques de Proxmox, Ceph et CLBS. Deux sources de données ont été configurées : InfluxDB pour les métriques de Proxmox, et Prometheus pour les métriques Ceph et applicatives.

Dashboard	Source	Métriques principales
Proxmox Cluster	InfluxDB	CPU, RAM, réseau par nœud
Ceph	Prometheus	Santé cluster, pools, latence OSD
CLBS	Prometheus	Historique des migrations

TABLE IV.1 – Dashboards Grafana configurés

Trois dashboards principaux ont ainsi été utilisés : un dashboard infrastructure Proxmox, un dashboard Ceph, et un dashboard personnalisé pour le service CLBS. La figure [IV.16](#) présente le dashboard dédié à l'infrastructure Proxmox, tandis que la figure [IV.17](#) illustre celui consacré au cluster Ceph.

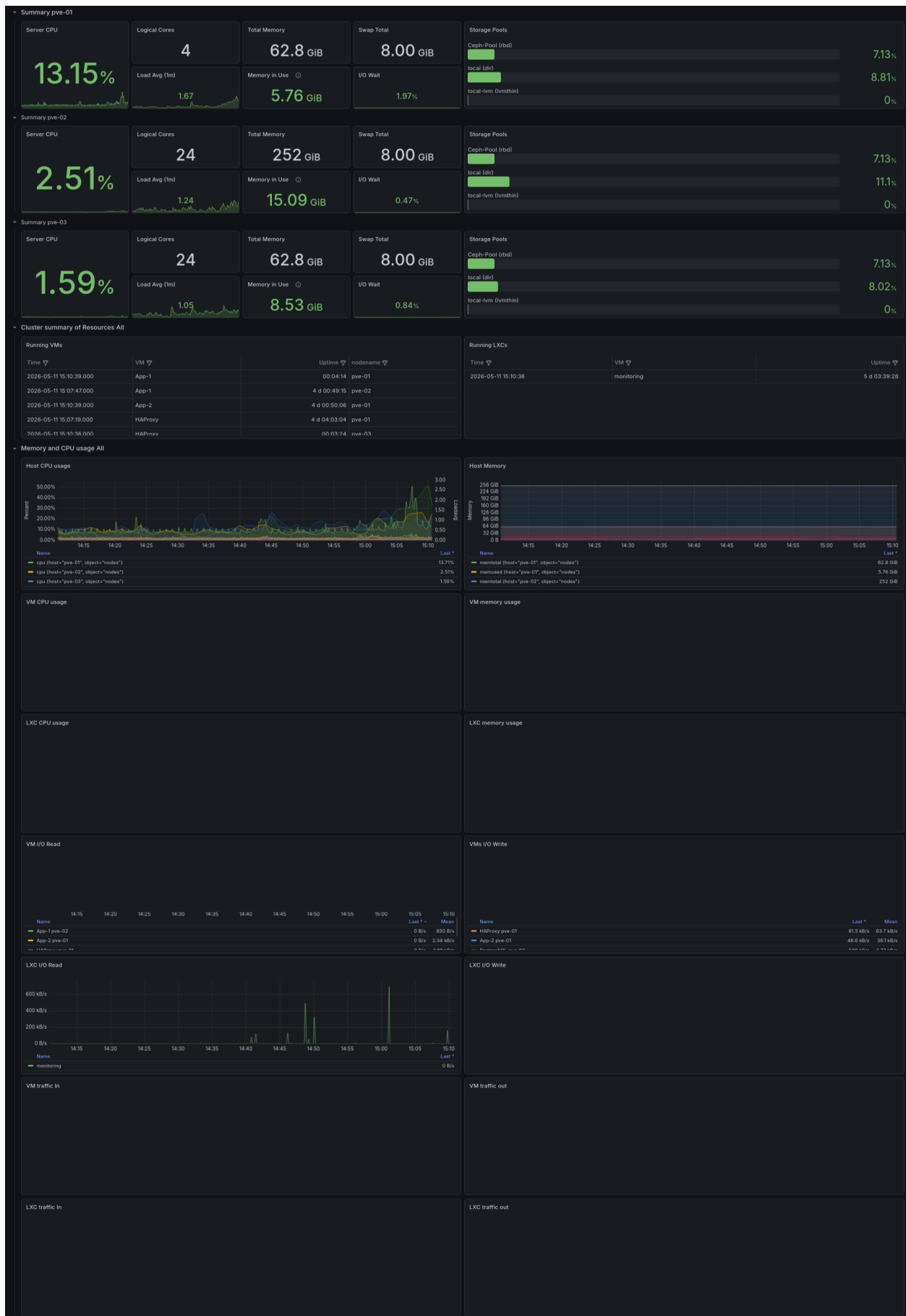
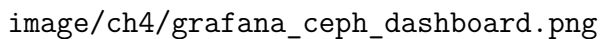


FIGURE IV.16 – Dashboard Grafana Infrastructure pour les nœuds Proxmox

La figure IV.16 présente le dashboard Grafana dédié à l'infrastructure Proxmox. Il affiche en temps réel les métriques CPU, mémoire et charge des trois nœuds, ainsi qu'un récapitulatif des VMs en cours d'exécution et des ressources disponibles dans le cluster.



image/ch4/grafana_ceph_dashboard.png

FIGURE IV.17 – Dashboard Grafana Ceph affichant l'état du cluster, des pools et la latence des OSDs

La figure IV.17 présente le dashboard Grafana dédié au cluster Ceph. On y retrouve l'état global de santé du cluster, les métriques I/O par pool, la latence des OSDs et le taux de réplication en temps réel, permettant de superviser en continu la couche stockage depuis la même interface unifiée.

IV.2.4 Implémentation du mécanisme CLBS

Le mécanisme CLBS constitue l'une des contributions principales de ce projet. Il s'agit d'un service d'équilibrage proactif développé spécifiquement pour pallier l'absence de DRS

natif dans Proxmox VE.

IV.2.4.1 Structure du service

Le service CLBS (*Cluster Load Balancing Service*) a été développé en langage Go dans une VM dédiée accessible sur le réseau de management. Son rôle est d'analyser périodiquement la charge CPU du cluster et de déclencher automatiquement des migrations live lorsque cela est nécessaire.

Le code est structuré en plusieurs composants : un lecteur InfluxDB pour les métriques historiques, un client API Proxmox pour piloter le cluster, un calculateur de bandes de charge, et un moteur principal de décision. Les paramètres de connexion sont placés dans un fichier `.env` chargé au démarrage du service.

IV.2.4.2 Logique de décision

À chaque cycle, le service récupère la charge des nœuds, calcule les bornes de charge modérée, puis classe les nœuds dans les catégories *heavy*, *moderate* ou *light*. Si un déséquilibre est détecté entre un nœud surchargé et un nœud léger, le moteur choisit la VM la plus adaptée à migrer.

Plusieurs mécanismes de protection ont été ajoutés pour éviter les migrations excessives, comme un temps de cooldown global, un cooldown individuel par VM et une suspension temporaire en cas de fréquence de migration trop élevée.

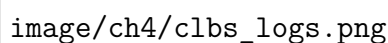
IV.2.4.3 Déploiement comme service systemd

Une fois compilé, le binaire Go a été déployé sur la VM CLBS puis intégré comme service `systemd` afin d'assurer un démarrage automatique et une relance en cas de panne.

```
systemctl enable clbs
systemctl start clbs
journalctl -u clbs.service -f -o cat
```

Figure IV.18 – Gestion du service CLBS avec systemd

Ces trois commandes activent le démarrage automatique du service au boot, le démarrent immédiatement, et affichent les journaux en temps réel. Comme le montre la figure IV.19, ces journaux permettent de suivre en direct les décisions de migration et les éventuelles erreurs de connexion à l'API Proxmox ou à InfluxDB.



image/ch4/clbs_logs.png

FIGURE IV.19 – Journaux systemd du service CLBS lors d’un cycle de détection et de migration

La figure IV.19 présente la sortie de la commande `journalctl` pour le service CLBS pendant un cycle de détection actif. On y observe la classification des nœuds par bande de charge (heavy/moderate/light), la sélection de la VM candidate à la migration, le déclenchement de l’opération via l’API Proxmox et l’activation du cooldown post-migration.

IV.2.4.4 Validation du mécanisme

Pour valider le comportement du service, deux VMs de test ont été créées puis soumises à une charge CPU artificielle avec l’outil `stress-ng`. Le service CLBS a détecté la surcharge et a déclenché une migration live vers un nœud moins chargé.

```
stress-ng --cpu 8
```

Figure IV.20 – Génération d’une charge CPU de test

La commande lance 8 workers saturant les cœurs CPU disponibles. Cette surcharge artificielle déclenche la logique de détection du service CLBS qui, après vérification des seuils, sélectionne la VM candidate et initie sa migration vers un nœud moins chargé. Comme l'illustre la figure IV.21, le résultat est directement visible dans le dashboard dédié.

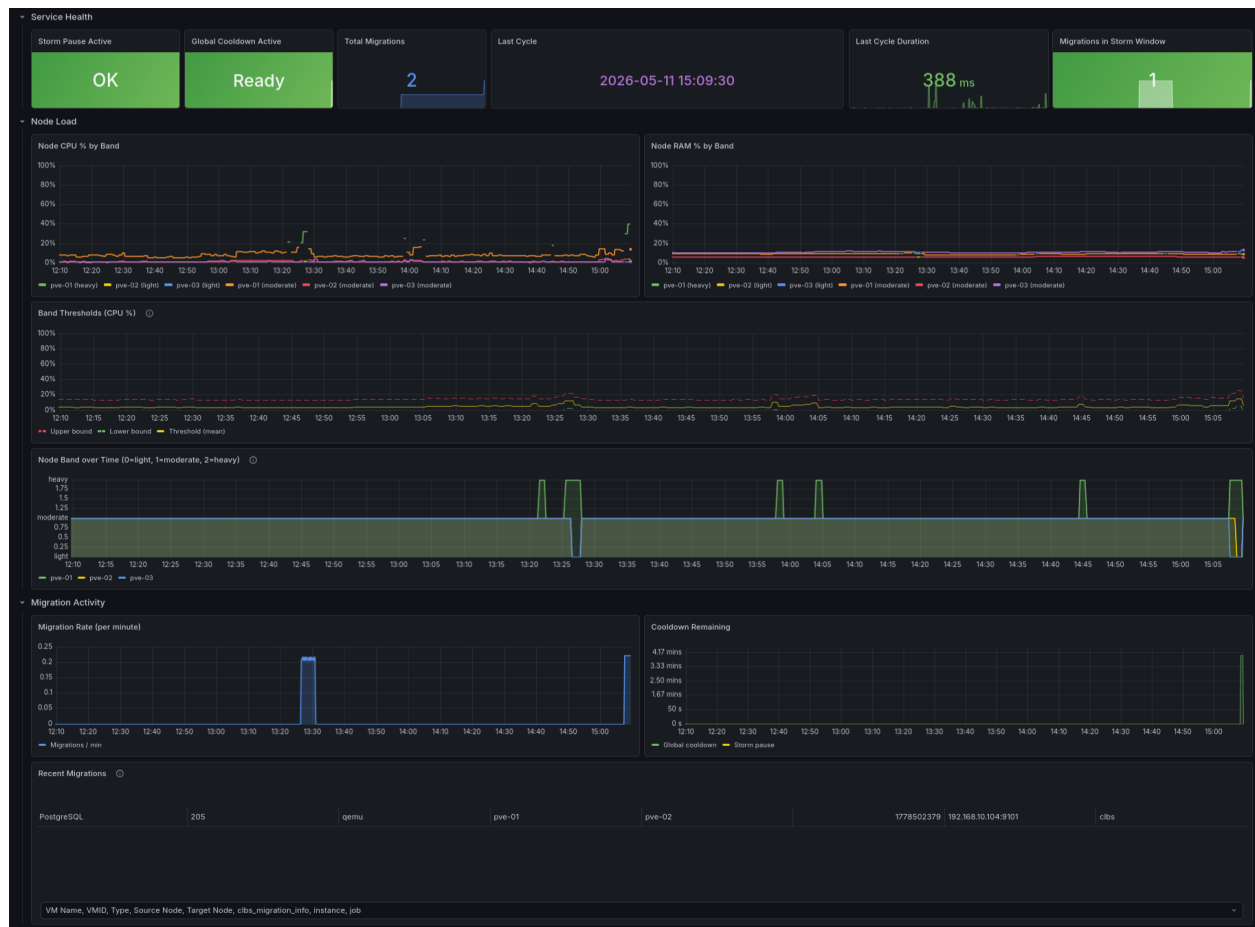


FIGURE IV.21 – Dashboard CLBS montrant une migration déclenchée automatiquement

La figure IV.21 présente le dashboard Grafana du service CLBS après le déclenchement d'une migration automatique. On y observe l'historique de charge des nœuds par bande (heavy/moderate/light), la migration enregistrée dans la section *Migration Activity*, ainsi que le cooldown restant avant le prochain cycle d'évaluation.

IV.2.5 Déploiement de l'application Sona-Web

L'application Sona-Web a été déployée sur l'infrastructure pour démontrer concrètement les capacités de haute disponibilité et de répartition de charge offertes par la plateforme hyperconvergente.

IV.2.5.1 Création des machines virtuelles

L'application Sona-Web a été déployée sous la forme de plusieurs VMs créées à partir d'un template Debian stocké dans le pool Ceph. Cette architecture comprend une VM HA-Proxy en frontal, deux VMs Flask pour la couche applicative, et une VM PostgreSQL pour

la base de données.

Les VMs ont été déployées sur les réseaux internes appropriés, avec attribution d'adresse IP par DHCP selon la zone de sécurité visée. Cette organisation permet de séparer la partie exposée, la partie applicative et la base de données.

IV.2.5.2 Validation de l'application

La validation a porté sur le fonctionnement du load balancing ainsi que sur la disponibilité de l'application. Des requêtes répétées à travers HAProxy ont montré une alternance correcte entre les serveurs applicatifs Flask, confirmant le bon fonctionnement du mécanisme de répartition de charge.

```
for i in $(seq 1 10); do
  curl -s http://172.16.80.x | grep "Server"
done
```

Figure IV.22 – Test de répartition HAProxy

Ce script envoie dix requêtes HTTP successives à travers HAProxy et extrait le champ indiquant le serveur ayant traité chaque requête. Les réponses alternent entre **app-1** et **app-2**, confirmant le bon fonctionnement de l'algorithme round-robin.

IV.2.5.3 Résultats de l'application Sona-Web

Afin de présenter concrètement les fonctionnalités de l'application déployée, des captures de l'interface Sona-Web ont été réalisées en conditions réelles d'utilisation. La figure IV.23 en présente le tableau de bord principal.

The image shows a screenshot of the Sona-Web dashboard. It is a web-based interface with a light blue header and a white main content area. The header contains the application name 'Sona-Web' and some navigation links. The main content area displays real-time statistics, including the current Flask instance, the number of requests recorded in the database, and the availability status of the two application servers.

FIGURE IV.23 – Interface principale de l’application Sona-Web

La figure IV.23 présente le tableau de bord principal de Sona-Web, accessible via HA-Proxy sur le port 8080. L’interface affiche en temps réel l’instance Flask ayant traité la requête courante, le nombre total de requêtes enregistrées en base de données, ainsi que l’état de disponibilité des deux serveurs applicatifs.

IV.2.6 Automatisation avec Ansible

L’automatisation est un élément clé de la reproductibilité de l’infrastructure. Les playbooks Ansible développés permettent de provisionner de nouvelles VMs en quelques minutes sans intervention manuelle.

IV.2.6.1 Environnement et structure du projet

Afin de simplifier le provisionnement des machines virtuelles et de rendre l’infrastructure plus reproductible, un environnement Ansible a été mis en place. Les connexions se font

via SSH depuis les postes des développeurs vers le réseau de management, sans installation d'agent sur les nœuds cibles.

```
ansible-infra/
├── ansible.cfg
├── secrets.yml
├── inventory/
│   ├── hosts.yml
│   └── proxmox_nodes.yml
└── playbooks/
    ├── provision_vm.yml
    └── provision_vm_batch.yml
```

Figure IV.24 – Structure du projet Ansible

Cette arborescence organise le projet en trois parties : les fichiers de configuration globaux à la racine, le répertoire `inventory/` définissant les hôtes cibles, et le répertoire `playbooks/` contenant les scripts d'automatisation. Le fichier `secrets.yml` est chiffré avec Ansible Vault pour protéger les identifiants sensibles.

```
[defaults]
inventory          = inventory/hosts.yml
remote_user        = ansible
private_key_file    = ~/.ssh/ansible_ed25519
host_key_checking   = False
interpreter_python   = /usr/bin/python3

[privilege_escalation]
become              = True
become_user          = root
```

Figure IV.25 – Fichier `ansible.cfg`

Ce fichier définit l'utilisateur distant `ansible`, la clé SSH privée Ed25519 et l'élévation automatique de privilèges vers `root`. La désactivation de `host_key_checking` simplifie le déploiement en environnement de laboratoire où les hôtes changent fréquemment.

IV.2.6.2 Inventaire

L'inventaire regroupe les nœuds Proxmox, les VMs permanentes, ainsi qu'un groupe dynamique destiné aux nouvelles VMs provisionnées par les playbooks.

```
all:
  children:
    proxmox_nodes:
      hosts:
        pve-01: {ansible_host: 192.168.10.100}
        pve-02: {ansible_host: 192.168.10.213}
        pve-03: {ansible_host: 192.168.10.84}
    vms:
      hosts:
        monitoring:
```

```
ansible_host: 192.168.10.99
provisioned_vms:
  hosts: {}
```

Figure IV.26 – Extrait de l'inventaire hosts.yml

L'inventaire est structuré en trois groupes : **proxmox_nodes** pour les nœuds physiques, **vms** pour les VMs permanentes, et **provisioned_vms** initialement vide, alimenté dynamiquement par les playbooks de provisionnement au fur et à mesure de la création de nouvelles machines.

IV.2.6.3 Playbook de provisionnement

Le playbook principal automatise la création d'une VM à partir d'un template Debian déjà préparé. Il configure les ressources matérielles, le réseau, le cloud-init, le démarrage de la VM, puis sa validation après boot.

```
ansible-playbook playbooks/provision_vm.yml \
-e "vm_id=110 vm_name=web-01"

ansible-playbook playbooks/provision_vm.yml \
-e "vm_id=115 vm_name=db-01 vm_cores=4 vm_memory=8192"
```

Figure IV.27 – Exemple d'utilisation du playbook provision_vm.yml

Ces deux exemples montrent la flexibilité du playbook : le premier crée une VM web avec les ressources par défaut, le second provisionne une VM base de données avec 4 vCPUs et 8 Go de RAM. L'adresse IP est calculée automatiquement à partir du VMID.

```
vm_ip: "172.16.60.{{ vm_id | int - 90 }}"
```

Figure IV.28 – Formule d'attribution automatique de l'adresse IP

Cette formule génère automatiquement une adresse IP cohérente à partir du VMID, évitant les conflits d'adressage. Par exemple, la VM avec l'ID 110 recevra l'adresse 172.16.60.20.

La configuration cloud-init repose sur trois commandes successives : attachement du disque cloud-init, injection de la configuration réseau, puis régénération de l'ISO cloud-init.

```
qm set {{ vm_id }} --ide2 {{ ceph_pool }}:cloudinit
qm set {{ vm_id }} --ipconfig0 ip={{ vm_cidr }},gw={{ vm_gw }} --
  nameserver {{ vm_dns }}
qm cloudinit update {{ vm_id }}
```

Figure IV.29 – Configuration cloud-init d'une VM Proxmox

Ces trois commandes attachent le disque cloud-init, injectent la configuration réseau (adresse IP, passerelle, DNS) et régénèrent l'ISO cloud-init afin que la VM applique automatiquement ces paramètres dès son premier démarrage, sans intervention manuelle.

IV.2.6.4 Provisionnement en lot et pipeline PXE/TFTP

Un second playbook `provision_vm_batch.yml` permet le provisionnement en lot de plusieurs VMs à partir d'une liste définie à l'avance dans l'inventaire. Plutôt que d'exécuter le playbook unitaire pour chaque machine, ce playbook itère sur une liste de définitions YAML et parallélise les créations, réduisant le temps de provisionnement d'un parc de dix VMs de plusieurs dizaines de minutes à quelques minutes seulement.

En complément de cette automatisation au niveau logiciel, un pipeline PXE/TFTP (*Pre-boot eXecution Environment / Trivial File Transfer Protocol*) a été mis en place pour couvrir l'étape la plus amont du cycle de vie d'un nœud physique : l'installation initiale du système d'exploitation. L'objectif est de pouvoir enregistrer un nouveau serveur Dell PowerEdge R720 vierge dans l'infrastructure sans aucune intervention manuelle, en s'appuyant uniquement sur le démarrage réseau.

Pourquoi PXE et TFTP ? Le démarrage réseau PXE repose sur une contrainte fondamentale : au moment du boot, la machine ne dispose d'aucun système d'exploitation, d'aucune pile TCP complète, et d'aucune bibliothèque logicielle. Le client PXE, intégré directement dans le firmware de la carte réseau, ne dispose que de quelques dizaines de kilo-octets de mémoire et d'une pile réseau UDP minimale. Cette contrainte exclut l'usage de protocoles comme FTP, SSH ou HTTP avec TLS, qui nécessitent tous un OS opérationnel. TFTP, défini par la RFC 1350 et opérant sur le port UDP/69, est le seul protocole compatible avec cet environnement *pre-OS* : son implémentation ne dépasse pas quelques kilo-octets, et il ne requiert ni authentification ni connexion TCP.

En revanche, la limitation inhérente à TFTP — un mécanisme de transfert *stop-and-wait* d'un bloc de 512 octets à la fois — le rend inadapté au transfert de fichiers volumineux. En pratique, TFTP est strictement réservé aux fichiers de démarrage (bootloader et kernel), dont la taille totale n'excède généralement pas 40 Mo. Les fichiers volumineux, comme l'image d'installation ISO (environ 800 Mo) ou le fichier de réponses automatiques (*preseed*), sont ensuite téléchargés via HTTP dès que le kernel Linux est en mémoire et dispose d'une pile TCP complète.

Protocole	Transport	Nécessite un OS	Usage dans le pipeline
TFTP	UDP/69	Non	Bootloader + kernel (≤ 40 Mo)
HTTP	TCP	Oui (après kernel)	Image ISO + preseed (≈ 800 Mo)
FTP	TCP	Oui	Inutilisable en pre-OS
SSH/SFTP	TCP	Oui	Post-installation uniquement

TABLE IV.2 – Comparaison des protocoles utilisés dans le pipeline PXE

Architecture du pipeline L'architecture retenue repose sur quatre composants principaux, hébergés dans des conteneurs LXC dédiés sur le cluster Proxmox. Le serveur DHCP (OPNsense) est chargé de fournir une adresse IP au nœud en démarrage réseau et de lui indiquer, via les options 66 et 67, l'adresse du serveur TFTP et le nom du fichier bootloader à télécharger. Un conteneur LXC dédié héberge le serveur TFTP (`tftpd-hpa`) ainsi que les fichiers de boot. Un second conteneur LXC héberge un serveur HTTP Nginx qui met à disposition l'image d'installation et le fichier *preseed*. Enfin, le nœud physique cible amorce la séquence depuis son firmware.

La séquence complète de démarrage réseau se déroule en six étapes successives :

1. La machine envoie une requête DHCP en broadcast pour obtenir une adresse IP et les options de boot (option 66 : adresse du serveur TFTP, option 67 : nom du bootloader).
2. OPNsense répond avec une adresse IP et les paramètres PXE.
3. Le client PXE télécharge le bootloader (`pxelinux.0` en mode BIOS Legacy, `grubx64.efi` en mode UEFI) via TFTP.
4. Le bootloader charge le menu de boot puis récupère via TFTP le kernel Linux (`vmlinuz`) et l'image disque initiale (`initrd.img`).
5. Le kernel s'exécute en mémoire vive et prend le contrôle de la machine ; il télécharge ensuite l'image d'installation et le fichier *preseed* via HTTP depuis le serveur Nginx.
6. L'installation de Proxmox VE se déroule de manière entièrement automatisée, sans aucune intervention manuelle.

Configuration DHCP et serveur TFTP La configuration des options PXE dans OPNsense renseigne l'adresse du serveur TFTP et le nom du fichier bootloader adapté au mode firmware de la machine cible.

```
# Option 66 - Adresse du serveur TFTP (Next Server)
next-server 192.168.10.X;

# Option 67 - Nom du fichier bootloader
# Pour BIOS Legacy :
filename "pxelinux.0";

# Pour UEFI 64 bits :
filename "grubx64.efi";
```

Figure IV.30 – Options DHCP pour le boot PXE

L'installation du serveur TFTP et la préparation de l'arborescence de fichiers de boot sont réalisées sur le conteneur LXC dédié. Le bootloader, les modules Syslinux, le kernel et l'initrd sont copiés dans le répertoire racine du service TFTP.

```
apt install -y tftpd-hpa pxelinux syslinux-common

# Cr ation de l'arborescence TFTP
mkdir -p /var/lib/tftpboot/pxelinux.cfg

# Copie du bootloader et des modules Syslinux
cp /usr/lib/PXELINUX/pxelinux.0 /var/lib/tftpboot/
cp /usr/lib/syslinux/modules/bios/*.c32 /var/lib/tftpboot/

# Copie du kernel et de l'initrd (extraits de l'ISO cible)
cp vmlinuz /var/lib/tftpboot/
cp initrd.img /var/lib/tftpboot/
```

Figure IV.31 – Installation et arborescence du serveur TFTP

Menu de boot PXE Le fichier de configuration du menu de boot PXE définit les entrées disponibles au démarrage réseau. L'entrée `install-auto` déclenche l'installation entièrement automatisée de Proxmox VE en pointant vers le fichier *preseed* hébergé sur le serveur HTTP.

```
DEFAULT menu.c32
PROMPT 0
TIMEOUT 100    # 10 secondes , puis boot automatique

MENU TITLE === Boot Reseau Proxmox ===

LABEL install-auto
    MENU LABEL Installation automatique Proxmox VE
    KERNEL vmlinuz
    APPEND initrd=initrd.img auto=true \
           url=http://192.168.10.X/preseed.cfg

LABEL local
    MENU LABEL Boot depuis le disque local
    LOCALBOOT 0
```

Figure IV.32 – Menu de boot PXE

Ce menu présente deux choix : une installation entièrement automatisée de Proxmox VE à partir du fichier *preseed*, et un démarrage local depuis le disque pour les nœuds déjà installés. Le délai de 10 secondes puis le boot automatique sur l'entrée par défaut garantit qu'aucune interaction clavier n'est requise lors du provisionnement à grande échelle.

Ce pipeline PXE/TFTP constitue le premier maillon de la chaîne d'automatisation complète de l'infrastructure. Une fois le nœud installé et accessible sur le réseau de management, les playbooks Ansible prennent le relais pour configurer Proxmox, rejoindre le cluster et intégrer les réseaux VxLAN, offrant ainsi un pipeline de bout en bout depuis un serveur vierge jusqu'à un nœud pleinement opérationnel.

IV.3 Partie II — Couche Stockage

La couche stockage repose entièrement sur Ceph, déployé directement en intégration native avec Proxmox VE. Cette section présente le déploiement du cluster Ceph et la validation de sa tolérance aux pannes.

IV.3.1 Déploiement du cluster Ceph

Le déploiement du cluster Ceph suit une séquence bien définie : initialisation du réseau cluster, création des monitors et managers, ajout des OSDs, puis configuration du pool de stockage utilisé par les VMs.

IV.3.1.1 Initialisation de Ceph

La couche stockage repose sur un cluster Ceph intégré directement à Proxmox. L'initialisation a été effectuée depuis l'interface web ou via la ligne de commande, en définissant un réseau cluster dédié et un réseau public séparé.

```
pveceph init --network 10.10.30.0/24
```

Figure IV.33 – Initialisation du cluster Ceph

Cette commande initialise Ceph en spécifiant le réseau dédié aux échanges internes entre OSDs (10.10.30.0/24), transporté sur le VxLAN 30. Ce réseau est physiquement séparé du réseau public Ceph (10.10.40.0/24) pour isoler le trafic de réplication interne.

Le réseau 10.10.30.0/24 a été réservé aux échanges internes Ceph, tandis que le réseau 10.10.40.0/24 a été utilisé comme réseau public.

IV.3.1.2 Déploiement des MON et MGR

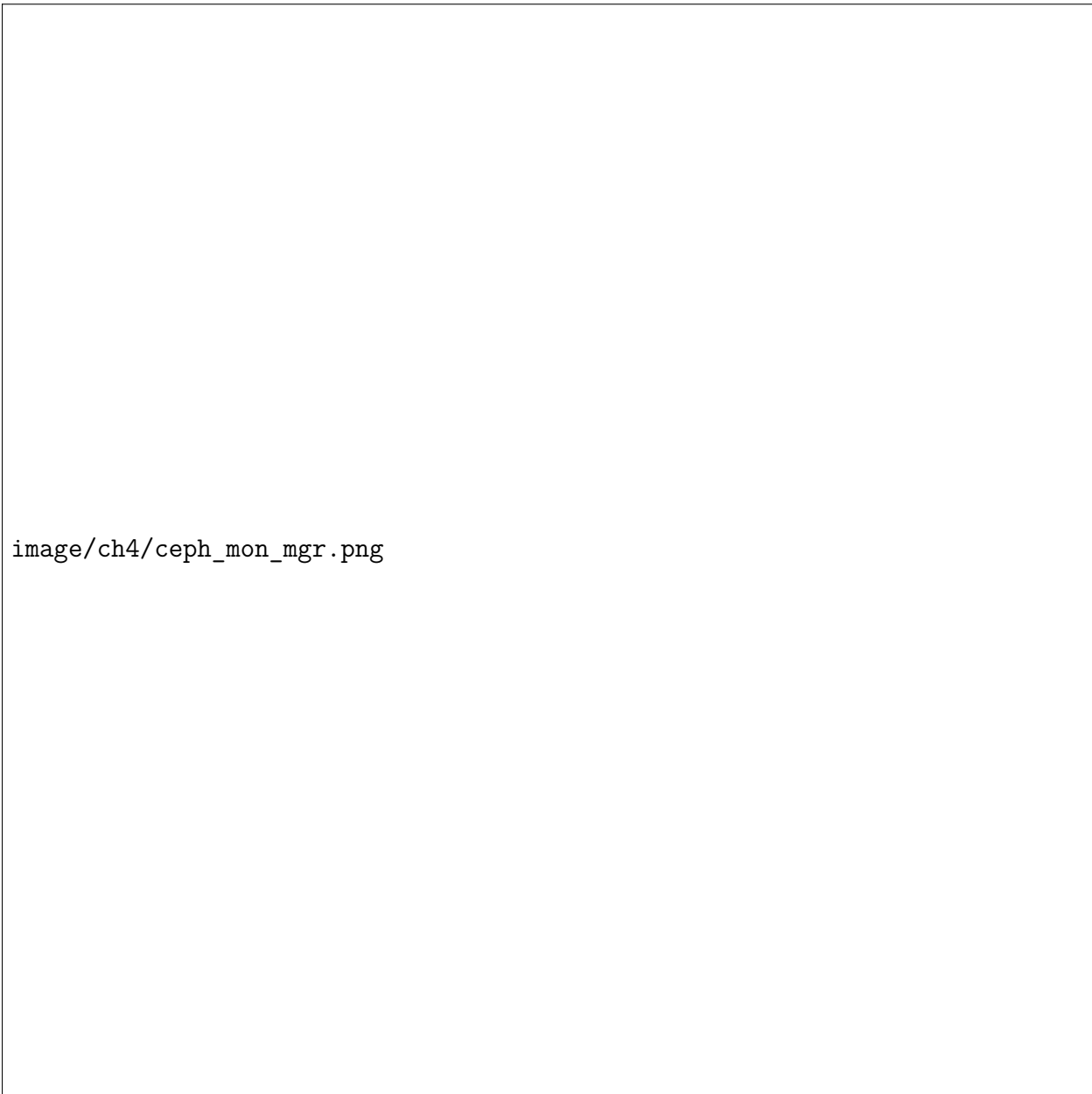
Un moniteur Ceph a été créé sur chaque nœud du cluster afin d'assurer le quorum et la haute disponibilité des métadonnées du cluster.

```
pveceph mon create
```

Figure IV.34 – Création d'un monitor Ceph

Cette commande est exécutée successivement sur chacun des trois nœuds. Avec trois moniteurs, le quorum Ceph est atteint dès que deux d'entre eux sont opérationnels, garantissant la disponibilité même en cas de panne d'un nœud.

Des managers Ceph ont aussi été déployés pour fournir les fonctions d'administration avancée et l'export des métriques vers Prometheus. Comme le montre la figure [IV.35](#), l'interface Proxmox permet de vérifier l'état de l'ensemble des moniteurs et managers déployés.



image/ch4/ceph_mon_mgr.png

FIGURE IV.35 – Interface Proxmox affichant les monitors et managers Ceph actifs sur les trois nœuds

La figure IV.35 présente la vue Ceph de l'interface Proxmox après le déploiement des monitors et managers. Les trois MON apparaissent avec l'état **online** et le quorum est atteint, tandis que les MGR affichent leur rôle actif ou en veille selon le nœud, confirmant la redondance complète de la couche de contrôle Ceph.

```
ceph mgr module enable prometheus
```

Figure IV.36 – Activation du module Prometheus pour Ceph

L'activation de ce module expose les métriques du cluster (état des OSDs, pools, latence, I/O) sur le port 9283 de chaque nœud, permettant à Prometheus de les collecter sans configuration supplémentaire côté Ceph.

IV.3.1.3 Déploiement des OSDs

Chaque nœud a reçu un disque dédié transformé en OSD afin de participer au stockage distribué. Le déploiement a été réalisé sur `/dev/sdb` pour chacun des trois serveurs.

```
pveceph osd create /dev/sdb
```

Figure IV.37 – Création d'un OSD Ceph

Cette commande formate le disque avec le système BlueStore de Ceph et l'intègre comme OSD dans le cluster. Le disque devient disponible pour la réplication des données selon la politique CRUSH configurée.

Au total, le cluster dispose de trois OSDs, distribués sur les trois nœuds physiques. La hiérarchie CRUSH a ensuite été vérifiée pour confirmer la bonne prise en compte de tous les nœuds de stockage.

```
ceph osd tree
```

Figure IV.38 – Affichage de l'arbre CRUSH

La commande affiche la hiérarchie CRUSH avec les trois nœuds physiques et leurs OSDs. Cette structure garantit que les trois répliques d'un même bloc seront toujours placées sur des serveurs physiques distincts, conformément à la politique de domaines de défaillance.

IV.3.1.4 Création du pool de stockage

Un pool RBD a été créé avec une réplication de taille 3 et une taille minimale de 2, ce qui garantit la disponibilité des données même en cas de perte d'un nœud.

```
pveceph pool create ceph-pool --size 3 --min-size 2
```

Figure IV.39 – Création du pool Ceph

Le paramètre `-size 3` impose trois répliques par objet, une par nœud physique. Le paramètre `-min-size 2` maintient le pool accessible tant qu'au moins deux répliques sont disponibles, offrant une tolérance à la panne d'un nœud sans coupure de service.

Ce pool a ensuite été intégré comme stockage partagé dans Proxmox pour héberger les disques des VMs et des conteneurs.

```
ceph -s  
ceph df
```

Figure IV.40 – État global du cluster Ceph

La commande `ceph -s` affiche un résumé de l'état de santé du cluster, tandis que `ceph df` donne la capacité totale, utilisée et disponible par pool. Ces deux commandes constituent la vérification de base après chaque modification de la configuration Ceph. Les figures [IV.41](#) et [IV.42](#) illustrent l'état obtenu après ce déploiement.

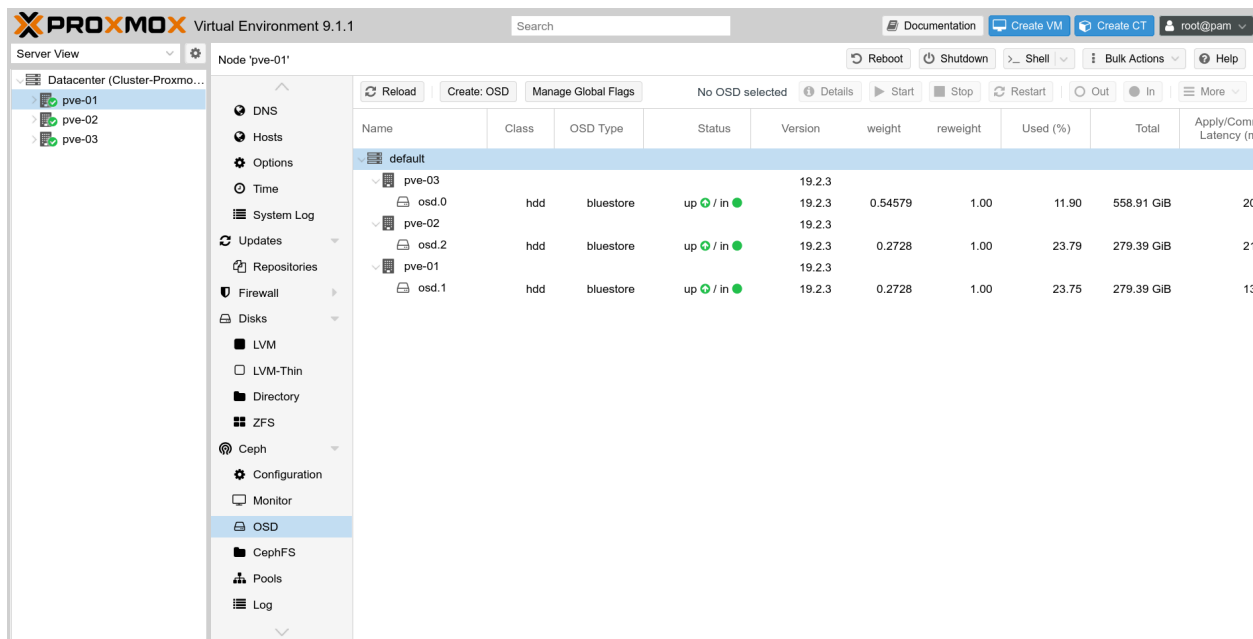


FIGURE IV.41 – État du cluster Ceph montrant les MON et OSD

La figure IV.41 affiche la vue OSD de l'interface Proxmox, listant les trois OSDs avec leur statut up/in, leur type de stockage BlueStore et leur poids dans la hiérarchie CRUSH. Tous les OSDs apparaissent actifs et correctement intégrés au cluster.

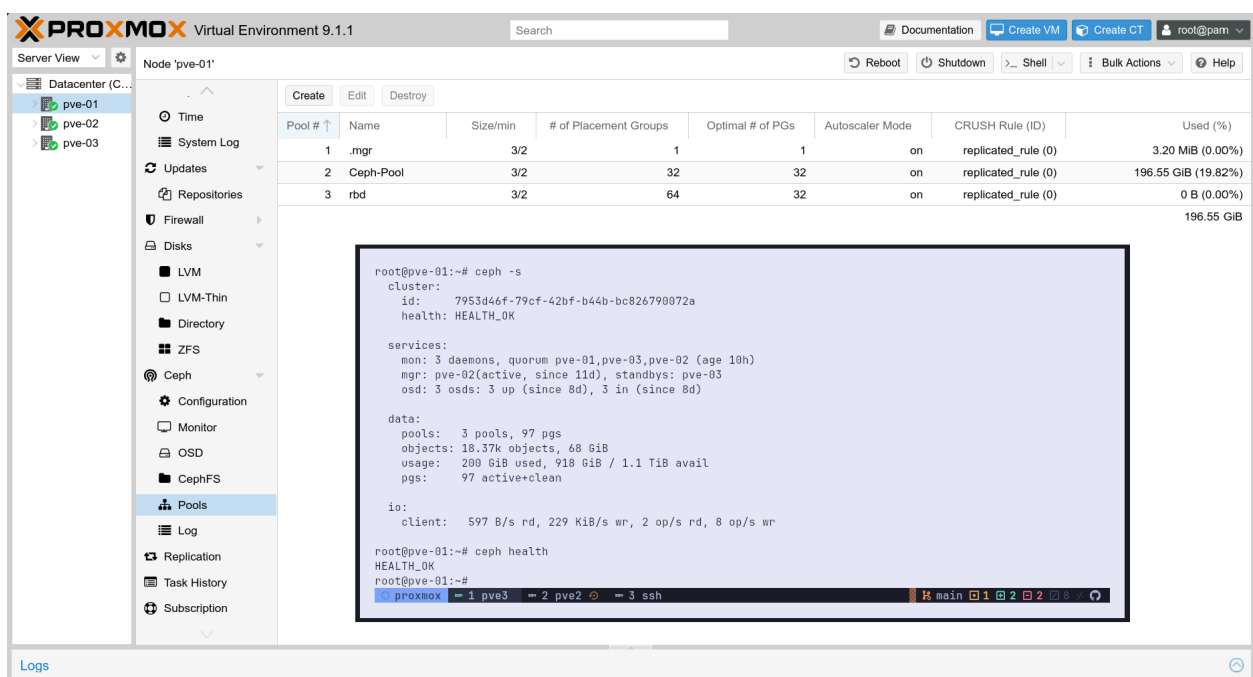


FIGURE IV.42 – État du cluster Ceph montrant le pool RBD

La figure IV.42 montre la vue des pools Ceph dans l'interface Proxmox. Le pool `ceph-pool` apparaît avec l'état `HEALTH_OK`, le facteur de réplication à 3 et les métriques d'I/O en temps réel, confirmant le bon fonctionnement de la couche stockage.

IV.3.2 Validation de la tolérance aux pannes

La tolérance aux pannes de la couche stockage a été vérifiée par un scénario de panne simulée. Lors de l’extinction d’un nœud, le cluster Ceph est passé temporairement à l’état `HEALTH_WARN`, mais les données sont restées accessibles grâce au facteur de réplication.

Après le retour du nœud défaillant, Ceph a automatiquement lancé la resynchronisation des données et l’état du cluster est revenu à `HEALTH_OK`. Ce test valide le bon comportement de la couche stockage distribuée.

IV.4 Partie III — Couche Réseau

La couche réseau est construite autour d’Open vSwitch et des tunnels VxLAN, permettant de créer plusieurs réseaux logiques isolés sur le même réseau physique Sonatrach. OPNsense assure le routage centralisé et le filtrage entre les différents segments.

IV.4.1 Configuration Open vSwitch et tunnels VxLAN

La configuration réseau s’appuie sur Open vSwitch pour la création des bridges et des tunnels VxLAN, et sur les fichiers d’interfaces Linux pour la persistance de la configuration.

IV.4.1.1 Installation d’Open vSwitch

Open vSwitch a été installé sur chaque nœud Proxmox afin de construire une couche réseau virtuelle flexible, adaptée au transport de plusieurs flux logiques sur le même réseau physique.

```
apt install openvswitch-switch
```

Figure IV.43 – Installation d’Open vSwitch

Cette commande installe le daemon OVS et les outils de gestion en ligne de commande (`ovs-vsctl`, `ovs-ofctl`). Elle doit être exécutée sur chacun des trois nœuds Proxmox avant toute configuration des bridges et des tunnels.

Un bridge principal a été créé sur chaque nœud, puis l’interface physique principale y a été attachée pour servir de point de transit à tous les tunnels VxLAN.

```
ovs-vsctl add-br vmbr1
ovs-vsctl add-port vmbr1 eno1
```

Figure IV.44 – Création du bridge OVS principal

La première commande crée le bridge OVS `vmbr1`, la seconde y attache l’interface physique `eno1` comme port de transit. Tous les tunnels VxLAN créés ultérieurement utilisent ce bridge, permettant de transporter plusieurs réseaux logiques sur une seule interface physique.

IV.4.1.2 Configuration complète des interfaces sur PVE-01

Le fichier `/etc/network/interfaces` du nœud PVE-01 centralise l'ensemble de la configuration réseau. Pour en faciliter la lecture, il est présenté ici en cinq parties logiques correspondant aux différents rôles des interfaces : la base physique, les réseaux d'infrastructure Ceph et Corosync, les réseaux de migration et de sauvegarde, puis les réseaux applicatifs et leurs bridges Linux. La figure IV.45 ci-dessous offre une vue synthétique de cette organisation en quatre couches avant que chacune ne soit détaillée.

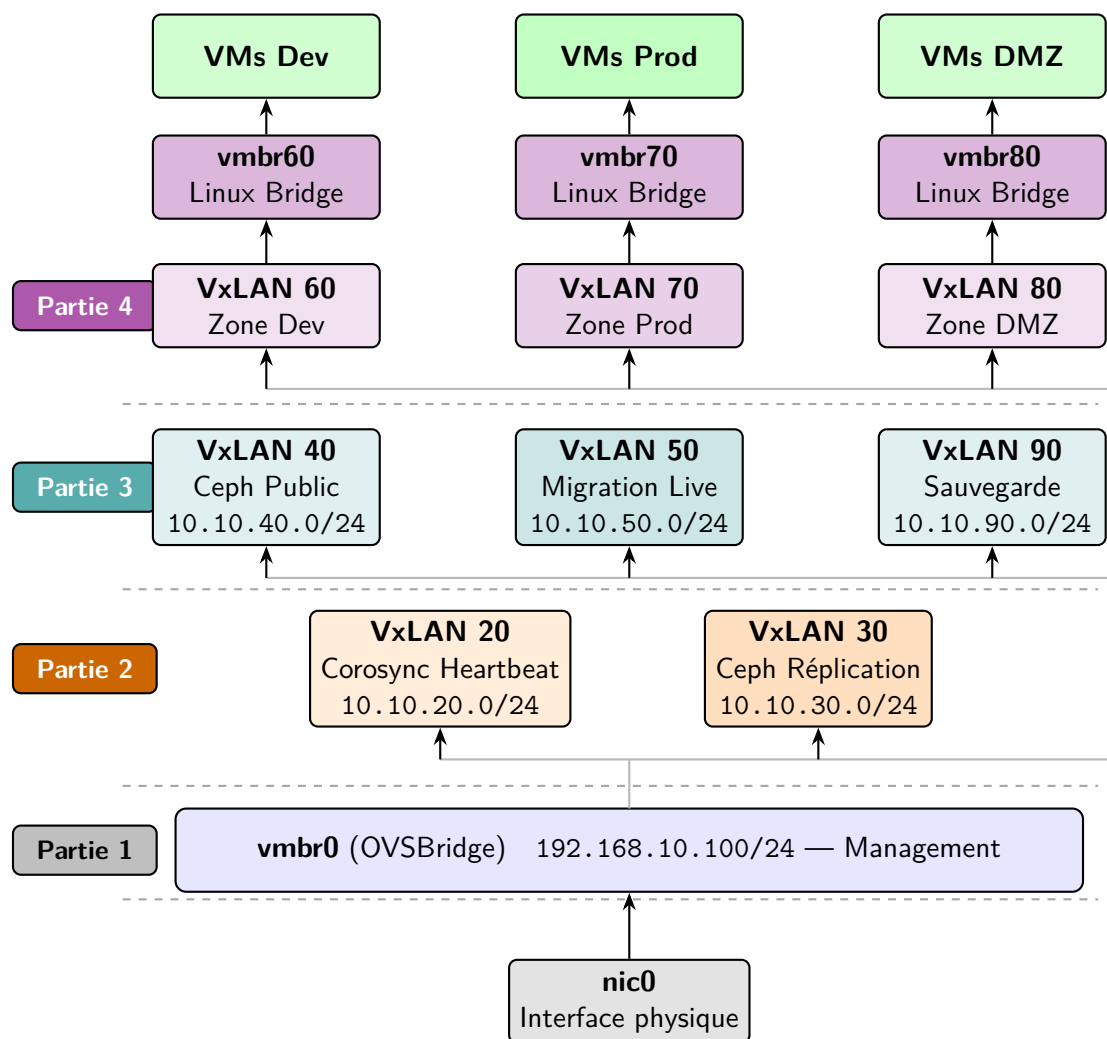


FIGURE IV.45 – Vue d'ensemble de la configuration réseau de PVE-01 — Quatre parties logiques

La figure IV.45 illustre l'organisation en quatre couches de la configuration réseau de PVE-01 : le bridge de management (Partie 1), les VxLAN d'infrastructure Corosync et Ceph interne (Partie 2), les VxLAN de migration, Ceph public et sauvegarde (Partie 3), et enfin les VxLAN applicatifs avec leurs bridges Linux (Partie 4).

Partie 1 — Interface physique et bridge de management Cette première partie définit l'interface physique `nic0` ainsi que le bridge OVS principal `vmbr0`, qui sert de point d'ancrage à l'ensemble des tunnels VxLAN. L'adresse de management `192.168.10.100/24` est attribuée directement sur ce bridge.

```
auto lo
iface lo inet loopback

iface nic0 inet manual

auto vmbr0
iface vmbr0 inet static
    address 192.168.10.100/24
    gateway 192.168.10.36
    ovs_type OVSBridge
    ovs_ports nic0

auto nic0
iface nic0 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0

iface nic1 inet manual
iface nic2 inet manual
iface nic3 inet manual
```

Figure IV.46 – Interfaces de base et bridge OVS de management

Le bridge `vmbr0` est de type `OVSBridge` et encapsule `nic0` comme port physique de sortie. C'est sur ce même bridge que tous les tunnels VxLAN seront connectés, ce qui permet de les transporter sur l'unique interface physique disponible.

Partie 2 — VxLAN Corosync (clé 20) et stockage Ceph interne (clé 30) Ces deux VxLAN transportent respectivement le trafic de heartbeat Corosync entre les nœuds du cluster, et le trafic de réplication interne Ceph entre les OSDs. Chaque tunnel est créé en mode point-à-point, avec une interface interne OVS associée pour l'adressage IP local.

```
# ===== COROSYNC VXLAN (key=20) =====
auto vxlan20-pve02
iface vxlan20-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=20

auto vxlan20-pve03
iface vxlan20-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=20

auto vxlan20-int
iface vxlan20-int inet static
    address 10.10.20.1/24
    ovs_type OVSIntPort
```

```
    ovs_bridge vmbr0

# ===== CEPH INTERNE VXLAN (key=30) =====
auto vxlan30-pve02
iface vxlan30-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=30

auto vxlan30-pve03
iface vxlan30-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=30

auto vxlan30-int
iface vxlan30-int inet static
    address 10.10.30.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0
```

Figure IV.47 – VxLAN Corosync (clé 20) et Ceph interne (clé 30)

Le VxLAN 20 isole le trafic Corosync du reste du réseau, garantissant que les heartbeats entre nœuds ne soient jamais perturbés par le trafic applicatif. Le VxLAN 30 isole quant à lui le trafic de réplication Ceph, dont la bande passante peut être importante lors de la resynchronisation d'un OSD.

Partie 3 — Ceph public (clé 40), migration live (clé 50) et sauvegarde (clé 90)

Ces trois VxLAN couvrent respectivement le réseau public Ceph utilisé pour les accès RBD des VMs, le réseau dédié aux migrations live de VMs entre nœuds, et le réseau de sauvegarde. Leur séparation évite toute interférence entre ces flux lors des opérations intensives.

```
# ===== CEPH PUBLIC VXLAN (key=40) =====
auto vxlan40-pve02
iface vxlan40-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=40

auto vxlan40-pve03
iface vxlan40-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=40

auto vxlan40-int
iface vxlan40-int inet static
```

```
address 10.10.40.1/24
ovs_type OVSPort
ovs_bridge vmbr0

# ===== MIGRATION LIVE VXLAN (key=50) =====
auto vxlan50-pve02
iface vxlan50-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=50

auto vxlan50-pve03
iface vxlan50-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=50

auto vxlan50-int
iface vxlan50-int inet static
    address 10.10.50.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== SAUVEGARDE VXLAN (key=90) =====
auto vxlan90-pve02
iface vxlan90-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=90

auto vxlan90-pve03
iface vxlan90-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=90

auto vxlan90-int
iface vxlan90-int inet static
    address 10.10.90.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0
```

Figure IV.48 – Ceph public (clé 40), migration (clé 50) et sauvegarde (clé 90)

La séparation du réseau public Ceph (VxLAN 40) du réseau de réplication interne (VxLAN 30) suit les recommandations de Ceph pour les environnements de production : le trafic client et le trafic OSD-OSD n'entrent pas en contention. Le VxLAN 50, dédié aux migrations live, préserve la qualité de service lors des déplacements de VMs par le mécanisme CLBS.

Partie 4 — Réseaux applicatifs (clés 60, 70, 80) et bridges Linux Les trois derniers VxLAN portent le trafic des zones applicatives Dev (clé 60), Prod (clé 70) et DMZ (clé 80). Chacun dispose d'une interface interne OVS et d'un bridge Linux associé (vmbr60, vmbr70, vmbr80), qui constitue le point de connexion effectif pour les interfaces réseau des VMs applicatives.

```
# ===== VXLAN 60 (Dev) =====
auto vxlan60-pve02
iface vxlan60-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=60

auto vxlan60-pve03
iface vxlan60-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=60

auto vxlan60-int
iface vxlan60-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 70 (Prod) =====
auto vxlan70-pve02
iface vxlan70-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=70

auto vxlan70-pve03
iface vxlan70-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=70

auto vxlan70-int
iface vxlan70-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 80 (DMZ) =====
auto vxlan80-pve02
iface vxlan80-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=80
```

```
auto vxlan80-pve03
iface vxlan80-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=80

auto vxlan80-int
iface vxlan80-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VM Bridges =====
auto vmbr60
iface vmbr60 inet manual
    bridge_ports vxlan60-int
    bridge_stp off
    bridge_fd 0

auto vmbr70
iface vmbr70 inet manual
    bridge_ports vxlan70-int
    bridge_stp off
    bridge_fd 0

auto vmbr80
iface vmbr80 inet manual
    bridge_ports vxlan80-int
    bridge_stp off
    bridge_fd 0

source /etc/network/interfaces.d/*
```

Figure IV.49 – VxLAN applicatifs (clés 60, 70, 80) et bridges Linux

Les bridges Linux `vmbr60`, `vmbr70` et `vmbr80` constituent la jonction entre le plan de données OVS et les interfaces réseau des VMs Proxmox. Chaque bridge est relié à l'interface interne OVS du VxLAN correspondant, formant ainsi un chemin de commutation continu depuis la VM applicative jusqu'à l'interface physique du nœud, à travers le tunnel VxLAN vers les autres nœuds du cluster. Cette architecture permet de transporter plusieurs réseaux logiques isolés sur le même réseau physique, sans dépendre d'un switch physique configuré en VLAN.

IV.4.2 Déploiement de la VM OPNsense

OPNsense joue un rôle central dans l'architecture réseau en assurant le routage inter-réseaux, le NAT sortant et le service DHCP pour les trois zones applicatives.

IV.4.2.1 Création et interfaces réseau

OPNsense a été déployé sous forme de machine virtuelle sur le cluster Proxmox. Son disque système est stocké sur le pool Ceph, ce qui permet de bénéficier du stockage distribué

et de la haute disponibilité.

La VM dispose de 4 Go de RAM, de 2 vCPUs et d'un disque de 20 Go. Quatre interfaces réseau lui ont été attribuées afin de jouer le rôle de routeur et pare-feu pour les différents segments de l'infrastructure.

Interface VM	Bridge Proxmox	Rôle	Adresse IP
net0 (em0)	vmbr0	WAN	192.168.10.65/24
net1 (em1)	vmbr60	LAN Dev	172.16.60.1/24
net2 (em2)	vmbr70	LAN Prod	172.16.70.1/24
net3 (em3)	vmbr80	LAN DMZ	172.16.80.1/24

TABLE IV.3 – Interfaces de la VM OPNsense

Comme le montre la figure IV.50, la console de gestion OPNsense permet de vérifier l'assignation correcte de ces quatre interfaces.

```
Enter a host name or IP address: 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: icmp_seq=0 ttl=111 time=79.301 ms
^C
*** OPNsense.internal: OPNsense 26.1.2_5 (amd64) ***

LAN (em1)      -> v4: 172.16.60.1/24
OPT1 (em2)     -> v4: 172.16.70.1/24
OPT2 (em3)     -> v4: 172.16.80.1/24
WAN (em0)      -> v4: 192.168.10.65/24

HTTPS: SHA256 DD D5 15 E5 28 38 7B A1 9B E2 1D 5B D4 9C 5D F5
           08 23 37 EC F7 A8 97 9D F6 CA CE 29 E9 BF 31 D5

0) Logout                7) Ping host
1) Assign interfaces      8) Shell
2) Set interface IP address 9) pfTop
3) Reset the root password 10) Firewall log
4) Reset to factory defaults 11) Reload all services
5) Power off system        12) Update from console
6) Reboot system           13) Restore a backup

Enter an option: 
```

FIGURE IV.50 – Configuration des interfaces de la VM OPNsense

La figure IV.50 montre la console de gestion OPNsense affichant les quatre interfaces réseau avec leurs adresses IP respectives, confirmant que le routeur virtuel est opérationnel avec ses interfaces WAN et LAN correctement assignées.

IV.4.2.2 NAT sortant

Le NAT sortant automatique a été activé dans OPNsense afin de permettre aux réseaux internes de sortir vers Internet via l'interface WAN. Cette configuration a simplifié le routage pour les VMs hébergées sur les réseaux Dev, Prod et DMZ.

IV.4.3 Configuration du service DHCP

Le service DHCP a été activé via `dnsmasq` sur les interfaces LAN internes. Ce choix a été retenu car il est simple à configurer et convient bien à un environnement de laboratoire.

Interface	Plage début	Plage fin	Gateway
LAN (Dev)	172.16.60.10	172.16.60.250	172.16.60.1
OPT1 (Prod)	172.16.70.10	172.16.70.250	172.16.70.1
OPT2 (DMZ)	172.16.80.10	172.16.80.250	172.16.80.1

TABLE IV.4 – Plages DHCP configurées sur OPNsense

Le bon fonctionnement de DHCP a été vérifié par une capture réseau sur l'interface LAN d'OPNsense pendant le démarrage d'une VM de test.

```
tcpdump -i em1 port 67 or port 68
```

Figure IV.51 – Capture du trafic DHCP avec tcpdump

Cette commande capture les échanges DHCP sur l'interface `em1` d'OPNsense. La sortie affiche successivement les messages Discover, Offer, Request et Acknowledge, validant le cycle complet d'attribution d'adresse IP à la machine virtuelle de test.

Cette vérification a montré le cycle normal des échanges DHCP, depuis le *Discover* jusqu'au *Offer*, puis l'attribution effective de l'adresse IP à la machine virtuelle.

IV.4.4 Configuration du pare-feu Proxmox

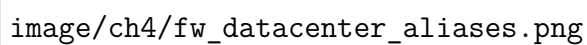
En complément du filtrage assuré par OPNsense au niveau applicatif, le pare-feu natif Proxmox a été configuré au niveau de l'hyperviseur pour protéger les réseaux d'infrastructure et restreindre les accès à la plateforme de gestion.

IV.4.4.1 Architecture à trois niveaux

Le pare-feu Proxmox s'organise en trois niveaux hiérarchiques : le niveau datacenter pour les règles globales communes à toute la plateforme, le niveau nœud pour les règles propres à chaque serveur physique, et le niveau VM pour les règles spécifiques à chaque machine virtuelle. Cette architecture permet d'appliquer le principe du moindre privilège à chaque couche de l'infrastructure.

IV.4.4.2 Configuration au niveau datacenter

Au niveau datacenter, des alias symboliques ont été définis pour simplifier l'écriture des règles. La figure IV.52 illustre les alias créés, parmi lesquels `haproxy`, `app1`, `app2` et `postgres`, chacun associé à l'adresse IP de l'instance correspondante.

The image is a placeholder for a screenshot of the Proxmox Firewall Aliases configuration interface. The text 'image/ch4/fw_datacenter_aliases.png' is displayed within the image frame, indicating the location of the original figure.

image/ch4/fw_datacenter_aliases.png

FIGURE IV.52 – Aliases définis au niveau datacenter dans le pare-feu Proxmox

La figure IV.52 présente la liste des alias configurés dans l'interface Proxmox sous *Datacenter -> Firewall -> Aliases*. Chaque alias associe un nom symbolique à une adresse IP, simplifiant la lecture et la maintenance des règles de filtrage sans nécessiter de modifier chaque règle individuellement en cas de changement d'adresse.

Des IPSets ont également été créés pour regrouper les adresses appartenant à un même domaine de sécurité. Comme le montre la figure IV.53, ces groupes permettent d'appliquer une règle à un ensemble de machines en une seule définition.

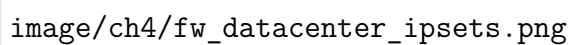
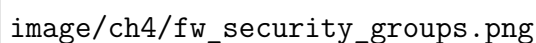
The image is a placeholder for a screenshot of the Proxmox IPSet configuration interface. It is labeled with the file path 'image/ch4/fw_datacenter_ipsets.png'.

FIGURE IV.53 – IPSets configurés au niveau datacenter regroupant les adresses par domaine

La figure [IV.53](#) présente les IPSets définis dans Proxmox sous *Datacenter -> Firewall -> IPSet*. On y distingue notamment les groupes d'adresses des nœuds de gestion et des instances Flask, utilisés pour filtrer les flux de manière centralisée sans dupliquer les adresses dans chaque règle individuelle.

Des groupes de sécurité ont enfin été créés pour encapsuler les règles de ports associées à chaque service d'infrastructure. La figure [IV.54](#) illustre ces groupes et leurs règles respectives.



image/ch4/fw_security_groups.png

FIGURE IV.54 – Groupes de sécurité Proxmox avec leurs règles de ports par service

La figure IV.54 présente les groupes de sécurité configurés sous *Datacenter -> Firewall -> Security Groups*. Les groupes définis regroupent les règles nécessaires à chaque service : **proxmox-management** pour les ports SSH (22), Web UI (8006), VNC (5900–5999) et SPICE (3128); **corosync-cluster** pour les ports UDP 5404–5405 du heartbeat; **vxlan-overlay** pour le port UDP 4789 indispensable aux tunnels VxLAN; et **ceph-storage** pour les ports MON (3300, 6789) et OSD (6800–7300).

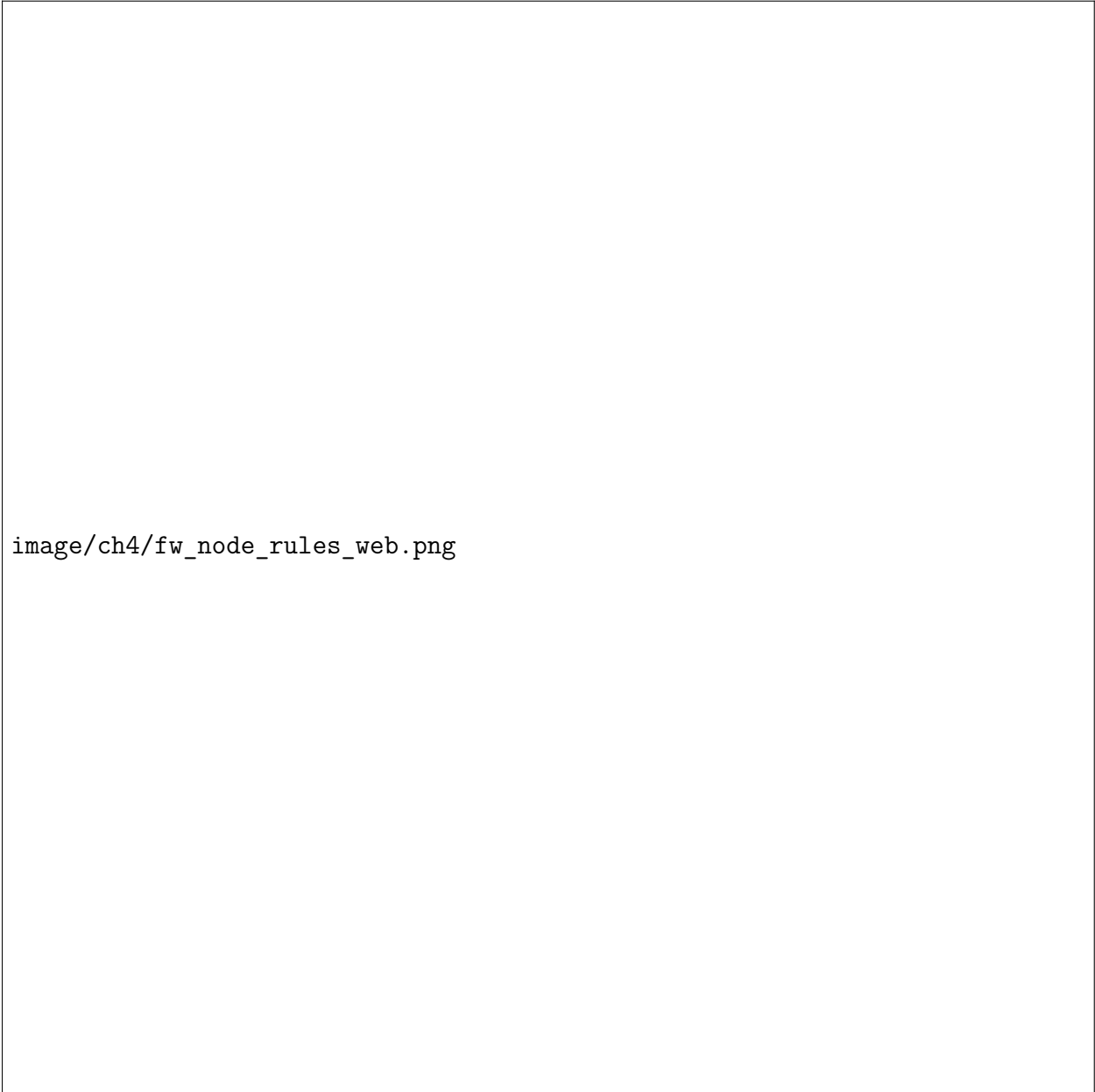
Deux règles complémentaires ont été ajoutées au niveau datacenter pour couvrir des besoins opérationnels spécifiques non couverts par les groupes de sécurité précédents. D’une part, une règle ICMP autorisant les requêtes *ping* depuis le réseau de management a été intégrée dans le groupe **proxmox-management**. Cette règle est indispensable aux diagnostics de connectivité entre nœuds, notamment pour vérifier rapidement la joignabilité d’un hôte lors d’une intervention. D’autre part, le port 9283 TCP a été ouvert dans le groupe **ceph-storage** depuis le réseau de management, afin de permettre à Prometheus de collecter les métriques exposées par le module MGR Ceph. Ce port est celui sur lequel le daemon **ceph-mgr** expose

son endpoint `/metrics` lorsque le module Prometheus est activé, et son ouverture conditionne directement le bon fonctionnement du job `ceph` déclaré dans `prometheus.yml`.

IV.4.4.3 Configuration au niveau nœud

Le deuxième niveau du pare-feu Proxmox s'applique directement à chaque nœud physique du cluster. Les règles définies à ce niveau protègent l'hyperviseur lui-même, indépendamment des VMs qu'il héberge. Elles filtrent les accès aux services système de Proxmox et aux protocoles de cluster, en autorisant uniquement les sources légitimes.

Sur chacun des trois nœuds PVE-01, PVE-02 et PVE-03, le pare-feu est activé et les groupes de sécurité définis au niveau datacenter sont appliqués. La figure IV.55 illustre les règles telles qu'elles apparaissent dans l'interface web Proxmox pour le nœud PVE-01.

The image is a screenshot of the Proxmox web interface showing the firewall rules for node PVE-01. The screenshot is mostly blank, with the text 'image/ch4/fw_node_rules_web.png' visible in the lower-left corner. The interface typically displays a table of rules with columns for rule name, action, direction, and priority, but the content of the table is not visible in this image.

image/ch4/fw_node_rules_web.png

FIGURE IV.55 – Règles de pare-feu au niveau nœud PVE-01 dans l'interface web Proxmox

La figure IV.55 présente la vue *Firewall* du nœud PVE-01 dans l'interface Proxmox. On y observe les groupes de sécurité appliqués et les règles spécifiques au nœud, définissant notamment les accès autorisés en entrée par réseau source et par port de service.

Des règles supplémentaires spécifiques à chaque nœud complètent ce filtrage, comme résumé dans le tableau IV.5.

Direction	Action	Source	Port / Protocole	Description
IN	ACCEPT	192.168.10.0/24	8006 TCP	Accès Web UI Proxmox
IN	ACCEPT	192.168.10.0/24	22 TCP	Accès SSH administration
IN	ACCEPT	192.168.10.0/24	ICMP	Diagnostic réseau (ping)
IN	ACCEPT	10.10.20.0/24	5404–5405 UDP	Heartbeat Corosync
IN	ACCEPT	10.10.30.0/24	6800–7300 TCP	Trafic OSD Ceph interne
IN	ACCEPT	10.10.30.0/24	3300, 6789 TCP	Moniteurs Ceph
IN	ACCEPT	10.10.40.0/24	6800–7300 TCP	Trafic OSD Ceph public
IN	ACCEPT	192.168.10.0/24	9283 TCP	Export métriques Ceph MGR (Prom)
IN	ACCEPT	10.10.50.0/24	any	Migration live (VxLAN 50)
IN	ACCEPT	192.168.10.0/24	4789 UDP	Tunnels VxLAN
IN	DROP	any	any	Règle par défaut

TABLE IV.5 – Règles de pare-feu au niveau nœud (appliquées sur PVE-01, PVE-02, PVE-03)

L'accès à l'interface web Proxmox (port 8006) et au service SSH (port 22) est restreint au seul réseau de management 192.168.10.0/24, empêchant tout accès non autorisé depuis les réseaux applicatifs. La règle ICMP, limitée au même réseau, permet les diagnostics de connectivité sans exposer les nœuds à des requêtes provenant des VMs internes. Le port 9283 est autorisé depuis le réseau de management uniquement, de sorte que seul le conteneur `monitoring` (192.168.10.99) peut interroger l'endpoint Prometheus du MGR Ceph ; cette restriction empêche toute exfiltration des métriques d'infrastructure depuis les réseaux applicatifs. Le trafic Corosync est limité au réseau VxLAN 20 (10.10.20.0/24), tandis que les communications Ceph sont isolées sur leurs VxLAN dédiés (30 et 40). Enfin, le réseau de migration (VxLAN 50) est autorisé sans restriction de port, car Proxmox utilise des ports dynamiques pour le transfert de l'état mémoire des VMs lors des migrations live.

Afin de vérifier que les règles sont correctement appliquées au niveau noyau, la commande suivante a été exécutée directement sur PVE-01 via SSH.

```
ssh root@192.168.10.100 \
    "iptables -L PVEFW-HOST-IN -n -v --line-numbers"
```

Figure IV.56 – Vérification des règles iptables actives générées par Proxmox sur PVE-01

Comme le montre la figure IV.57, la sortie confirme que chaque règle définie dans l'interface web est bien traduite en entrée iptables active, comme le valide le Tableau IV.5 ci-dessus.



image/ch4/fw_iptables_host_in.png

FIGURE IV.57 – Chaîne iptables PVEFW-HOST-IN active sur PVE-01 (vérification SSH)

La figure IV.57 présente la sortie de la commande `iptables -L PVEFW-HOST-IN` exécutée sur PVE-01. Chaque ligne correspond à une règle de filtrage active : on y retrouve les autorisations par réseau source et par port définies dans l'interface Proxmox, traduites en règles noyau effectives, validant ainsi la cohérence entre la configuration logique et l'application réelle du filtrage.

La commande complémentaire suivante a permis de lister les IPSets actifs au niveau noyau, confirmant que les groupes d'adresses définis dans Proxmox sont bien propagés dans le sous-système `ipset` du kernel.

```
ssh root@192.168.10.100 "ipset list"
```

Figure IV.58 – Vérification des IPSets actifs sur PVE-01

La Figure IV.53 confirme déjà que ces IPSets sont correctement définis au niveau data-center ; leur propagation vers le noyau Linux est assurée automatiquement par Proxmox.

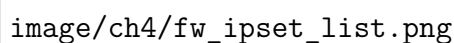
The image is a placeholder for a screenshot of the command 'ipset list' output on PVE-01 via SSH. The text 'image/ch4/fw_ipset_list.png' is displayed, indicating the location of the image file.

FIGURE IV.59 – IPSets actifs sur PVE-01 vérifiés via SSH (`ipset list`)

La figure IV.59 présente la sortie de la commande `ipset list` sur PVE-01. Chaque IP-Set affiché correspond à un groupe d'adresses défini dans l'interface Proxmox : les membres listés confirment que la configuration du pare-feu au niveau datacenter est bien synchronisée et active sur chaque nœud physique.

Ces règles au niveau nœud constituent un premier rempart de défense en profondeur, garantissant que même en cas de compromission d'une VM applicative, l'hyperviseur reste inaccessible depuis les réseaux internes.

IV.4.4.4 Règles au niveau des VMs

Chaque VM dispose de règles de pare-feu adaptées à son rôle. Le principe appliqué est celui du moindre privilège : seuls les ports strictement nécessaires sont ouverts en entrée, et les flux sortants sont autorisés uniquement vers les destinations légitimes.

VM	Direction	Action	Source	Port / Protocole
HAProxy (201)	IN	ACCEPT	0.0.0.0/0	8080 TCP
HAProxy (201)	IN	ACCEPT	management-net	8404 TCP
HAProxy (201)	IN	ACCEPT	management-net	22 TCP
HAProxy (201)	IN	ACCEPT	management-net	ICMP
HAProxy (201)	OUT	ACCEPT	–	8000 TCP
Flask app-1/2	IN	ACCEPT	haproxy	8000 TCP
Flask app-1/2	IN	ACCEPT	management-net	22 TCP
Flask app-1/2	OUT	ACCEPT	–	5432 TCP
PostgreSQL (205)	IN	ACCEPT	app1, app2	5432 TCP
PostgreSQL (205)	IN	ACCEPT	management-net	22 TCP

TABLE IV.6 – Règles de pare-feu par VM applicative

Ces règles garantissent que chaque VM ne communique qu’avec les composants dont elle dépend directement. Pour la VM HAProxy, deux règles supplémentaires ont été ajoutées : le port 8404 TCP, restreint au réseau de management, expose l’interface web de statistiques native d’HAProxy, permettant de consulter en temps réel le nombre de connexions actives, les codes de réponse et l’état de chaque backend sans passer par Grafana. La règle ICMP, également limitée au réseau de management, autorise les diagnostics de connectivité vers le frontal sans l’exposer aux réseaux internes. Tout autre flux est implicitement bloqué par la politique par défaut, assurant une isolation stricte entre les composants de l’application.

IV.4.5 Validation de la haute disponibilité réseau et applicative

Un scénario de panne a été réalisé afin de tester le comportement global de l’infrastructure en cas de perte d’un nœud physique. L’arrêt brutal de PVE-01 a permis d’observer la détection de panne par Corosync, le redémarrage des services protégés sur un autre nœud, puis la resynchronisation automatique après le retour du serveur.

Cette séquence a montré que le cluster restait opérationnel malgré la défaillance, et que les mécanismes de haute disponibilité répondaient correctement à ce type d’événement. La chronologie observée était la suivante : détection de la défaillance par Corosync, déclenchement du basculement HA, redémarrage des VMs protégées sur les nœuds survivants, puis resynchronisation Ceph après le retour du nœud défaillant.

IV.5 Partie IV – Services Complémentaires d’Infrastructure

Cette partie présente le déploiement des services complémentaires qui complètent l’infrastructure hyperconvergée : la synchronisation temporelle, la centralisation des journaux, le système de notification, la gestion du code source et la plateforme d’automatisation opérationnelle.

IV.5.1 Synchronisation Temporelle : NTP/Chrony

La cohérence temporelle est une exigence fondamentale dans un cluster distribué. Une dérive de quelques secondes entre les nœuds peut perturber le quorum Corosync, fausser les journaux de corrélation et provoquer des incohérences dans les métriques Prometheus. Un conteneur LXC dédié héberge un serveur Chrony configuré en Stratum 3, qui fait référence pour l'ensemble des nœuds Proxmox et des VMs de l'infrastructure.

IV.5.1.1 Installation et configuration du serveur Chrony

Le service Chrony a été installé dans un conteneur LXC léger sur le cluster. La configuration du fichier `/etc/chrony/chrony.conf` définit les serveurs de référence amont et la plage réseau des clients autorisés.

```
# Serveurs NTP de r f r e n c e a m o n t
pool pool.ntp.org iburst

# Autoriser les clients du r s e a u d e m a n a g e m e n t
allow 192.168.10.0/24

# Fichier de d r i v e
driftfile /var/lib/chrony/drift

# Activer le mode serveur local si perte de synchronisation amont
local stratum 3
```

Figure IV.60 – Configuration du serveur Chrony

La directive `allow 192.168.10.0/24` ouvre le service aux requêtes NTP provenant de tous les nœuds et VMs du réseau de management. La directive `local stratum 3` garantit que le serveur continue à distribuer une heure de référence cohérente, même en cas de perte temporaire de connectivité vers les serveurs `pool.ntp.org`.

Une fois la configuration appliquée, le service est activé et démarré.

```
systemctl enable chrony
systemctl start chrony
```

Figure IV.61 – Activation du service Chrony

Sur chaque nœud Proxmox et sur chaque VM, le fichier `chrony.conf` client pointe vers l'adresse IP du LXC Chrony interne en remplacement des serveurs publics, centralisant ainsi toutes les synchronisations sur la référence locale.

IV.5.1.2 Validation de la synchronisation

La vérification du bon fonctionnement du serveur Chrony s'effectue depuis le LXC via deux commandes complémentaires. La première confirme la connexion aux sources NTP amont, la seconde liste les clients actifs.

[CAPTURE À INSÉRER]

FIGURE IV.62 – Sources NTP actives sur le serveur Chrony interne [CAPTURE : se connecter en SSH au LXC Chrony, exécuter la commande `chronyc sources -v`, capturer la sortie montrant les serveurs pool.ntp.org avec l'étoile (*) indiquant la source active]

[CAPTURE À INSÉRER]

FIGURE IV.63 – Clients NTP synchronisés sur le serveur Chrony interne [CAPTURE : sur le même LXC Chrony, exécuter `chronyc -n clients`, capturer la sortie listant les adresses IP des nœuds Proxmox (192.168.10.100, .213, .84) et des VMs avec leur dernier accès NTP]

IV.5.2 Centralisation des Journaux : Loki et Alloy

La collecte et la centralisation des journaux système constituent un élément essentiel pour le diagnostic des incidents et l'audit de l'infrastructure. Loki, déployé dans le conteneur LXC `monitoring` existant, joue le rôle d'agrégateur de journaux. L'agent Grafana Alloy, installé sur chaque nœud Proxmox, collecte les fichiers de log locaux et les transmet à Loki pour indexation.

IV.5.2.1 Déploiement de Loki

Loki a été installé dans le conteneur LXC `monitoring` (192.168.10.99), aux côtés de Prometheus et Grafana déjà en place. Cette co-localisation simplifie la configuration de Grafana, qui dispose ainsi de Loki comme source de données supplémentaire sans configuration réseau additionnelle.

```
apt install loki
systemctl enable loki
systemctl start loki
```

Figure IV.64 – Installation de Loki

Loki écoute par défaut sur le port 3100. La source de données Loki a ensuite été ajoutée dans Grafana en pointant vers `http://localhost:3100`.

IV.5.2.2 Déploiement de l'agent Grafana Alloy

L'agent Grafana Alloy a été installé sur chacun des trois nœuds Proxmox. Sa configuration définit les chemins de fichiers de journaux à surveiller et l'adresse du serveur Loki cible.

```
local.file_match "varlogs" {
  path_targets = [{"__path__" = "/var/log/**/*.*.log"}]
}

loki.source.file "varlogs" {
  targets      = local.file_match.varlogs.targets
  forward_to   = [loki.write.default.receiver]
}

loki.write "default" {
```

```
endpoint {  
  url = "http://192.168.10.99:3100/loki/api/v1/push"  
}  
}
```

Figure IV.65 – Configuration de l'agent Alloy pour la collecte des journaux

Cette configuration collecte tous les fichiers `*.log` sous `/var/log/` de chaque nœud et les transmet en flux continu vers Loki. Les labels `host` et `filename` sont automatiquement ajoutés à chaque entrée, permettant de filtrer les journaux par nœud dans Grafana.

IV.5.2.3 Validation

[CAPTURE À INSÉRER]

FIGURE IV.66 – Statut de l'agent Grafana Alloy sur PVE-01 [CAPTURE : SSH sur PVE-01, exécuter `systemctl status alloy`, capturer la sortie montrant le service en état `active (running)`]

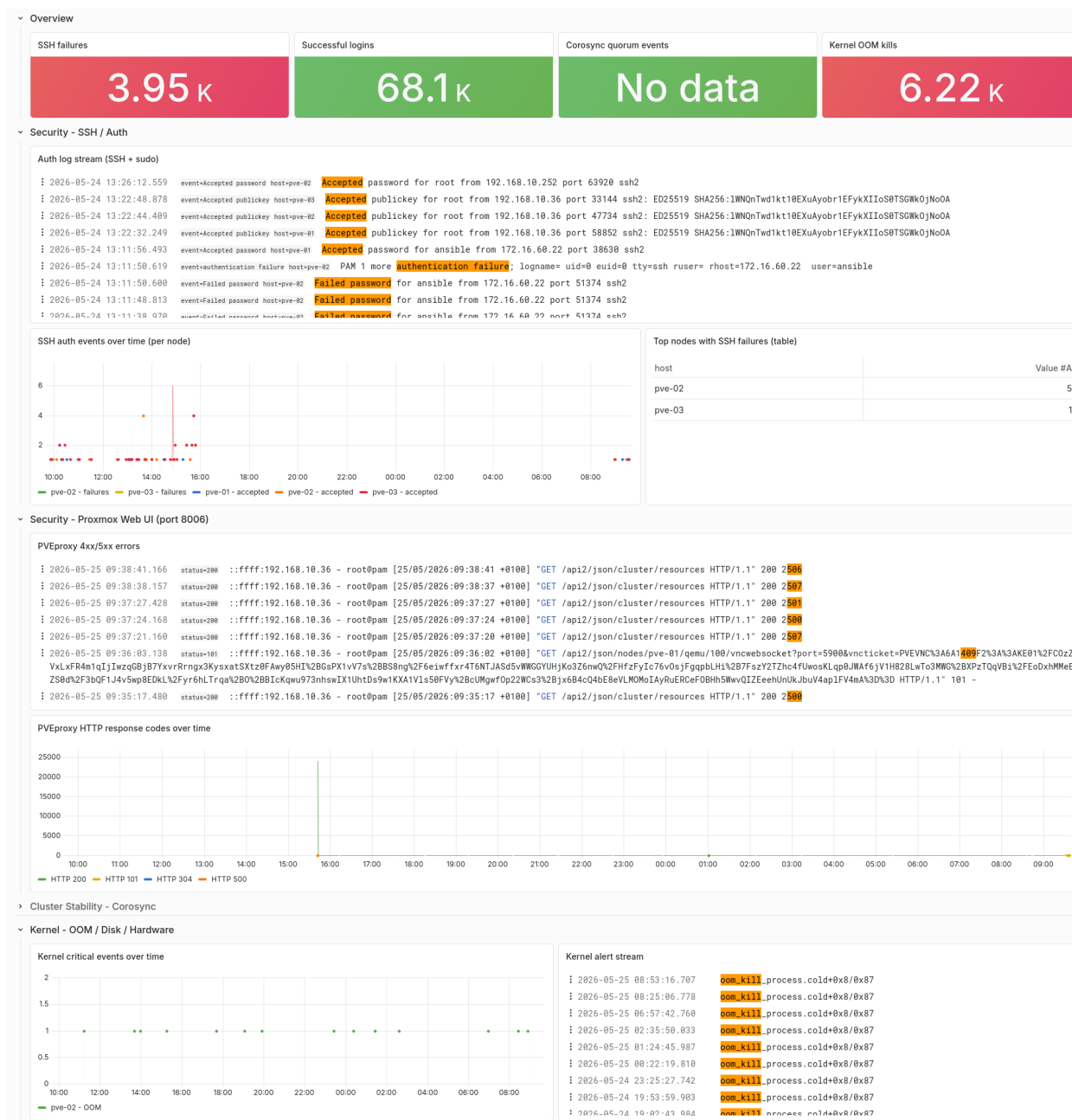


FIGURE IV.67 – Journaux de l’infrastructure visualisés dans Grafana via Loki

IV.5.3 Système de Notification : Alertmanager

La supervision active de l’infrastructure ne se limite pas à la visualisation des métriques : elle inclut la notification automatique des équipes en cas d’anomalie détectée. Le module Grafana Alerting a été configuré pour évaluer des règles d’alerte sur les métriques collectées et transmettre les notifications via un contact point Gmail utilisant le protocole SMTP.

IV.5.3.1 Configuration des règles d’alerte

Les règles d’alerte sont définies directement dans Grafana sous *Alerting* -> *Alert Rules*. Chaque règle spécifie une expression PromQL ou une requête InfluxDB, un seuil de déclenchement, une durée de maintien avant passage en état *Firing*, et le groupe de contact destinataire.

Parmi les règles configurées figurent notamment la détection d'une charge CPU élevée sur les nœuds Proxmox, la surveillance de l'état de santé du cluster Ceph, et la détection d'un OSD passé hors ligne. Ces règles couvrent les scénarios de défaillance les plus critiques de l'infrastructure.

IV.5.3.2 Configuration du contact point Gmail

Le contact point Gmail a été créé dans Grafana sous *Alerting -> Contact Points*. La configuration utilise le serveur SMTP de Google avec authentification par mot de passe d'application, garantissant la compatibilité avec les comptes protégés par la double authentification.

```
# Dans grafana.ini ou via l'interface web :  
[smtp]  
host      = smtp.gmail.com:587  
user      = adresse@gmail.com  
password  = mot_de_passe_application  
from_address = adresse@gmail.com  
starttls_policy = MandatoryStartTLS
```

Figure IV.68 – Paramètres SMTP du contact point Gmail

L'activation de `MandatoryStartTLS` garantit que les identifiants SMTP ne transitent jamais en clair, conformément aux exigences de sécurité de l'infrastructure.

IV.5.3.3 Validation

[CAPTURE À INSÉRER]

FIGURE IV.69 – Règles d'alerte configurées dans Grafana Alerting [CAPTURE : ouvrir Grafana, aller dans Alerting -> Alert Rules, capturer la liste des règles définies (ex : CPU nœud, santé Ceph) avec leur état affiché (Normal / Pending / Firing)]

[CAPTURE À INSÉRER]

FIGURE IV.70 – Contact point Gmail configuré dans Grafana pour les notifications [CAPTURE : dans Grafana -> Alerting -> Contact Points, capturer la configuration du contact point Gmail avec l'adresse email visible (masquer le mot de passe SMTP si affiché)]

IV.5.4 Gestion du Code Source et CI/CD : GitLab

La traçabilité du code et l'automatisation des tests constituent des pratiques essentielles dans un environnement d'infrastructure moderne. Un LXC Turnkey GitLab CE a été déployé sur le cluster pour centraliser les dépôts de code de l'infrastructure et y associer des pipelines d'intégration continue.

IV.5.4.1 Déploiement du LXC Turnkey GitLab CE

Le template Turnkey GitLab CE disponible dans le catalogue Proxmox a été utilisé pour simplifier le déploiement initial. Ce template embarque GitLab CE préconfiguré avec Nginx,

PostgreSQL et les workers Sidekiq, réduisant le temps d'installation à quelques minutes.

Le conteneur LXC a été créé avec les ressources suivantes : 4 vCPUs, 8 Go de RAM et 50 Go de stockage sur le pool Ceph, permettant d'héberger confortablement les dépôts et les artefacts de pipeline de l'infrastructure.

IV.5.4.2 Création des dépôts

Deux dépôts ont été créés dans l'instance GitLab pour couvrir les besoins de versionnement de l'infrastructure. Le dépôt `clbs` contient le code source du service d'équilibrage de charge développé en Go, y compris les fichiers de configuration, les tests unitaires et le fichier de pipeline CI/CD. Le dépôt `ansible-infra` regroupe l'ensemble des playbooks Ansible, l'inventaire et les fichiers de secrets chiffrés avec Ansible Vault.

IV.5.4.3 Configuration des pipelines CI/CD

Un fichier `.gitlab-ci.yml` a été défini dans chaque dépôt pour automatiser les vérifications à chaque push. Pour le projet `clbs`, le pipeline comprend les stages *Lint* (vérification syntaxique du code Go avec `golint`), *Tests* (exécution des tests unitaires avec `go test`) et *Build* (compilation du binaire). Pour le projet `ansible-infra`, le pipeline effectue une vérification syntaxique des playbooks avec `ansible-lint`.

IV.5.4.4 Validation

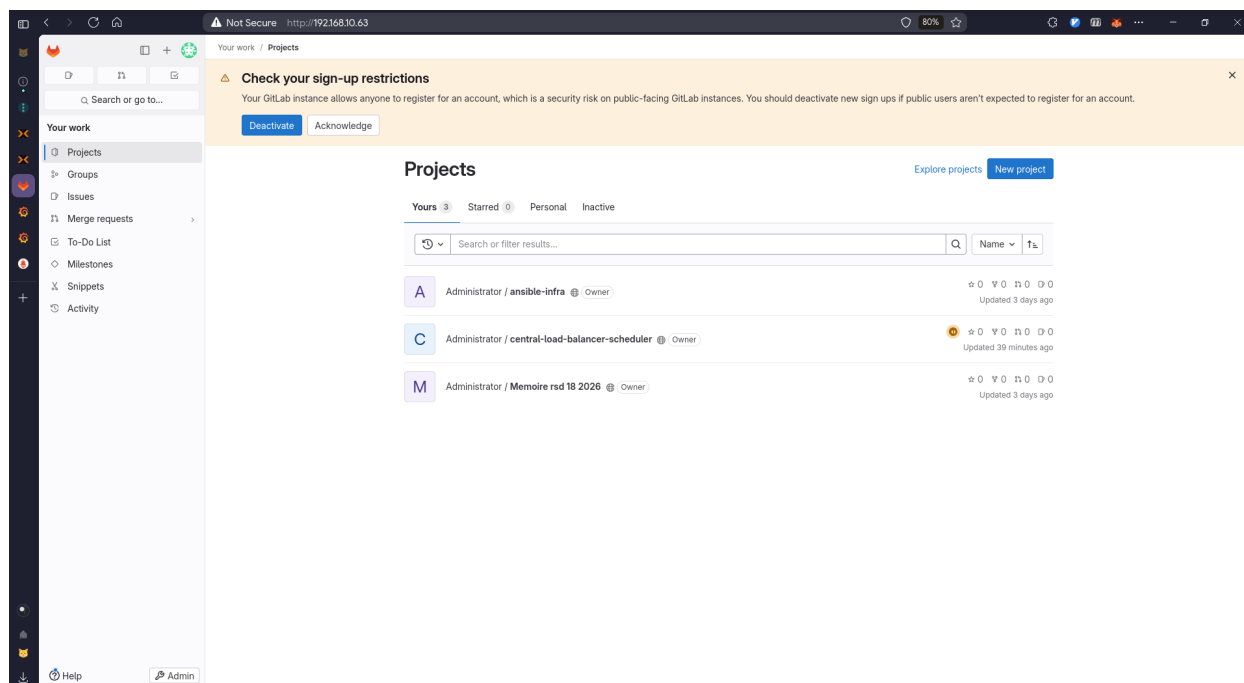


FIGURE IV.71 – Page Projects de l'instance GitLab avec les dépôts de l'infrastructure

[CAPTURE À INSÉRER]

FIGURE IV.72 – Pipeline CI/CD réussi pour le projet clbs dans GitLab [CAPTURE : dans GitLab -> projet clbs -> CI/CD -> Pipelines, capturer un pipeline récent avec ses stages (Lint, Tests, Build) tous affichés en vert (passed), montrant le numéro de pipeline et le commit associé]

IV.5.5 Automatisation Opérationnelle : Rundeck

La délégation de tâches opérationnelles récurrentes à une interface centralisée permet de réduire les erreurs humaines et de tracer l'ensemble des interventions sur l'infrastructure. Rundeck, déployé dans un conteneur LXC dédié, offre une interface web permettant de déclencher des playbooks Ansible sans accès SSH direct aux nœuds.

IV.5.5.1 Déploiement du LXC Rundeck

Le conteneur LXC Rundeck a été créé sur le cluster Proxmox avec 2 vCPUs et 4 Go de RAM. Rundeck a été installé depuis les dépôts officiels et configuré pour écouter sur le port 4440.

```
apt install -y rundeck
systemctl enable rundeckd
systemctl start rundeckd
```

Figure IV.73 – Installation de Rundeck

IV.5.5.2 Connexion au dépôt ansible-infra

Dans l'interface Rundeck, un projet a été créé et configuré pour utiliser le dépôt **ansible-infra** hébergé sur l'instance GitLab locale comme source des playbooks. Cette intégration garantit que Rundeck exécute toujours la version versionnée et validée par le pipeline CI/CD des playbooks, plutôt que des copies locales non contrôlées.

Les jobs Rundeck ont été définis pour chaque opération récurrente : provisionnement d'une nouvelle VM, intégration d'un nœud au cluster, déploiement d'une mise à jour de configuration. Chaque job référence le playbook Ansible correspondant dans le dépôt GitLab et expose les variables nécessaires sous forme de paramètres saisissables dans l'interface web.

IV.5.5.3 Validation

[CAPTURE À INSÉRER]

FIGURE IV.74 – Interface Rundeck affichant les jobs de provisionnement Ansible [CAPTURE : ouvrir l'interface web Rundeck (port 4440), aller dans Jobs, capturer la liste des jobs configurés (provisionnement VM, intégration nœud, etc.) avec leurs noms visibles]

[CAPTURE À INSÉRER]

FIGURE IV.75 – Résultat d’exécution d’un job Rundeck avec le log Ansible [CAPTURE : dans Rundeck -> Activity, ouvrir le détail d’une exécution récente, capturer la page affichant le statut **Succeeded** et les tâches Ansible déroulées avec leurs états (ok, changed, skipped)]

Le service OpenLDAP, prévu en section III.7.4 pour centraliser l’authentification de Proxmox, Grafana, GitLab et Rundeck, n’a pas été déployé dans le cadre de ce projet. Son intégration constitue une perspective d’évolution à court terme de l’infrastructure.

IV.6 Conclusion

Ce chapitre a présenté l’implémentation réelle de l’infrastructure hyperconvergée open source mise en place dans le cadre du projet. La couche calcul a permis de déployer Proxmox, le cluster, le monitoring, l’automatisation et les services applicatifs. La couche stockage a été assurée par Ceph avec réplication et tolérance aux pannes. Enfin, la couche réseau a été construite avec Open vSwitch, les tunnels VxLAN, OPNsense et les services d’adressage associés.

Les services complémentaires déployés dans la Partie IV — synchronisation temporelle, centralisation des journaux, notifications, versionnement du code et automatisation opérationnelle — complètent l’infrastructure en lui apportant les briques transverses indispensables à son exploitation en conditions réelles.

Les différents tests réalisés ont confirmé la cohérence de l’architecture proposée et sa capacité à répondre aux objectifs du projet dans un environnement de laboratoire structuré. Le chapitre suivant présentera l’évaluation des performances obtenues ainsi qu’une analyse des résultats expérimentaux.

Bibliographie

- [1] I. N. Sonatrach, « Historique et évolution », Institut numérique, 2012. [En ligne]. Disponible : <https://www.institut-numerique.org/2-historique-et-evolution-5028f58cb09d3>
- [2] J. Schneider, S. Susnjara et I. Smalley, « Hyperconverged infrastructure explained », IBM Think, févr. 2024. [En ligne]. Disponible : <https://www.ibm.com/think/topics/hyperconverged-infrastructure>
- [3] A. Pande, « Proxmox vs. Harvester : Can the enterprise-grade platform beat the community favorite? », XDA Developers, févr. 2025. [En ligne]. Disponible : <https://www.xda-developers.com/proxmox-vs-harvester/>
- [4] SUSE, « Harvester HCI Documentation », Harvester HCI. [En ligne]. Disponible : <https://harvesterhci.io/>
- [5] Proxmox Server Solutions, « Proxmox Virtual Environment Overview », Proxmox. [En ligne]. Disponible : <https://www.proxmox.com/en/products/proxmox-virtual-environment/overview>
- [6] Open vSwitch Authors, « What is Open vSwitch? », Documentation Open vSwitch. [En ligne]. Disponible : <https://docs.openvswitch.org/en/latest/intro/what-is-ovs/>
- [7] Open vSwitch Authors, « Features », Open vSwitch. [En ligne]. Disponible : <https://www.openvswitch.org/features/>
- [8] Broadcom, « VMware DRS Overview : Optimizing Resource Allocation in vSphere Cluster », Broadcom Knowledge Base. [En ligne]. Disponible : <https://knowledge.broadcom.com/external/article/391137/vmware-drs-overview-optimizing-resource.html>
- [9] InfluxData, « InfluxDB v2 Documentation », InfluxData. [En ligne]. Disponible : <https://docs.influxdata.com/influxdb/v2/>
- [10] TechWrix, « Containers vs VMs : Choosing the Right Approach for Your Proxmox VE », TechWrix. [En ligne]. Disponible : <https://www.techwrix.com/containers-vs-vms-choosing-the-right-approach-for-your-proxmox-ve-part-4/>
- [11] Red Hat, « Chapter 1 : The Ceph Architecture », Red Hat Ceph Storage 4 Architecture Guide. [En ligne]. Disponible : https://docs.redhat.com/en/documentation/red_hat_ceph_storage/4/html/architecture_guide/the-ceph-architecture_arch
- [12] A. Chandak, K. Jaju, A. Kanfade, P. Lohiya et A. Joshi, « Dynamic Load Balancing of Virtual Machines using QEMU-KVM », *International Journal of Computer Applications*, vol. 46, n° 6, pp. 10–14, mai 2012.

BIBLIOGRAPHIE

- [13] J. Watts, « A practical approach to dynamic load balancing », Mémoire de master, California Institute of Technology, 1995. [En ligne]. Disponible : https://thesis.caltech.edu/6924/1/Watts_j_1995.pdf
- [14] Corosync Project, « The Corosync Cluster Engine », Corosync. [En ligne]. Disponible : <https://corosync.github.io/corosync/>
- [15] Grafana Labs, « Grafana Documentation », Grafana Labs. [En ligne]. Disponible : <https://grafana.com/docs/grafana/latest/>
- [16] W. Tarreau, « HAProxy version 2.8 : configuration manuelle », HAProxy. [En ligne]. Disponible : <https://www.haproxy.org/download/2.8/doc/configuration.txt>
- [17] S. A. Weil, S. A. Brandt, E. L. Miller et D. D. Long, « Ceph : A scalable, high-performance distributed file system », dans *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI'06)*, USENIX Association, 2006, pp. 307–320.
- [18] S. A. Weil, S. A. Brandt, E. L. Miller et C. Maltzahn, « CRUSH : Controlled, Scalable, Decentralized Placement of Replicated Data », dans *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing (SC'07)*, ACM, 2007.
- [19] Proxmox Server Solutions, « Proxmox VE Ceph Server », Proxmox VE Wiki, 2024. [En ligne]. Disponible : https://pve.proxmox.com/wiki/Ceph_Server
- [20] T. C. Wilcox Jr., « Dynamic Load Balancing of Virtual Machines Hosted on Xen », Mémoire de master, Brigham Young University, avr. 2009. [En ligne]. Disponible : <https://scholarsarchive.byu.edu/cgi/viewcontent.cgi?article=2653&context=etd>
- [21] GitLab, « GitLab Requirements », Documentation GitLab. [En ligne]. Disponible : <https://docs.gitlab.com/install/requirements/>
- [22] KVM Authors, « Kernel-based Virtual Machine », Linux KVM. [En ligne]. Disponible : https://www.linux-kvm.org/page/Main_Page
- [23] QEMU Project, « QEMU Documentation », QEMU. [En ligne]. Disponible : <https://www.qemu.org/docs/master/>
- [24] libvirt Project, « libvirt : The virtualization API », libvirt.org. [En ligne]. Disponible : <https://libvirt.org/>
- [25] Proxmox Server Solutions, « High Availability Manager », Proxmox VE Wiki. [En ligne]. Disponible : https://pve.proxmox.com/wiki/High_Availability
- [26] M. Mahalingam et al., « Virtual eXtensible Local Area Network (VXLAN) : A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks », RFC 7348, IETF, août 2014. [En ligne]. Disponible : <https://datatracker.ietf.org/doc/html/rfc7348>
- [27] Microsoft, « Dynamic Optimization in VMM », Microsoft Learn. [En ligne]. Disponible : <https://learn.microsoft.com/en-us/system-center/vmm/>
- [28] oVirt Project, « Scheduling Policies », oVirt Documentation. [En ligne]. Disponible : <https://www.ovirt.org/documentation/>

BIBLIOGRAPHIE

- [29] OpenStack Foundation, « Nova Scheduler », OpenStack Documentation. [En ligne]. Disponible : <https://docs.openstack.org/nova/latest/admin/scheduling.html>
- [30] Prometheus Authors, « Prometheus Documentation », Prometheus. [En ligne]. Disponible : <https://prometheus.io/docs/>
- [31] Red Hat, « Ansible Documentation », Ansible. [En ligne]. Disponible : <https://docs.ansible.com/>
- [32] Deciso B.V., « OPNsense Documentation », OPNsense. [En ligne]. Disponible : <https://docs.opnsense.org/>
- [33] D. Mills et al., « Network Time Protocol Version 4 : Protocol and Algorithms Specification », RFC 5905, IETF, juin 2010. [En ligne]. Disponible : <https://datatracker.ietf.org/doc/html/rfc5905>